

Requirements

Cheng Chen, Ph.D.

10/27/2026

Email: Cheng.chen@uah.edu

Office: OKT N133

Outline

- What are engineering requirements?
  - Definition, Examples, Evolution, Standards
- NASA Requirements Process
- Challenges of Requirements Definition
  - Flowdown and Allocation - Isoperformance
  - Validation and Verification
  - Writing good requirements
- What happens at the SRR?

What are engineering requirements?(1/2)

- Requirements describe the **necessary functions and features of the system** we are to conceive, design, implement and operate.
  - Performance
  - Schedule
  - Cost
  - Other Characteristics (e.g. lifecycle properties)

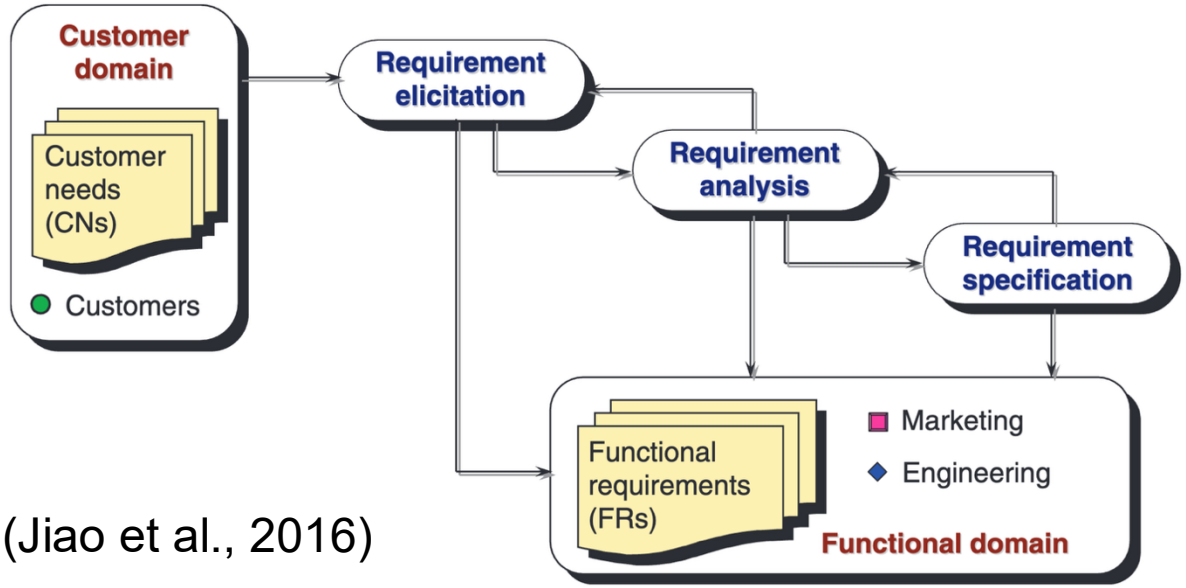
Requirement-Driven Design (RDD)
Vs
Value-driven design (VDD)

- Requirements are often organized hierarchically
  - At a high-level requirements focus on what should be achieved, not how to achieve it
  - Requirements are specified at every level, from the overall system to each hardware and software component.

- **Critically important to establish properly**
  - **Many of the cost overruns presented in Lecture 1 are caused by over-ambitious or missing requirements**

What are engineering requirements? (1/2)

- Requirements are all examples of possible reasons for establishing new design goals. (Beitz et al., 1996)
- Requirements define the needs of stakeholders such as users, customers, suppliers, developers, and manufacturers and how to satisfy those needs. (Hull et al., 2002)
  - "1.1.1 Headlamps shall be so made that they give adequate illumination without dazzle when emitting the passing beam, and good illumination when emitting the driving beam." (Morkos, 2012)
  - The [Actor] shall transition to [Target State] from the [Origin State] when [Control Flow Name]. (London, 2014)
- Requirement elicitation is an iterative process and incremental process



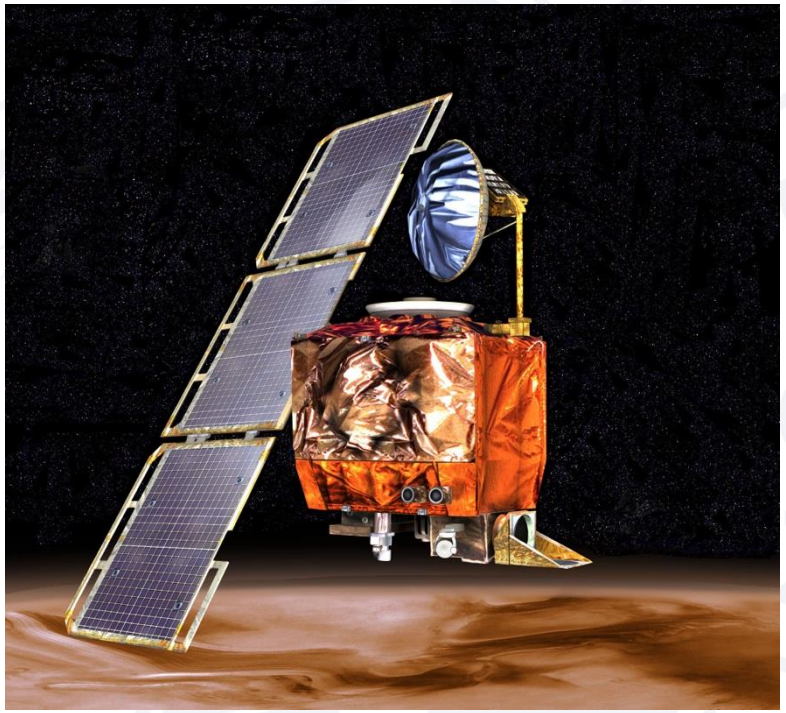
(Jiao et al., 2016)

Why requirement management matters?

Reduce cost	**Improve organizational project success and reduce costs** - 51% of projects fail and program dollars are wasted (PMI, 2014) - Reduce error cost escalation (Haskins 2004) e.g., Cost to fix an error: 1 unit in the requirement phase 3 - 8 units in the design phase 7 - 16 units in the manufacturing phase 21 – 78 units in the test phase 29 – 1500 unites in the operation phase
Reduce error	**Reduce human error and laborious operations in confirmation management** (Shishko & Aster, 1995; Saaksvuori, 2008) - Risk of formatting and editorial errors - Engineers expend 15–40% of their working hours doing routine searches of the information - Affects training, operations resource management, human factors, safety, etc.

Poor requirements example: MCO

- **Mars Climate Orbiter (MCO)** was launched by NASA on **December 11, 1998**
- Intended to study **Martian climate**, weather and surface changes and act as **communications relay back to Earth**
- **However, disintegrated during orbit insertion on Sept 23, 1999** → approach too close → **requirements not followed**
- **Units confusion problem**: Ground Software produced output in **non-SI units** (lbf-sec) instead of **SI units**: N-s
  - Calculation of total momentum produced by engine burns needed by GNC
- **Contract between NASA and Lockheed Martin did specify SI-units**
  - This requirement was flowed down to the Software Interface Specification (SIS), but not verified later and not implemented in the AMD



From the incident report:

*"Items that the mission assurance manager could have addressed for MCO included ensuring that the AMD file met **the requirements** of the SIS…"*

ftp://ftp.hq.nasa.gov/pub/pao/reports/1999/MCO_report.pdf

Good requirements example: DC-3

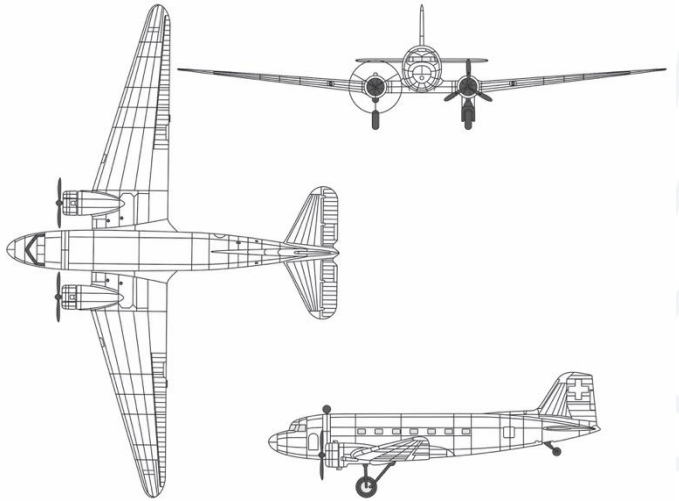
- Requirements based on desired improvements to DC-2

- Very simple
  - 3-page RfP(McDonnell Museum)
  - Marathon phone call between Smith and Douglas

- Key Requirements
  - Range: 1000 miles
  - Cruise Speed: 150 mph
  - Passengers: 20-30
    - Depending on configuration
  - Twin Engines
  - Rugged and Economical

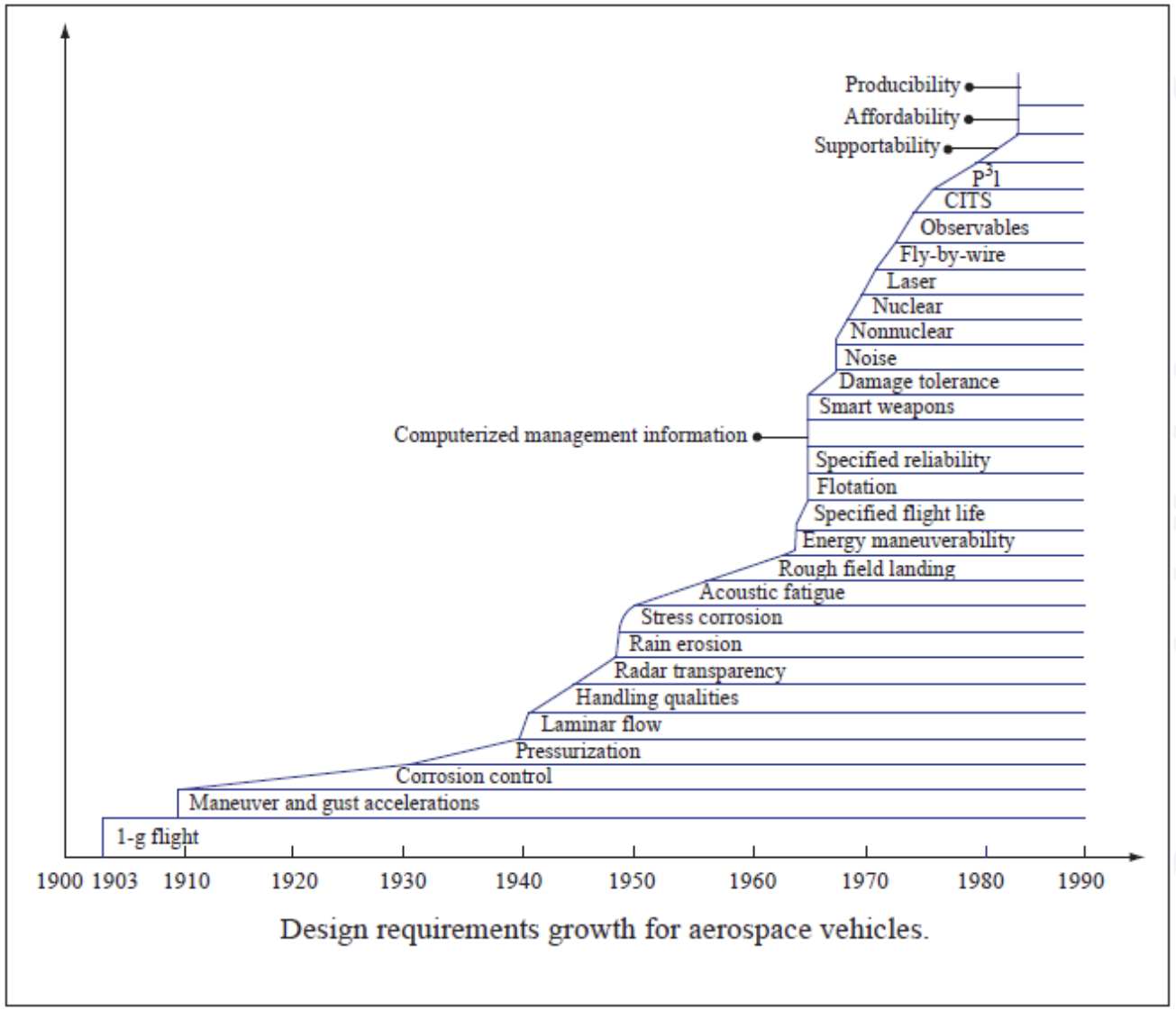
1st flight: 17 Dec 1935
Over 10,000 built





Requirements Explosion since 1970s

More and more requirements were added as systems grew in performance and complexity



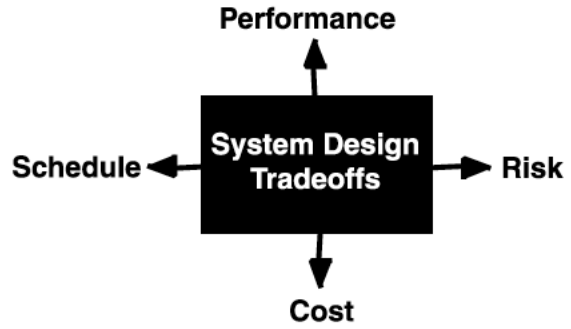
Design requirements growth for aerospace vehicles.

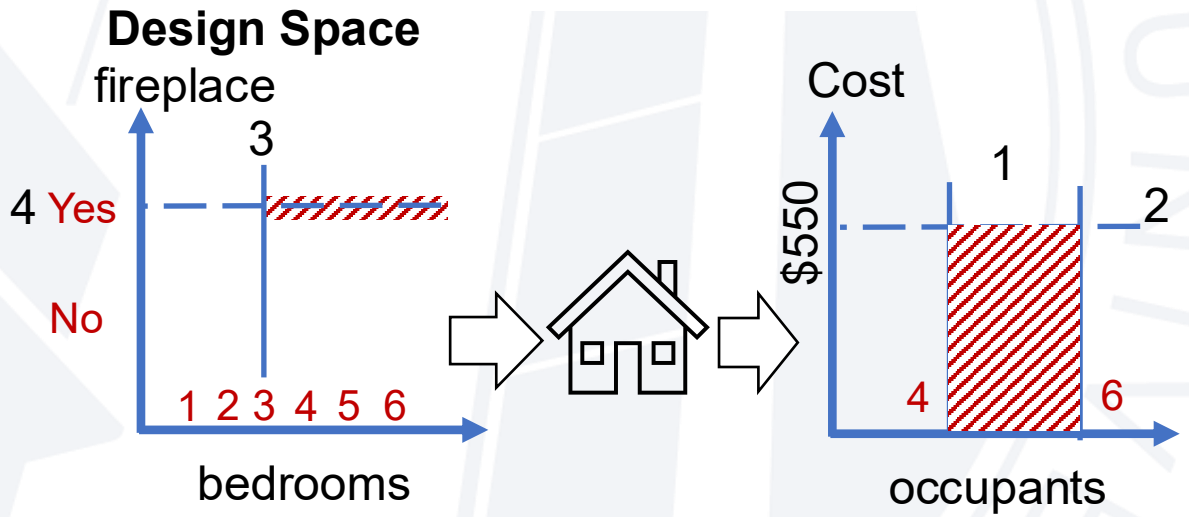
Requirements Standards

- NASA Systems Engineering Handbook:
  - NASA/SP-2007-6105
    - Section 4.2 (pp. 40-48) – Technical Requirements Definition
    - Section 6.2 (pp. 131-135) – Requirements Management
    - Appendix C (pp. 279-281) – How to write a good Requirement
    - Appendix D (pp. 282-283) – Requirements Verification Matrix
- International Council of Systems Engineering (INCOSE)
  - Systems Engineering Handbook, Version 3.1
  - Requirements Working Group
    - http://www.incose.org/ChaptersGroups/WorkingGroups/processes
- ISO/IEC 15288 (IEEE STD 15288-2008)
  - Systems and software engineering
  - System life cycle processes
    - 6.4.1 Stakeholder Requirements Definition Process

Requirements set constraints and goals in the design and objective space

- When designing systems we always have tradeoffs between performance, cost, schedule and risk
- **"Shall"** … Requirements help set constraints and define the boundaries of the design space and objective space
- **"Should"** … requirements set goals once "shall" requirements are satisfied
- Two main spaces:
  - **Design Space** – *the things we decide as engineers*
  - **Objective Space** – *the things our systems/products achieve and what our customers care about*





1. "The house shall sleep between 4 and 6 people"
2. "The total build cost should be less than $550K"
3. "The house shall have at least 3 bedrooms"
4. "The house should have a fireplace"

Concept Question

- Is there any difference in meaning between the words "*Requirements*" and "*Specifications*"?
  - No, they are essentially the same.
  - Yes, requirements are the input to the design process, while specifications are the output.
  - Yes, specifications include the requirements, but also contain other things such as blueprints etc…
  - I am not sure.

Requirements vs. Specifications

Requirements specify <u>what</u> the product or system shall/should do:

- **Functions** it shall perform
- How well it should perform these
- Degree of automation of the system (what operators must do)
- Compatibility with other devices etc…

Specification describe <u>how</u> system is built and works:

- The **Form** it is made of
  - Materials used in the system
  - Overall dimensions
- Schematics, Blueprints etc…
- User Interface

“Description”

• Large enough to accommodate big dishes
• 1,200 Watts of power to reheat food quickly
• One touch settings for different food types (rice, pizza, frozen meals) …
• Etc…

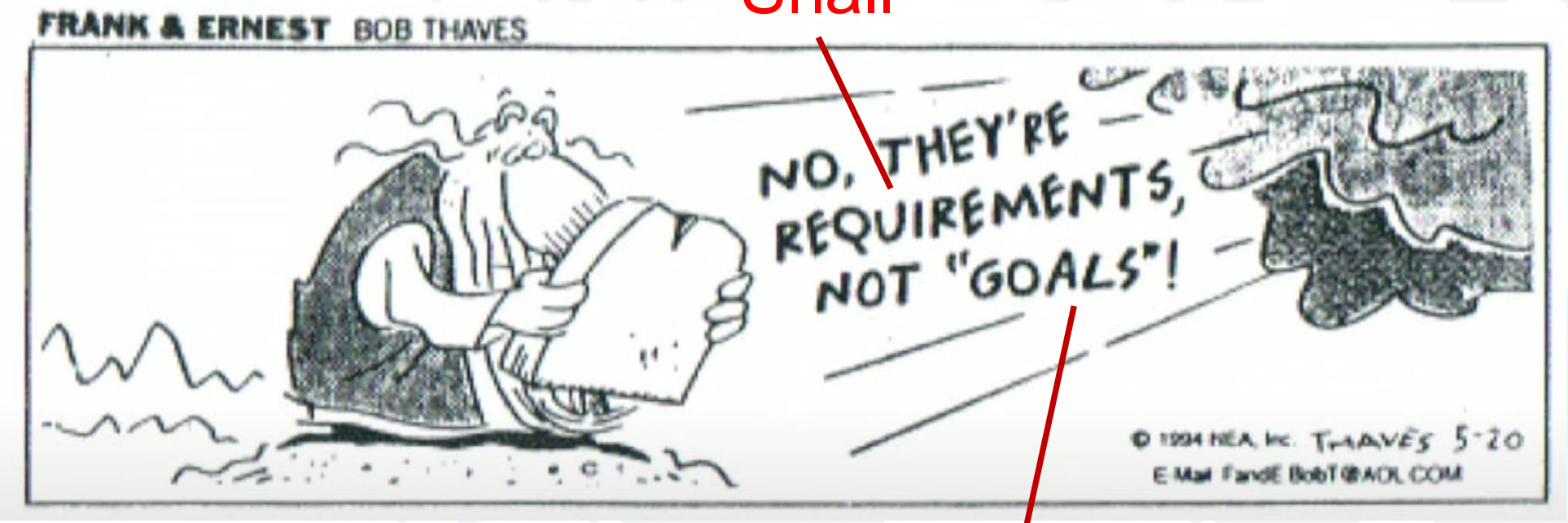


Kenmore Elite Countertop 2.2 cu ft Microwave Oven

“Specification”

• Stainless steel exterior
• Dimensions: 24” x 14” x 19”
• Weight: 45.5 lbs
• General warranty: 1 year
• Power cord: included
• Etc …

Questions?



Shall

Should

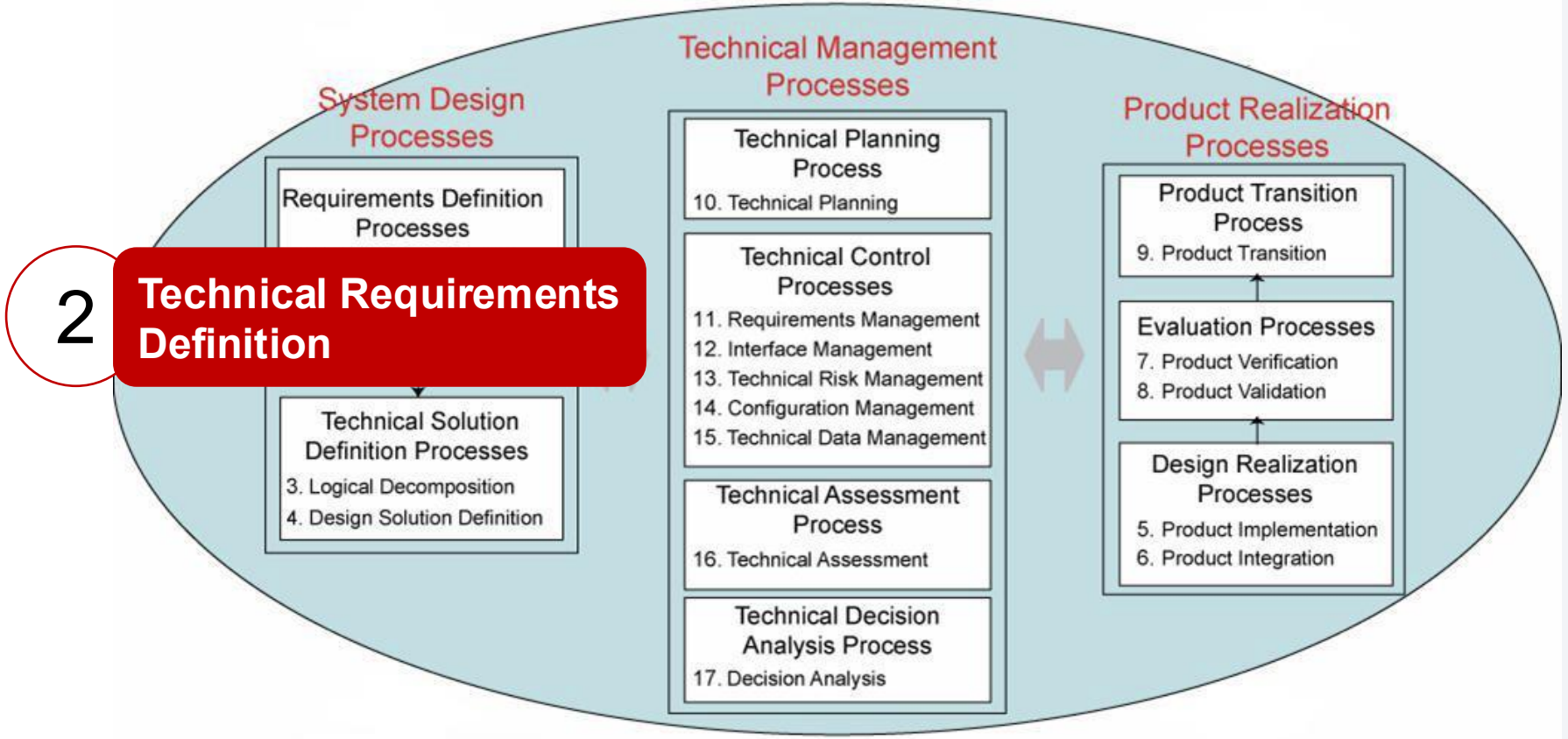
"Thou shalt not steal"

Email: Cheng.chen@uah.edu
Office: OKT N149

Outline

- What are engineering requirements?
  - Definition, Examples, Evolution, Standards
- NASA Requirements Process
- Challenges of Requirements Definition
  - Flowdown and Allocation - Isoperformance
  - Validation and Verification
  - Writing good requirements
- What happens at the SRR?

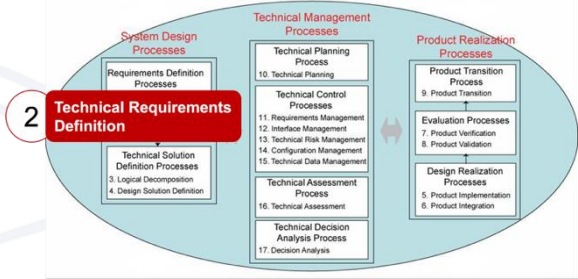
Technical Requirements Definition Process



SE Engine

Requirement 16 (Section 3.2.2.1) "The Center Directors or designees shall establish and maintain a process, to include activities, requirements, guidelines, and documentation, for definition of the technical requirements from the set of agreed upon stakeholder expectations for the applicable WBS model."

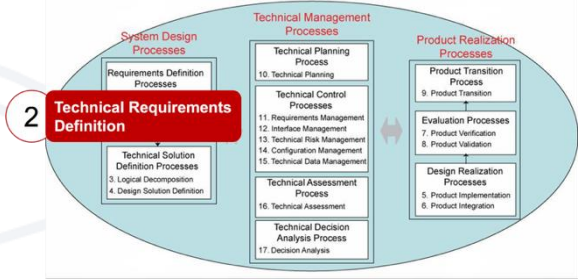
Purpose of Technical Requirements Definition



- The Technical Requirements Definition Process
  - Is used to transform the baselined stakeholder expectations (input) into unique, quantitative, and measurable technical requirements (output)

- Requirements
  - Come in many flavors
  - Should be expressed as well-written "shall" statements that can be used for defining a design solution

Purpose of Technical Requirements Definition

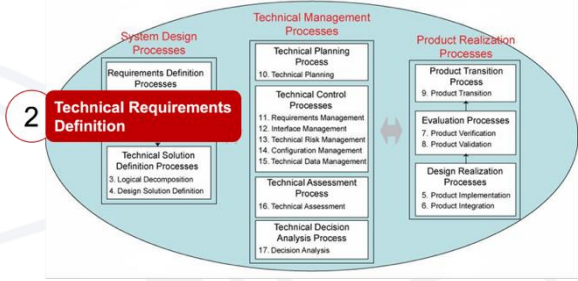


- **Establishes the basis for agreement between the stakeholders and the developers on what the product is to do**

- **Reduces the development effort because less rework is required to address poorly written, missing, and misunderstood requirements.**
  - Forces the relevant stakeholders to consider rigorously all of the requirements before design begins
  - Careful review can reveal omissions, misunderstandings, and inconsistencies early in the development cycle

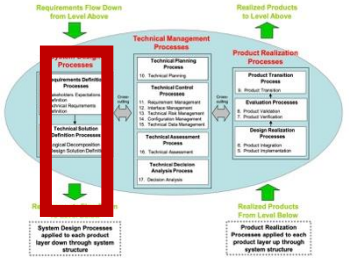
- **Provides a basis for estimating costs and schedules**
  - The description of the product to be developed as given in the requirements is a realistic basis for estimating project costs and can be used to evaluate bids or price estimates

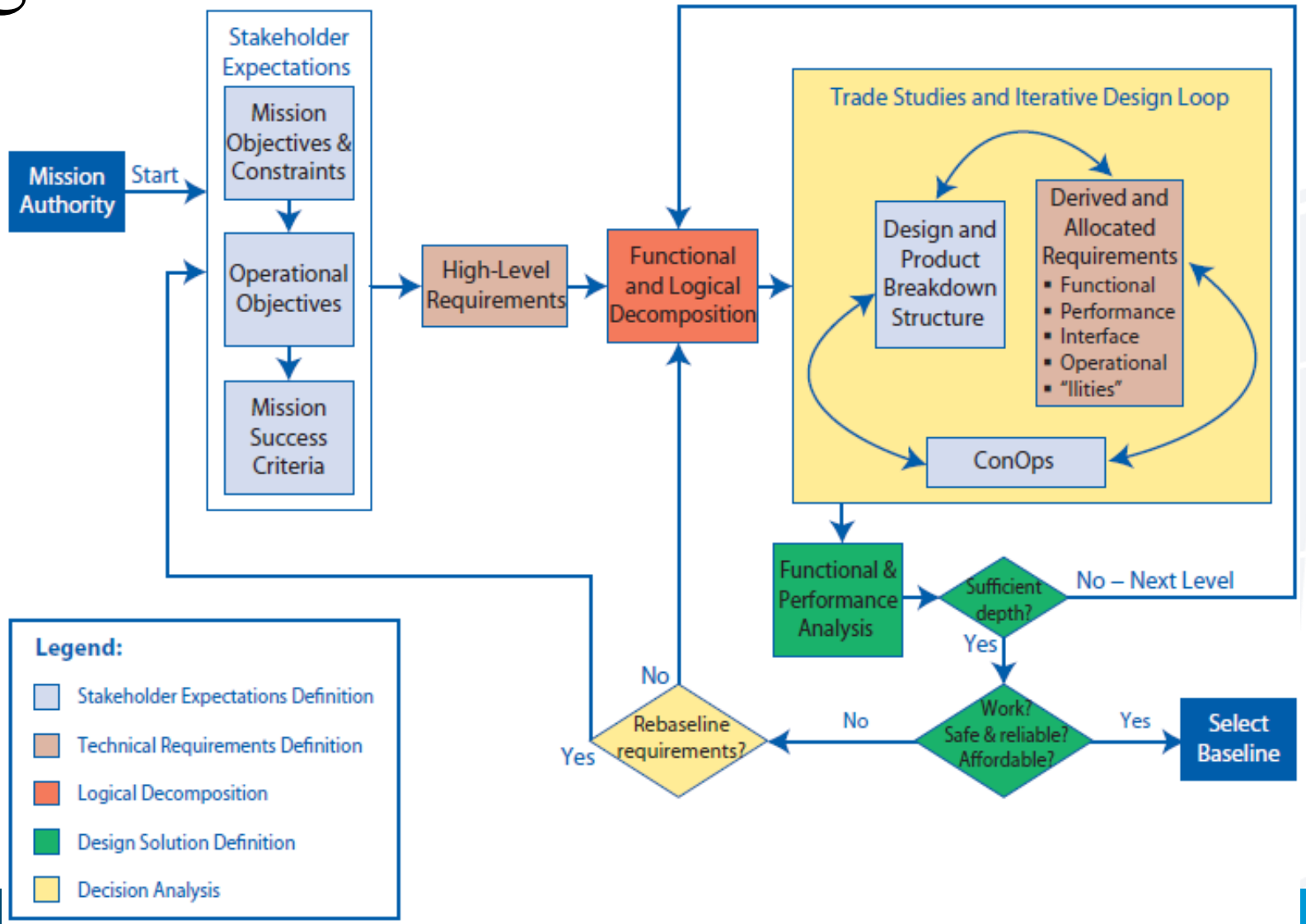
Importance of Technical Requirements Development (2/2)



- **Provides a baseline for verification**
  - Organizations can develop their validation and verification plans much more productively from a good requirements document.
  - The requirements document provides a baseline against which compliance can be measured.
  - The requirements are also used to provide the stakeholders with a basis for acceptance of the system.
- **Facilitates transfer of the product to new users.**
- **Serve as a basis for later enhancement or alteration of the finished product.**

Interrelationships Among the System Design Processes





What are the different types of requirements?

- **Functional Requirements define what functions need to be done to accomplish the mission objectives**
- Example: The Thrust Vector Controller (TVC) shall provide vehicle control about the pitch and yaw axes.
  - This statement describes a high-level function that the TVC must perform.
  - Statement has form of Actor – Action Verb – object acted on

- **Performance Requirements** define how well the system needs to perform the functions
- Example: The TVC shall gimbal the engine a maximum of 9 degrees, +/- 0.1 degree

- **Constraints are requirements that cannot be traded off with respect to cost, schedule or performance**
- Example: The TVC shall weigh less than 120 lbs.

- **Interface Requirements**
- Example: The TVC shall interface with the J-2X per conditions specified in the CxP 72262 Ares I US J-2X Interface Control Document, Section 3.4.3.

- **Environmental requirements**
- Example: The TVC shall use the vibroacoustic and shock [loads] defined in CxP 72169, Ares I Systems Vibroacoustic and Shock Environments Data Book in all design, analysis and testing activities.

- **Other -illities requirement types described in the SE Handbook include: human factors, reliability requirements, and safety requirements.**

What are the attributes of acceptable requirements?

- A complete sentence with a **single** “shall” per numbered statement

- Characteristics for **each** Requirement Statement:
  - **Clear and consistent** – readily understandable
  - **Correct** – does not contain error of fact
  - **Feasible** – can be satisfied within natural physical laws, state of the art technologies, and other project constraints
  - **Flexibility** – Not stated as to how it is to be satisfied
  - **Without ambiguity** – only one interpretation makes sense
  - **Singular** – One actor-verb-object requirement
  - **Verify** – can be proved at the level of the architecture applicable

- Characteristics for **pairs and sets** of Requirement Statements:
  - **Absence of redundancy** – each requirement specified only once
  - **Consistency** – terms used are consistent
  - **Completeness** – usable to form a set of “design-to” requirements
  - **Absence of conflicts** – not in conflict with other requirements or itself

Non-Functional Classifications

- **Non-functional requirements** may affect the overall architecture of a system rather than the individual components.
  - For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.
  - A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.
- **Product requirements**
  - Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.
  - Example: The Mentcare system shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Downtime within normal working hours shall not exceed five seconds in any one day.
- **Organisational requirements**
  - Requirements which are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.
  - Example: Users of the Mentcare system shall authenticate themselves using their health authority identity card.
- **External requirements**
  - Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.
  - Example: The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

Practice Exercise: Writing Effective Requirements

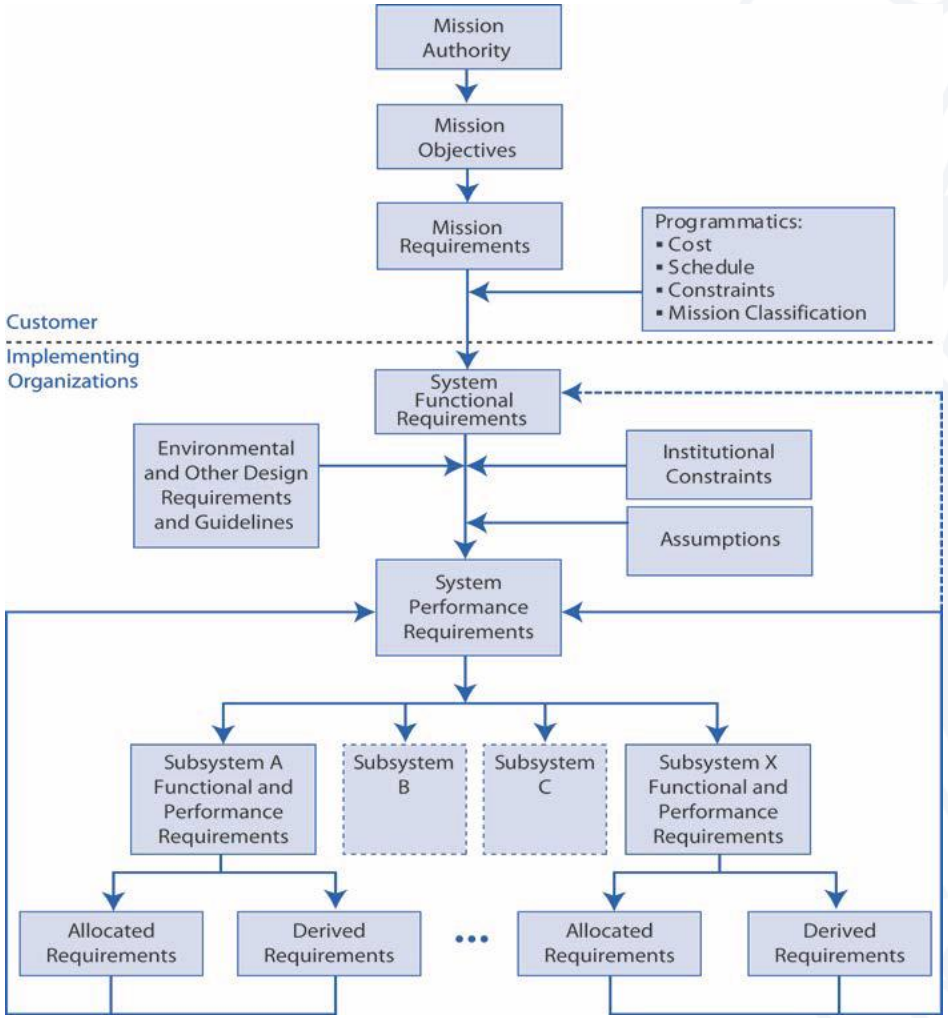
- **Turn to your partner exercise**
- Together write a good requirement that was (possibly) used in the development of one of the following solutions
  - ❑ A – RESCUE! Disposable Fly Trap
  - ❑ B – New BMW i3 electric car
  - ❑ C – Vending machine







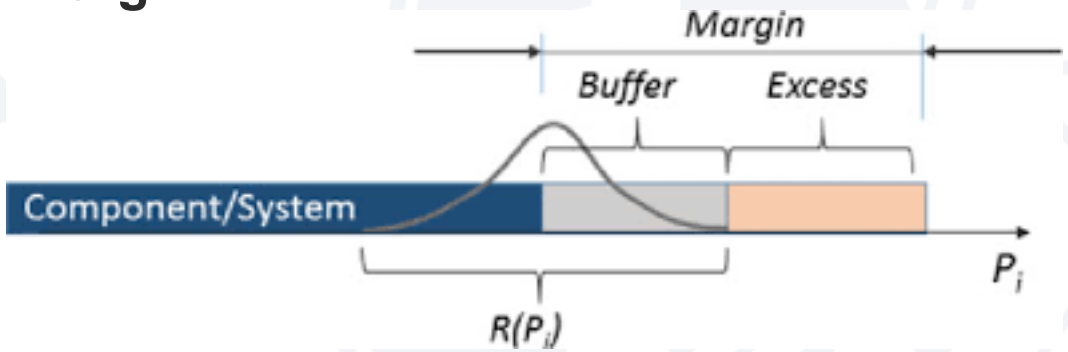
Requirements Decomposition, Allocation and Validation



- Requirements are decomposed in a hierarchical structure starting with the highest level requirements.
- These high-level requirements are decomposed into functional and performance requirements and allocated across the system.
- These are then further decomposed and allocated among the elements and subsystems. This complete set of design-to requirements is achieved.
- At each level of decomposition (system, subsystem, component, etc.), the total set of derived requirements must be validated against the stakeholder expectations or higher level parent requirements.

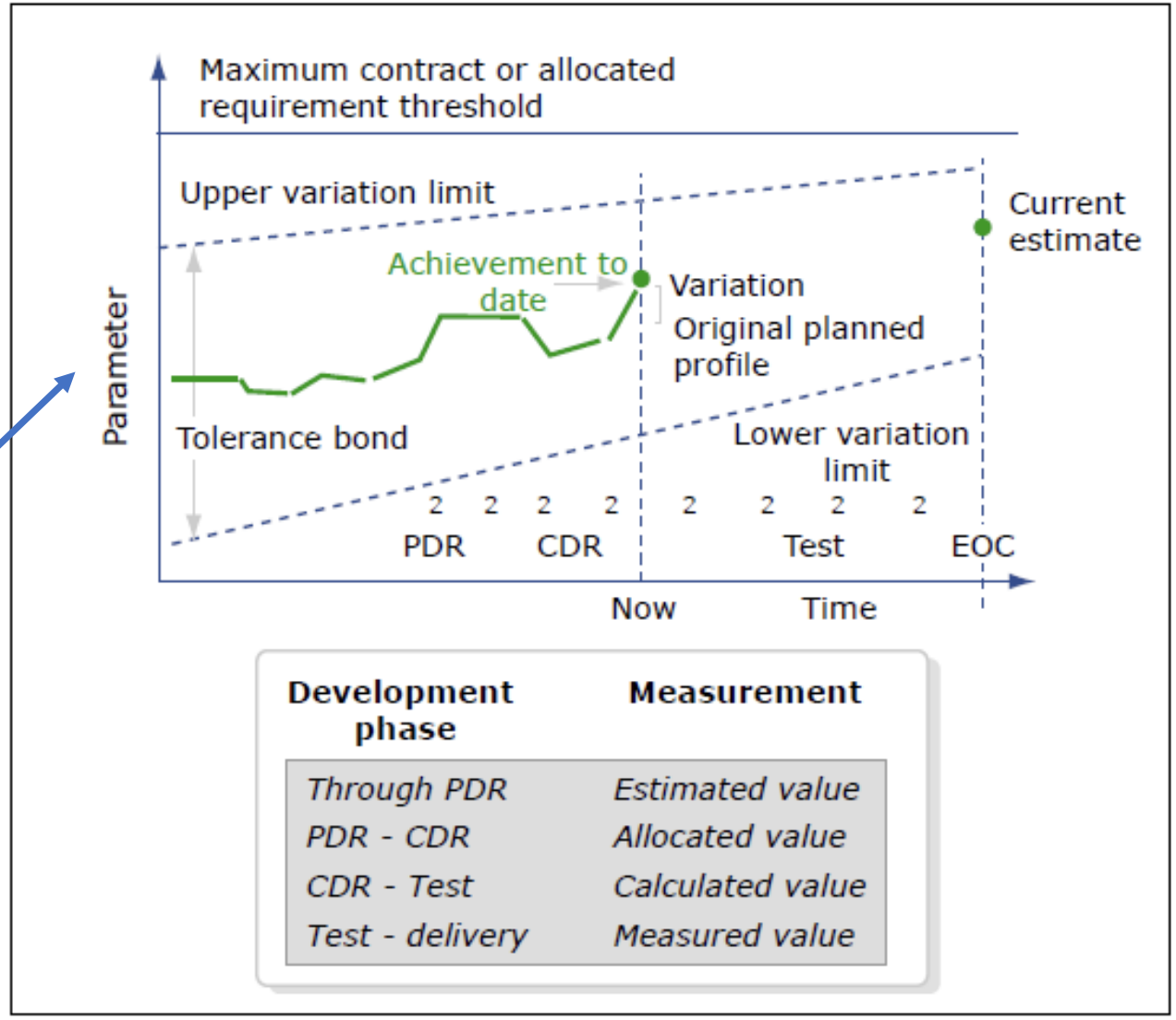
Requirements Margins Management

- **Due to uncertainty during early design, must use appropriate requirements margins (e.g. for mass, power, memory etc…)**
- **Margins are *reserves*, that are not allocated to subsystems but are controlled by project managers**
- **For example mass growth has been experienced in almost all vehicle development programs**
  - Mass growth can typically range from 10-60%
  - Depends on *novelty* of the project
- **Typical Guidelines: Establish SRR Mass + 30% margin**
  - pre-PDR keep 20% mass margin
  - pre-CDR keep 10% mass margin
  - pre-IOC keep 5% mass margin



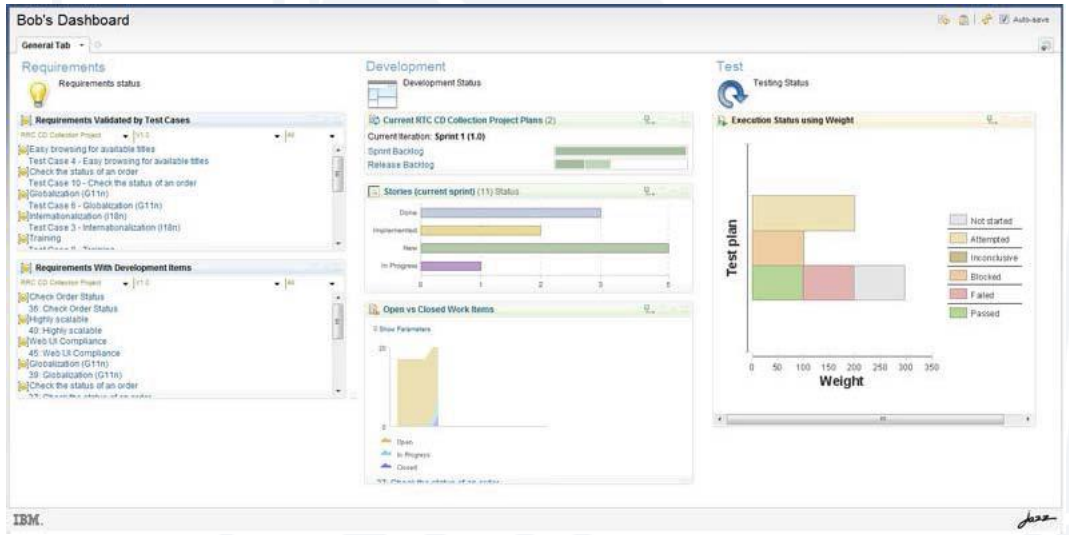
Monitoring a Requirement

Technical performance Measure (TPM)



Requirements Capture: Documents vs. Database

- **Where / how are requirements captured?**
  - Low cost “easy” solution: Create a document (e.g. in MS Word or Excel) to capture and revise the requirements. Use hyperlinks to link requirements.
  - This works okay for smaller projects with dozens or a few hundred requirements (e.g. about 3 levels of decomposition - (7+/-2)^3 = 125 - 729
  - For larger projects with >1,000 requirements need to use a relational database
  - **Commercial Tools**, e.g. DOORS are available (but can be expensive)

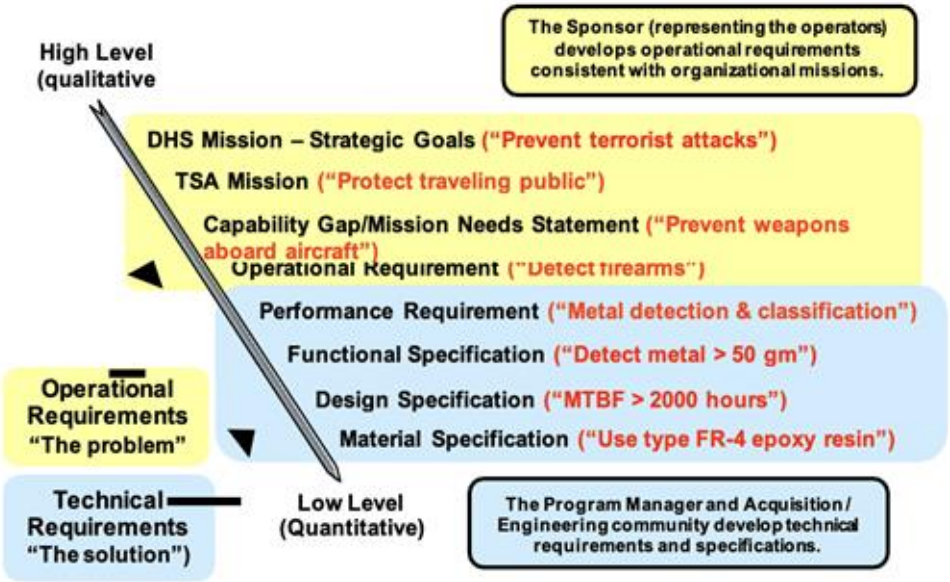


Requirements

Example of hierarchical requirements with links

- R1.0 The system shall fit into a volume not exceeding 1.0 m^3
  - R1.1 The system width shall be between 0.5m and 1.0m
  - R1.2 The system height shall be between 0.5m and 1.0m
  - R1.3 The system depth shall be between 0.5m and 1.0m
- R2.0 The system shall be made entirely from Aluminum 6060 alloy
  - R2.1 The system shall not contain any internal voids or cavities
- R3.0 The shape of the system must be a cube
  - R3.1 The angles between sides shall be 90 deg +/-1 deg
  - R3.2 etc…

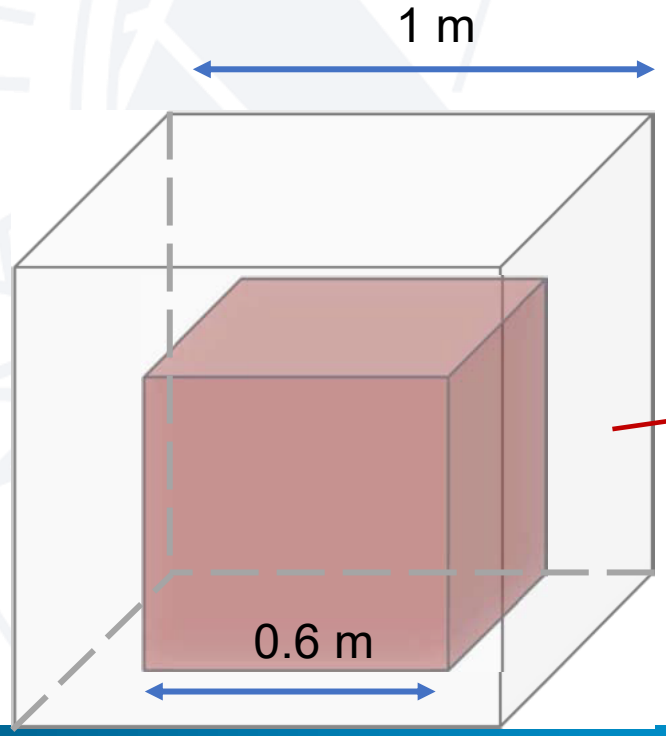
Homeland Security: TSA example



Derived requirement

- R4.0 The mass of the system shall not exceed 2,700 kg
  - R4.1 The center of mass of the system must be located at least 0.25 meters from the edge of its volumetric envelope
  - R4.2 The mass of the system must be verified using a Mettler-Toledo XYZ Bench Scale

What kind of object would satisfy these requirements?



Al-6060 cube with 0.6 m side length
V= 0.216 m3
mass=583.2 kg

Outline

- What are engineering requirements?
  - Definition, Examples, Evolution, Standards
- NASA Requirements Process
- Challenges of Requirements Definition
  - Flowdown and Allocation - Isoperformance
  - Validation and Verification
  - Writing good requirements
- What happens at the SRR?

Requirements Allocation

- Decompose system requirements into lower levels of design.
  - Define all the lower level functions which must be performed to satisfy the requirement
  - Create architecture of sub-components to provide those functions

- Allocate a level of performance to each lower level function
  - Specify interface requirements to other sub-systems

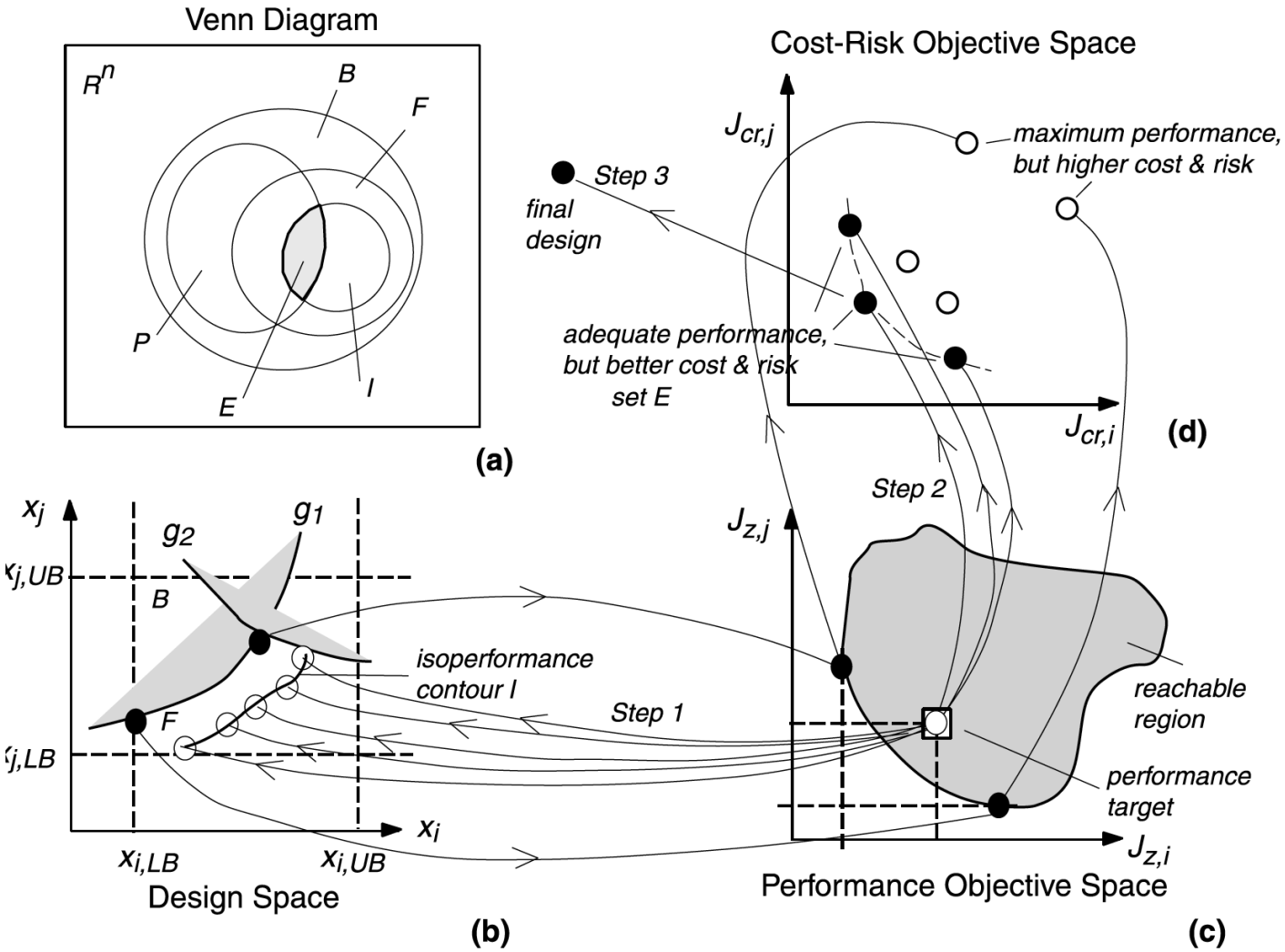
- Closure - Ensure that satisfaction of the set of requirements at the lower level will guarantee satisfaction of the higher level requirement.

Isoperformance Methodology

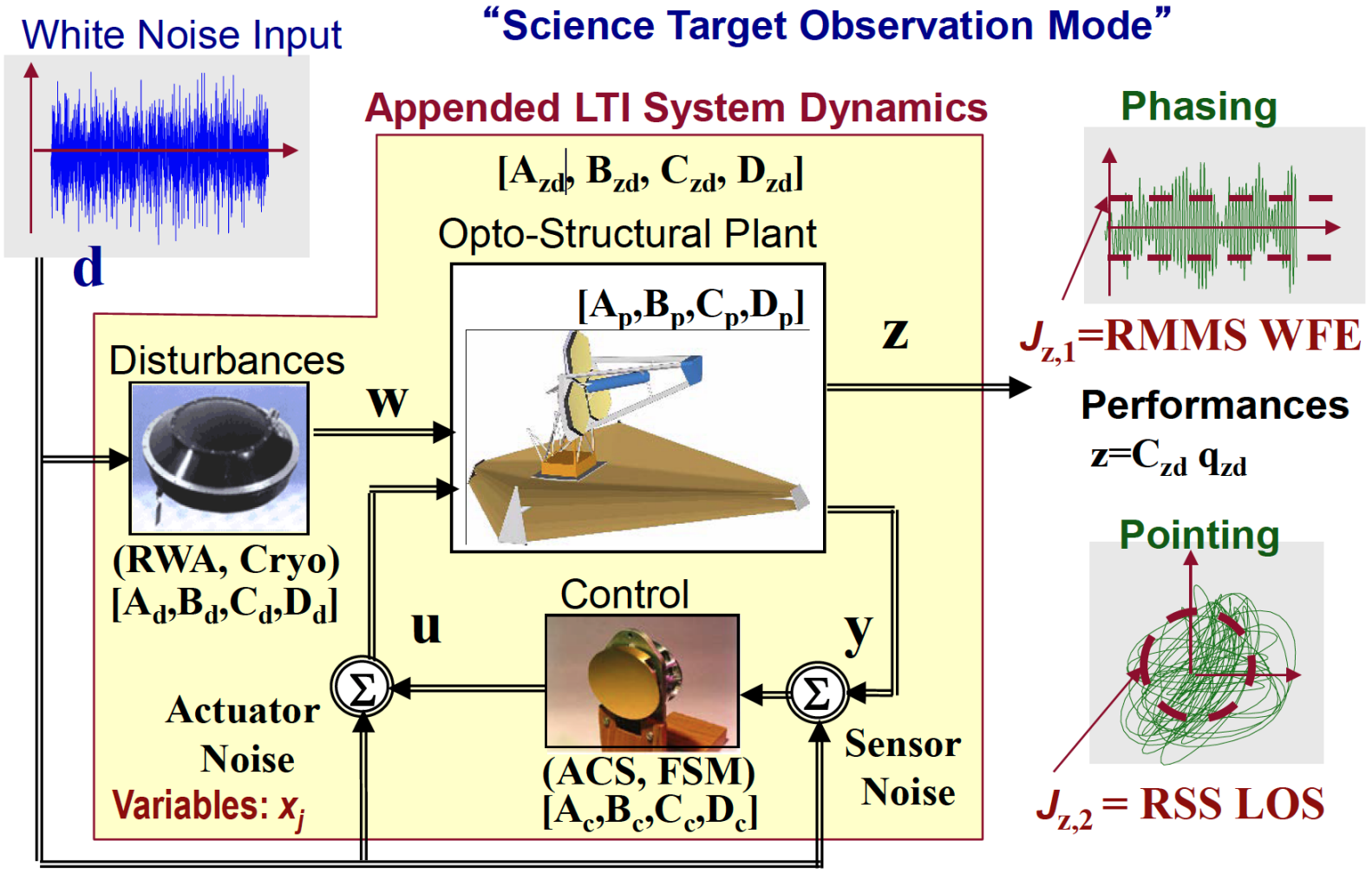
- "Isoperformance" is a methodology that helps to better do requirements flow-down and allocation
- Starts with a vector of desired performance targets/requirements
- Runs simulation models of systems to determine if
  - A) the vector of targets is feasible
  - B) and if so, produce a set of non-unique feasible combinations of designs to establish correct sub-system requirements

Reading [2b]: de Weck, O.L. and Jones M. B., "Isoperformance: Analysis and Design of Complex Systems with Desired Outcomes", Systems Engineering, 9 (1), 45-61, January 2006 (17 pages)

Requirements and Isoperformance



Isoperformance for Requirements Allocation



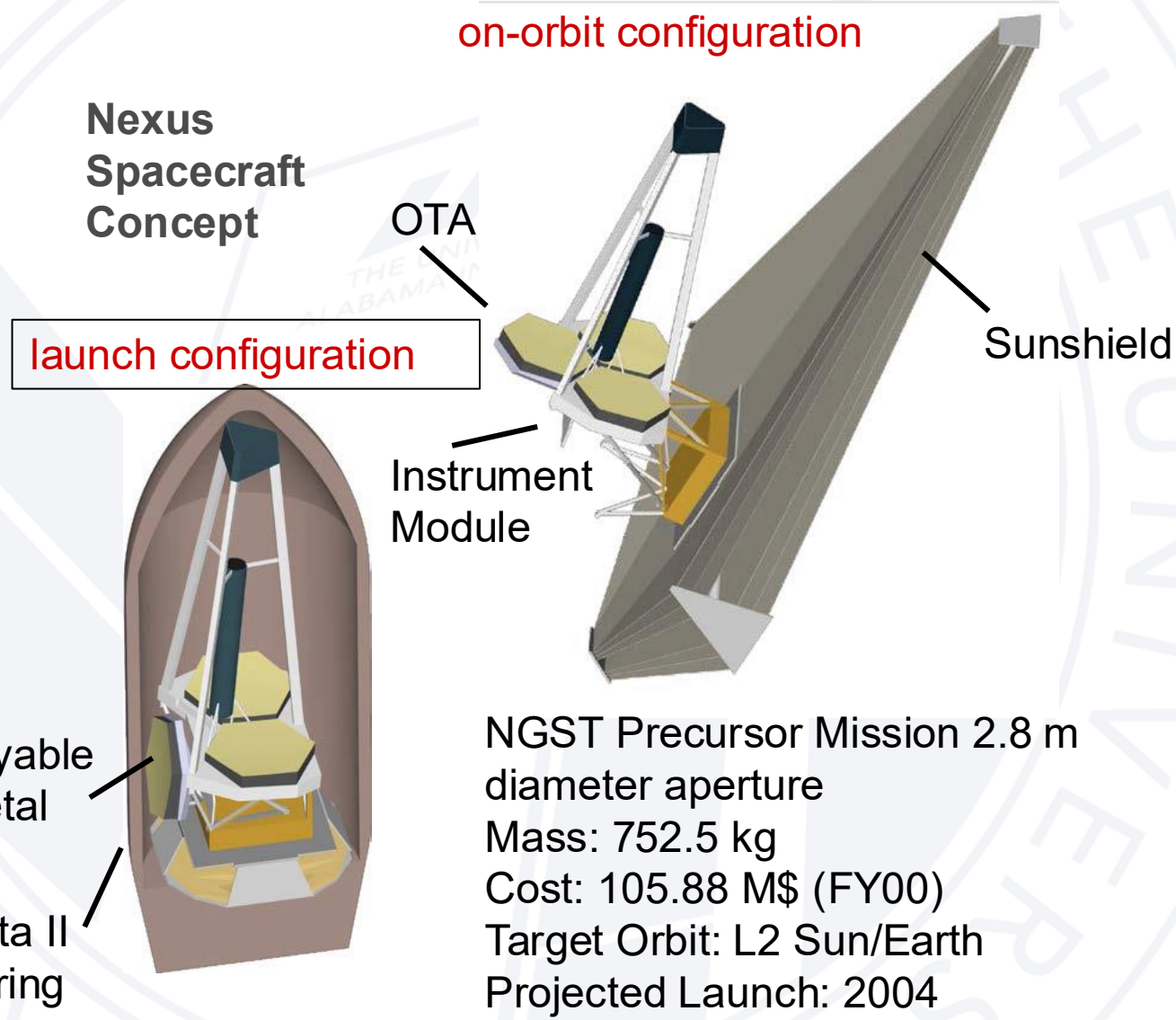
Example: Nexus Case Study

Purpose of this case study:

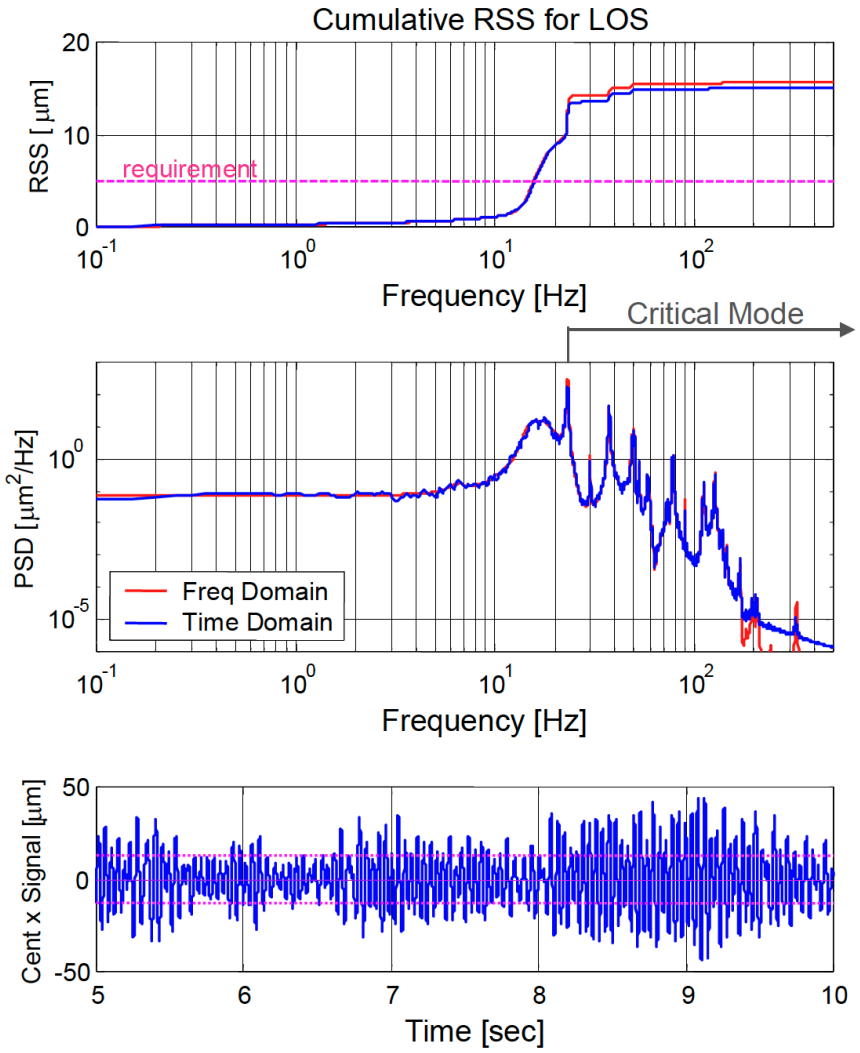
Demonstrate the usefulness of Isoperformance on a realistic conceptual design model of a high-performance spacecraft

The following results are shown:

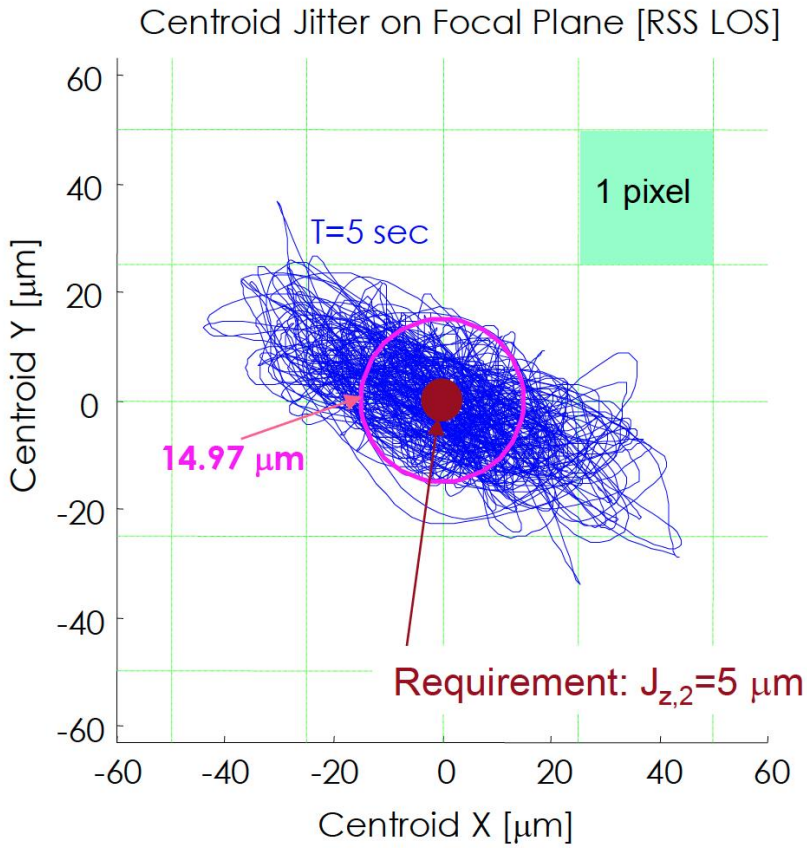
- Integrated Modeling
- Nexus Block Diagram
- Baseline Performance Assessment
- Sensitivity Analysis
- Isoperformance Analysis (2)
- Multiobjective Optimization
- Error Budgeting



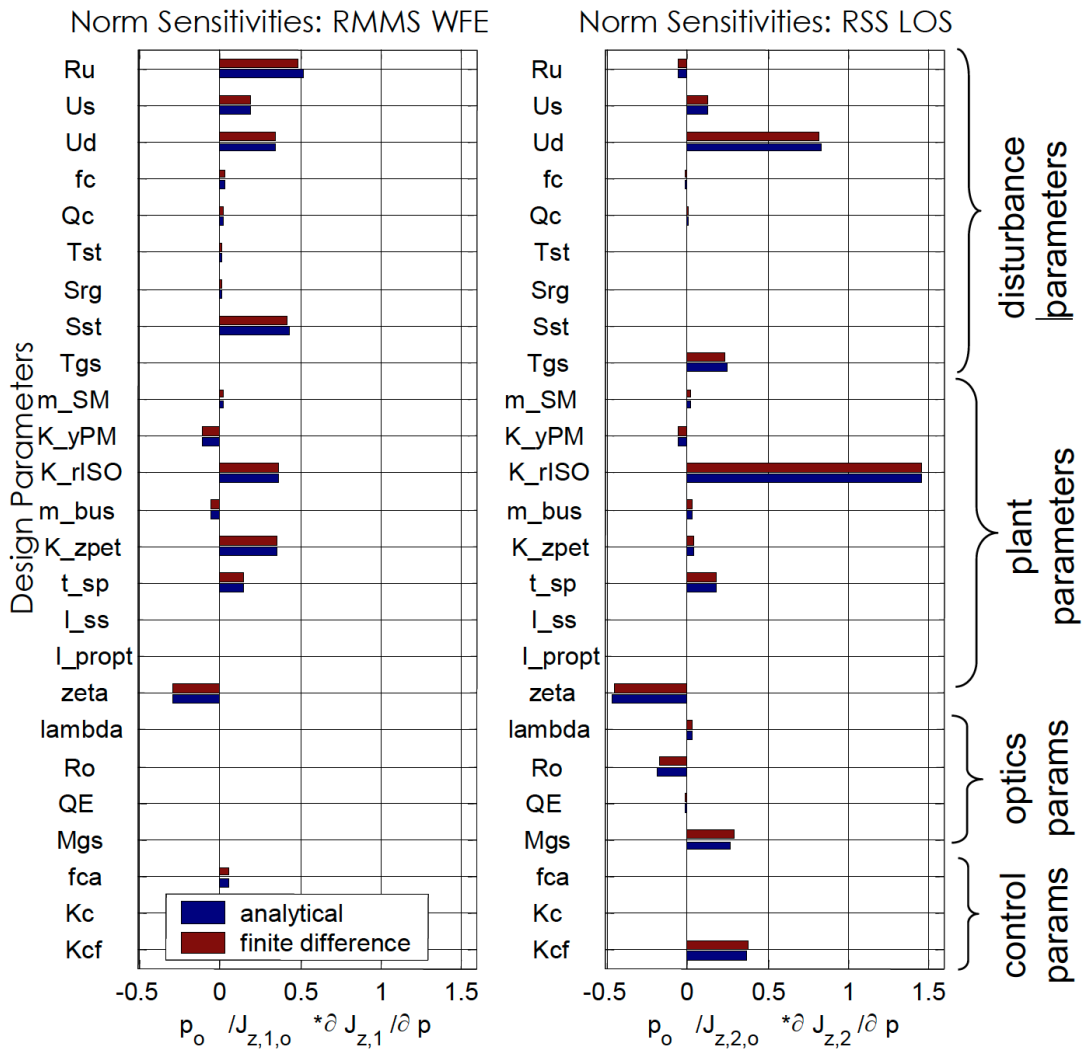
Initial Performance Assessment J_z(p^o)



Results	Lyap/Freq	Time	
$J_{z,1}$ (RMMS WFE)	**25.61**	**19.51**	[nm]
$J_{z,2}$ (RSS LOS)	**15.51**	**14.97**	[μm]



Nexus Sensitivity Analysis



Graphical Representation of Jacobian evaluated at design po, normalized for comparison.

$$\overline{\nabla J}_z = \frac{p_o}{J_{z,o}} \begin{bmatrix} \frac{\partial J_{z,1}}{\partial R_u} & \frac{\partial J_{z,2}}{\partial R_u} \\ \cdots & \cdots \\ \frac{\partial J_{z,1}}{\partial K_{cf}} & \frac{\partial J_{z,2}}{\partial K_{cf}} \end{bmatrix}$$

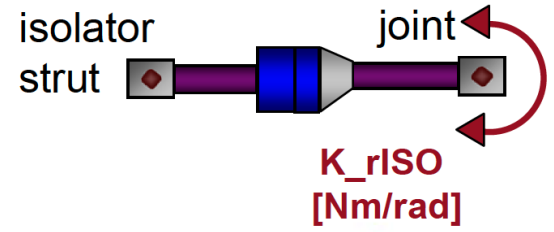
RMMS WFE most sensitive to:

- Ru - upper op wheel speed [RPM]
- Sst - star track noise 1σ [asec]
- K_rISO - isolator joint stiffness [Nm/rad]
- K_zpet - deploy petal stiffness [N/m]

RSS LOS most sensitive to:

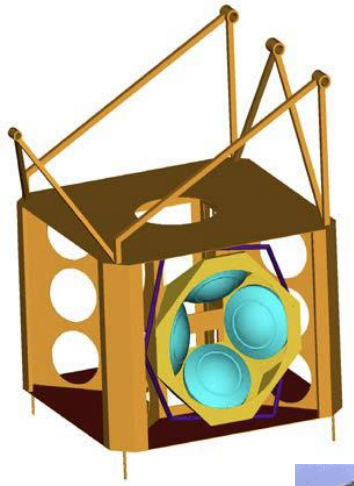
- Ud - dynamic wheel imbalance [gcm2]
- K_rISO - isolator joint stiffness [Nm/rad]
- zeta - proportional damping ratio [-]
- Mgs - guide star magnitude [mag]
- Kcf - FSM controller gain [-]

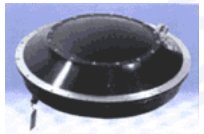
2D-Isoperformance Analysis



K_rISO
[Nm/rad]

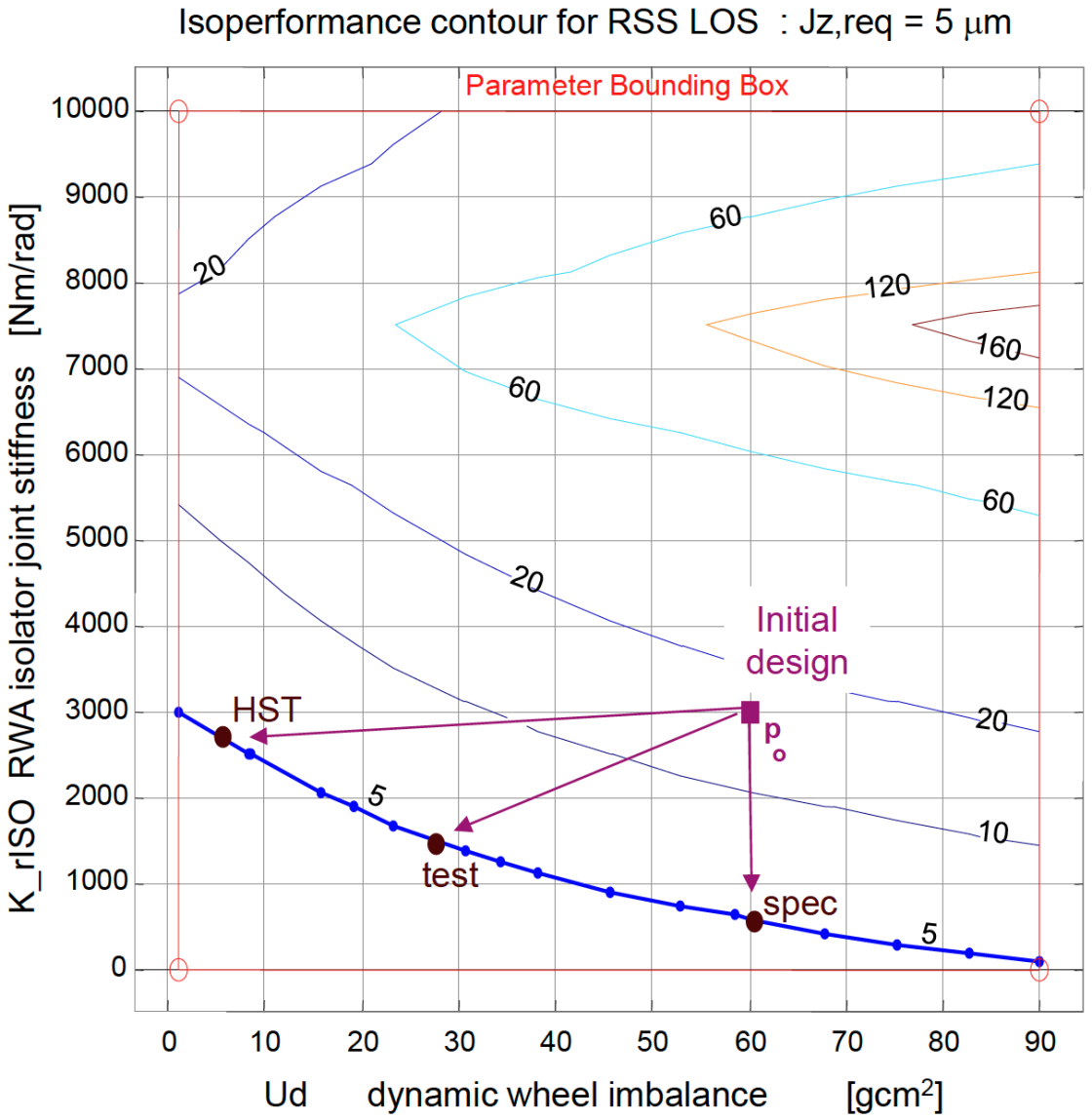
CAD
Model





E-wheel

Ud=mrd
[gcm^2]



Concept Question

- **A balloon shall lift a payload of 1,000 kilograms, including its own mass**. It can use either helium (ρ_*He*=0.2 kg/m3) or hydrogen (ρ_*H*=0.1 kg/m3) as a lift gas. The standard density of air is ρ_*Air*=1.3 kg/m3.

- *r_b*= radius of balloon, *m_b* = mass of balloon
- Which of the following requirements is *infeasible*?

  A – The balloon shall have a radius of 6.1 meters. The balloon shall use 99.9% pure helium as a lift gas.

  B – The balloon shall have a radius of 5.9 meters. The balloon shall use 99.9% pure hydrogen as a lift gas.

  C – The balloon shall have a radius of 5.9 meters. The balloon shall use 99.9% helium as a lift gas.

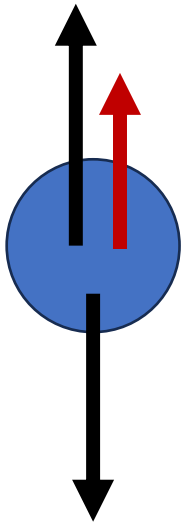
  D – The balloon shall have a radius of 6.1 meters. The balloon shall use 99.9% hydrogen as a lift gas.

  E – All these requirements are okay.

  F – None of these requirements are feasible.

Concept Question - Solution

$$F_L = \rho_{air} V_b g$$



$$F_W = \left(m_b + \rho_{He} V_b\right) g$$

For lift need FR>=0

$$V_b \geq \frac{m_b}{\rho_{air} - \rho_{gas}}$$

$$\rho_{He} = 0.2 \ [\text{kg/m}^3]$$

$$\rho_H = 0.1 \ [\text{kg/m}^3]$$

$$\rho_{air} = 1.3 \ [\text{kg/m}^3]$$

$$g = 10 \ [\text{m/s}^2]$$

$$V_b = \frac{4}{3} \pi r_b^3$$

Case	Gas	R_b	F_R>0	Feasible
A	He	6.1 m	Yes	Yes
B	H2	5.9 m	Yes	Yes
C	**He**	**5.9 m**	**No**	**No**
D	H2	6.1 m	Yes	Yes

Common problems with requirements

- Writing implementations (“How”) instead of requirements (“What”)
  - Forces the design
  - Implies the requirement is covered
- Using incorrect terms
  - Avoid “support”, “but not limited to”, “etc”, “and/or”
- Using incorrect sentence structure or bad grammar

Common problems continued

- Writing unverifiable requirements
  - E.g., minimize, maximize, rapid, user-friendly, easy, sufficient, adequate, quick
- Missing requirements
  - Requirement drivers include

Functional	Performance	Interface
Environment	Facility	Transportation
Training	Personnel	Reliability
Maintainability	Operability	Safety

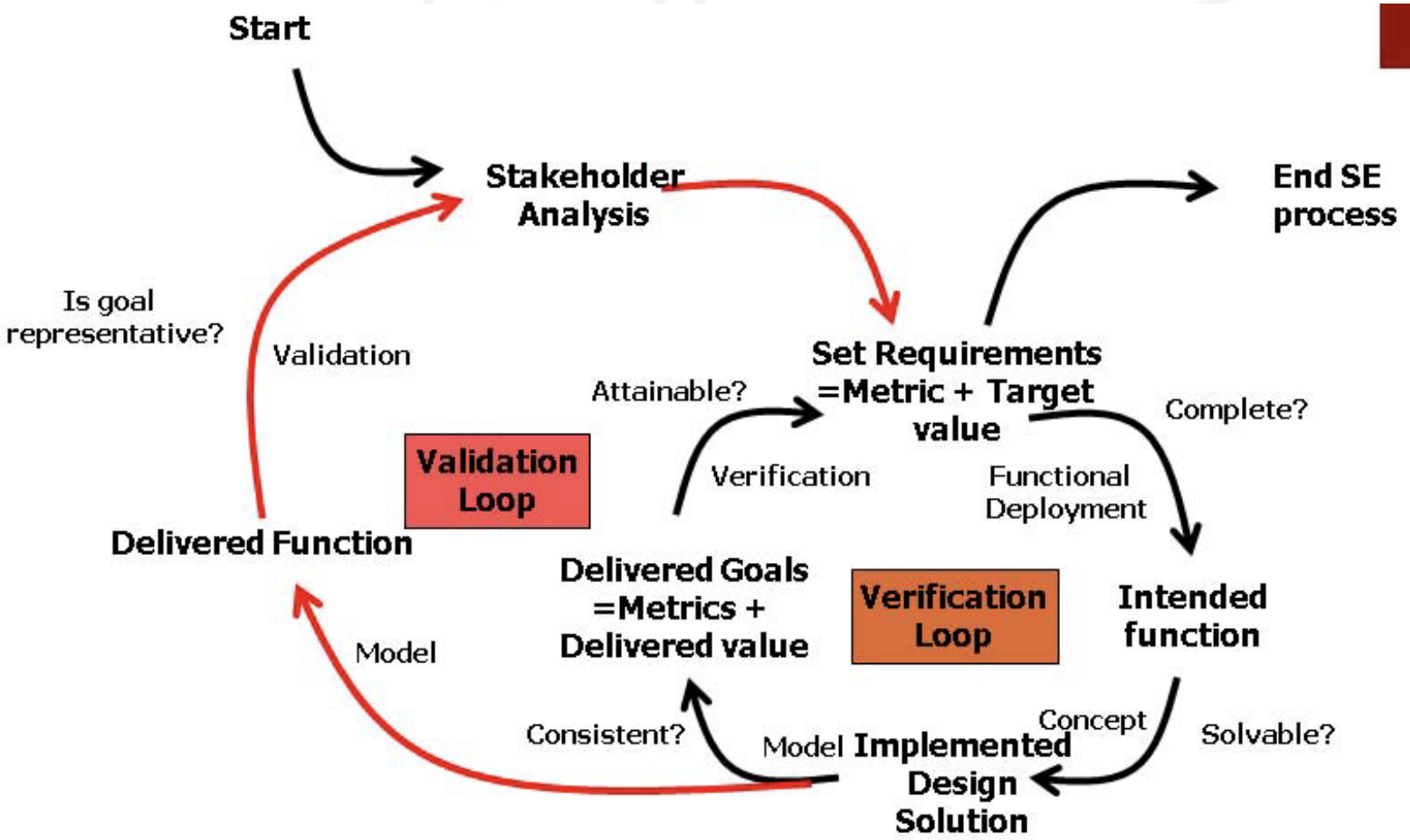
- Requirements only written for “first use”
- Over-specifying

Verification

- Every requirement must be verified to ensure that the proposed design actually satisfies the requirement by Examination,
  - Test,
  - Demonstration, or
  - Analysis

- Requirements documentation specifies the development phase and method of verification

Verification and Validation Loops



Outline

- What are engineering requirements?
  - Definition, Examples, Evolution, Standards
- NASA Requirements Process
- Challenges of Requirements Definition
  - Flowdown and Allocation - Isoperformance
  - Validation and Verification
  - Writing good requirements
- What happens at the SRR?

System Requirements Review (SRR)

- SRR is primarily a human / social peer-review process
- Main goal of SRR: **Vet the requirements as written**. Find any missing, misstated, redundant or otherwise unsatisfactory requirements.



NASA JPL

What happens at SRR ?

- **SRR** typically occurs rather early in a program, during Phase A (Concept and Technology Development) and before Phase B (Preliminary Design)
- Typically need to have the top two levels of requirements (L0 = mission requirement / CONOPS), that is L1 (major systems) and L2 (major subsystems) clearly defined
- Requirements need to be written down, validated and accessible to everyone on the team (document or database)
- Requirements are put under formal configuration management (=version control) at this point. Adding, deleting or modifying requirements requires management approval. Systems Engineers are very involved.

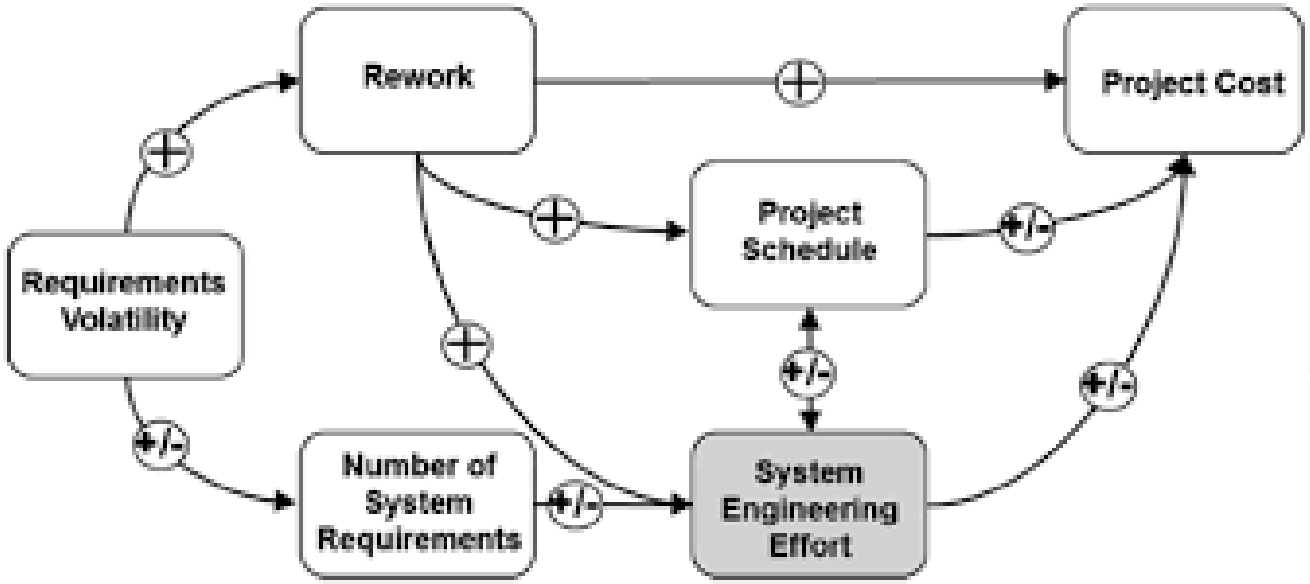
Table 6.7-1 Program Technical Reviews

Review	Purpose
Program/ System Requirements Review	The P/SRR examines the functional and performance requirements defined for the program (and its constituent projects) and ensures that the requirements and the selected concept will satisfy the program and higher level requirements. It is an internal review. Rough order of magnitude budgets and schedules are presented.
Program/ System Definition Review	The P/SDR examines the proposed program architecture and the flowdown to the functional elements of the system.

NASA SE Handbook, p.170

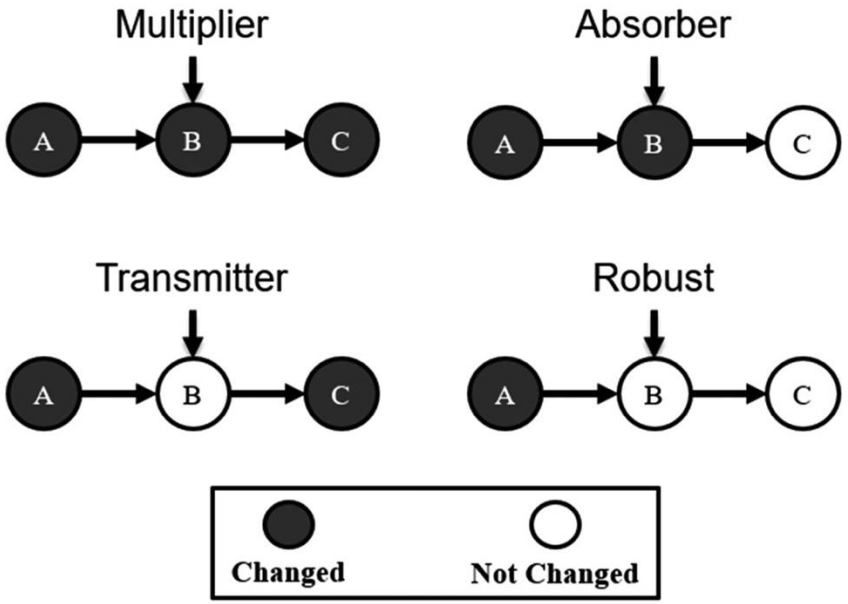
Requirements Volatility

- **Requirements Volatility** is the degree to which requirements continue to change during a project, even post-SRR
  - % of addition, removal, modification of requirements per time period
  - Variety of causes of requirements volatility (see below), generally undesirable



Reading [2a]: Peña, M. and Valerdi, R. (2015), Characterizing the Impact of Requirements Volatility on Systems Engineering Effort. *Systems Engineering*, 18: 59–70. doi: 10.1111/sys.21288

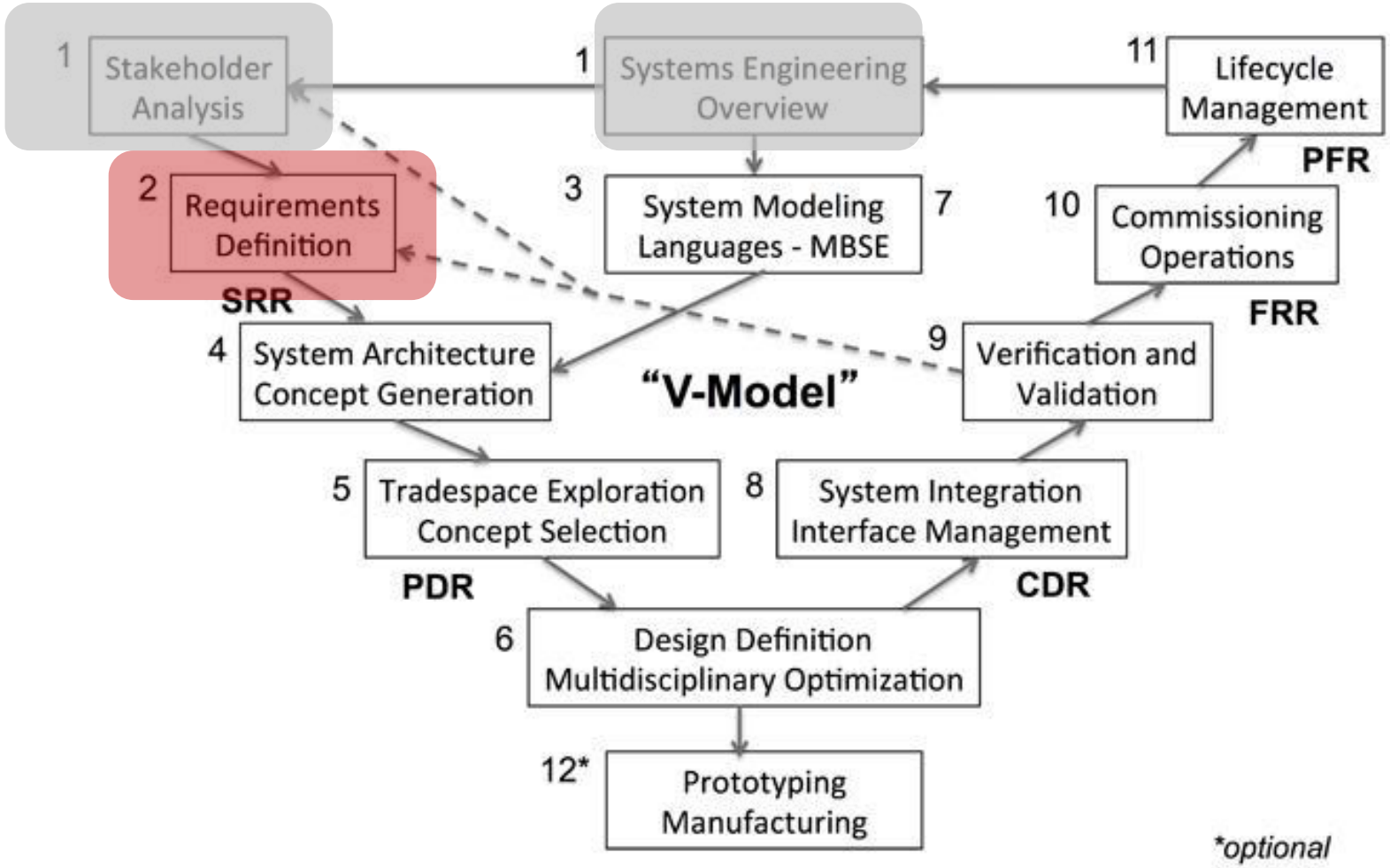
Requirements Volatility



Volatility scores of node on a relationship	Scoring scheme
Multiplicity (M)	Sum of multiplicative inverses of path lengths from it to nodes it multiplies the change to. $M_i = \sum \frac{1}{PL_{i \to j(\{i,j\} \subset \{1,2,3,\ldots,n\}, i \neq j)}}$ Both R_i and R_j changed.
Absorbance (A)	Sum of multiplicative inverses of path lengths to it from changed nodes it absorbs the change from. $A_i = \sum \frac{1}{PL_{i \to j(\{i,j\} \subset \{1,2,3,\ldots,n\}, i \neq j)}}$ Both R_j and $R_i$changed.
Transmittance (T)	Sum of multiplicative inverses of path lengths from it to nodes it transmits the change to. $T_i = \sum \frac{1}{PL_{i \to j(\{i,j\} \subset \{1,2,3,\ldots,n\}, i \neq j)}}$ R_i not changed. R_j changed.
Robustness (R)	Sum of multiplicative inverses of path lengths to it from changed nodes it is not affected by. $R_i = \sum \frac{1}{PL_{j \to i(\{i,j\} \subset \{1,2,3,\ldots,n\}, i \neq j)}}$ R_i not changed. R_j changed.

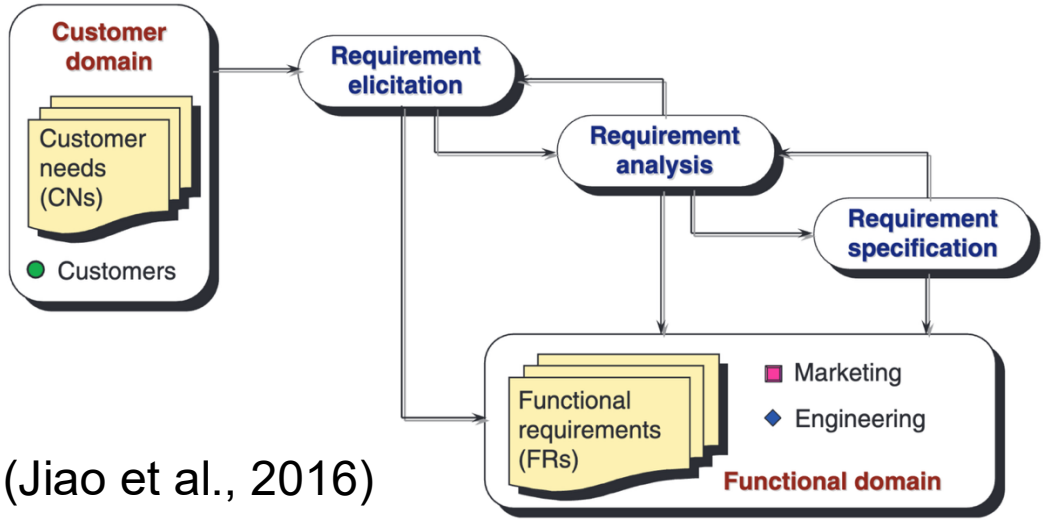
Hein, Phyo Htet, et al. "Reasoning support for predicting requirement change volatility using complex network metrics." *Journal of Engineering Design* 33.11 (2022): 811-837.

The "V-Model" of Systems Engineering



Outline

- Functional and non-functional requirements
- Requirements engineering processes
- Requirements elicitation
- Requirements specification
- Requirements validation
- Requirements change



(Jiao et al., 2016)

Functional Requirements

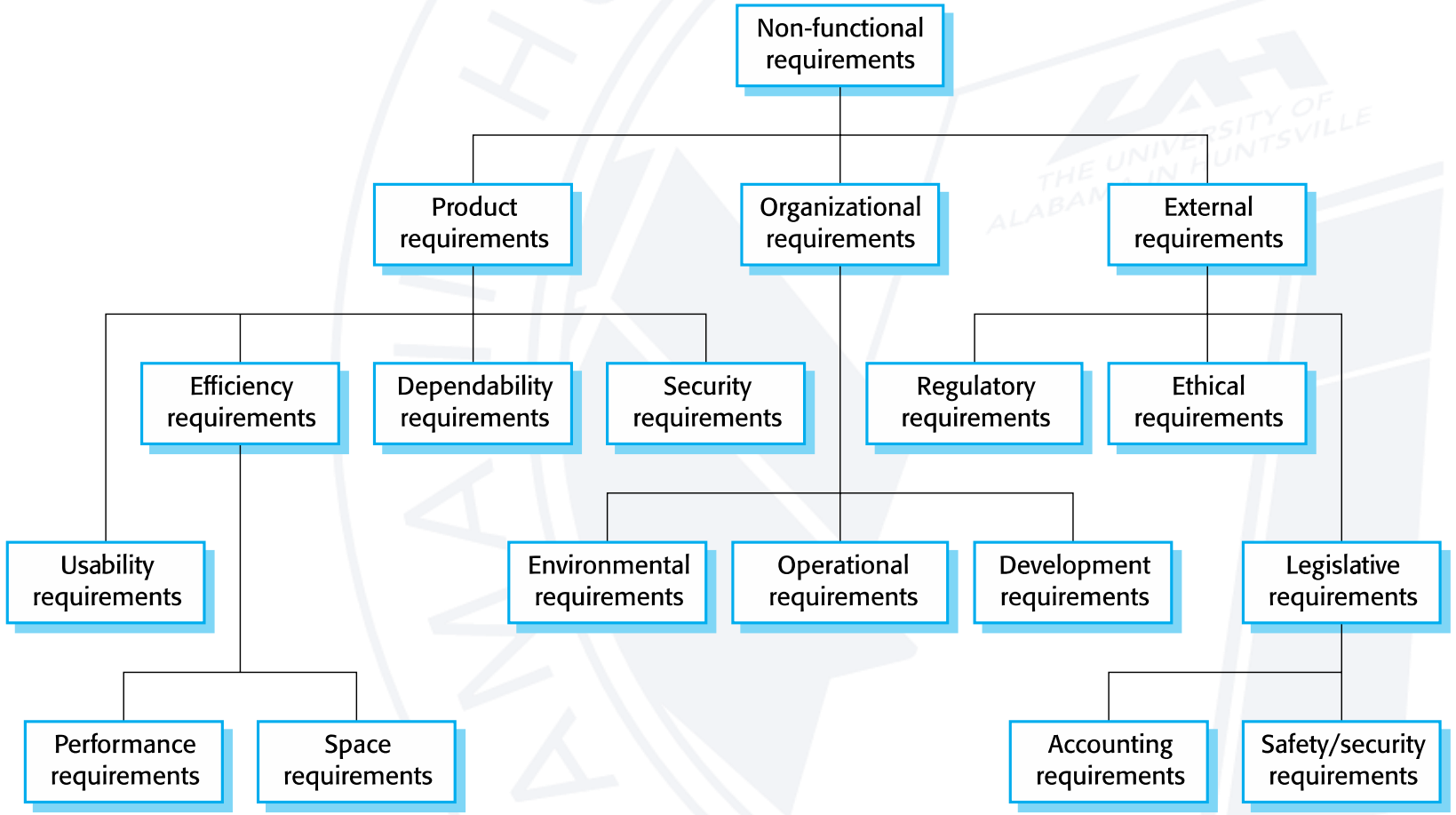
- Describe functionality or system services.
- Depend on the type of software, expected users and the type of system where the software is used.
- Functional user requirements may be high-level statements of what the system should do.
- Functional system requirements should describe the system services in detail.

Non-Functional Requirements

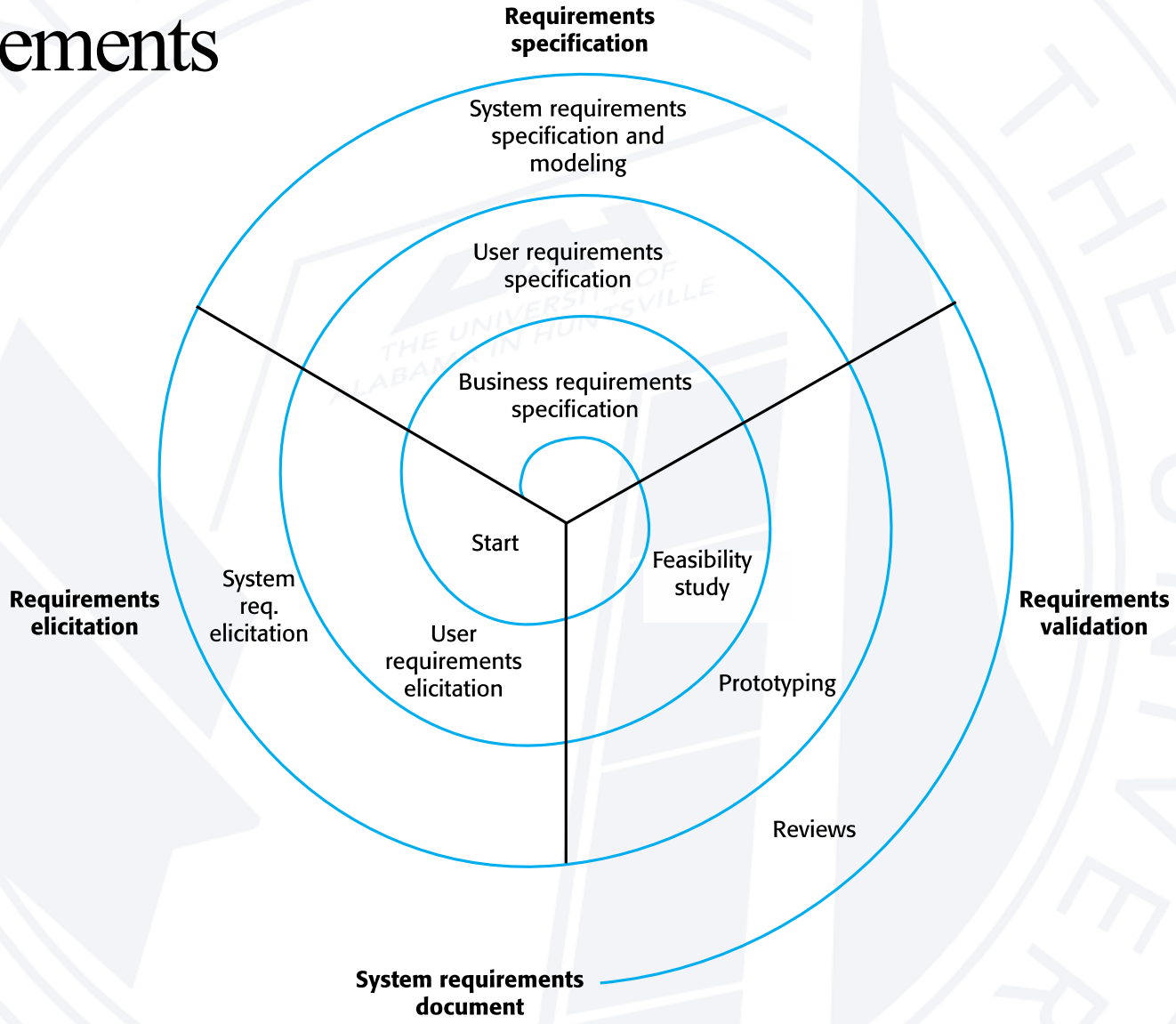
- Non-functional requirements may be very difficult to state precisely and imprecise requirements may be difficult to verify.

- Goal
  - A general intention of the user such as ease of use.
- Verifiable non-functional requirement
  - A statement using some measure that can be objectively tested.
- Goals are helpful to developers as they convey the intentions of the system users.

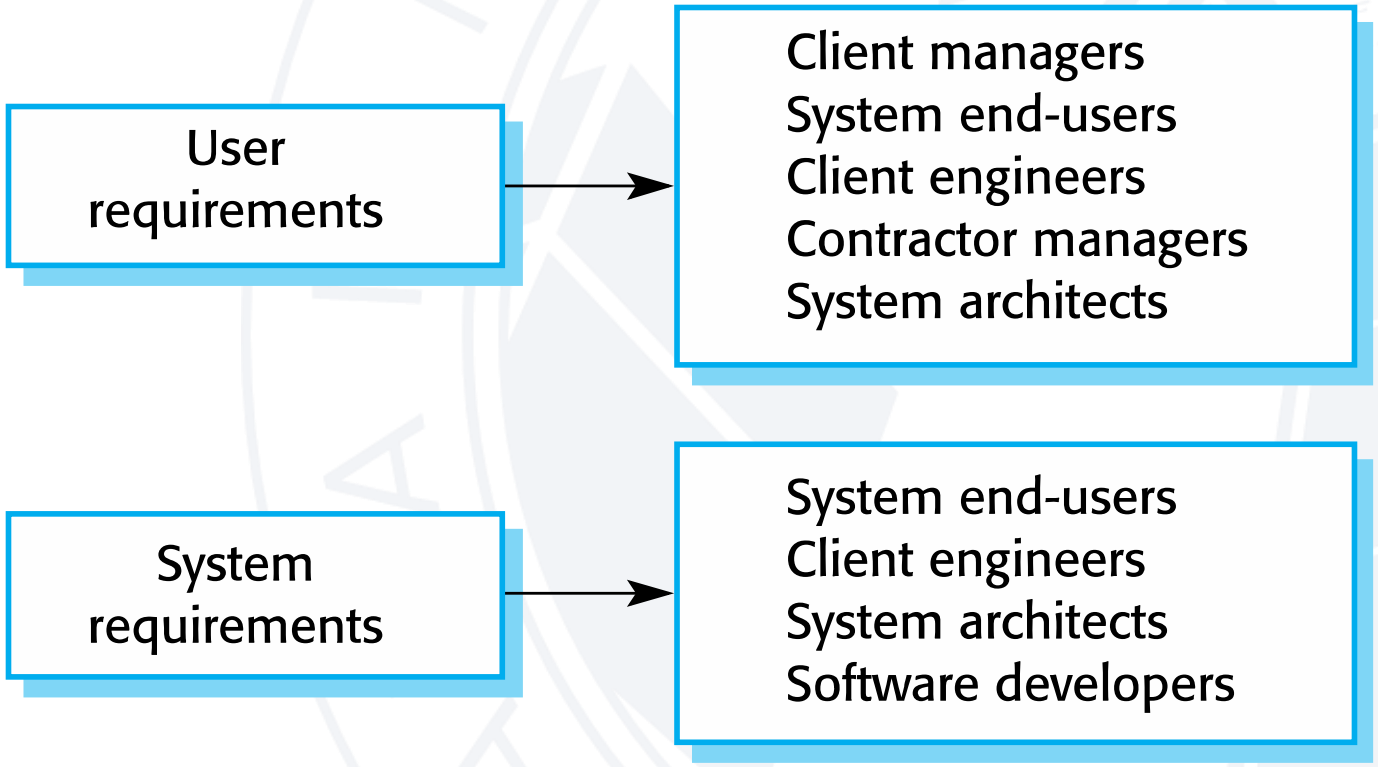
Types of Nonfunctional Requirement



A Spiral View of the Requirements Engineering Process



Readers of Different Types of Requirements Specification



Metrics for Specifying Nonfunctional Requirements

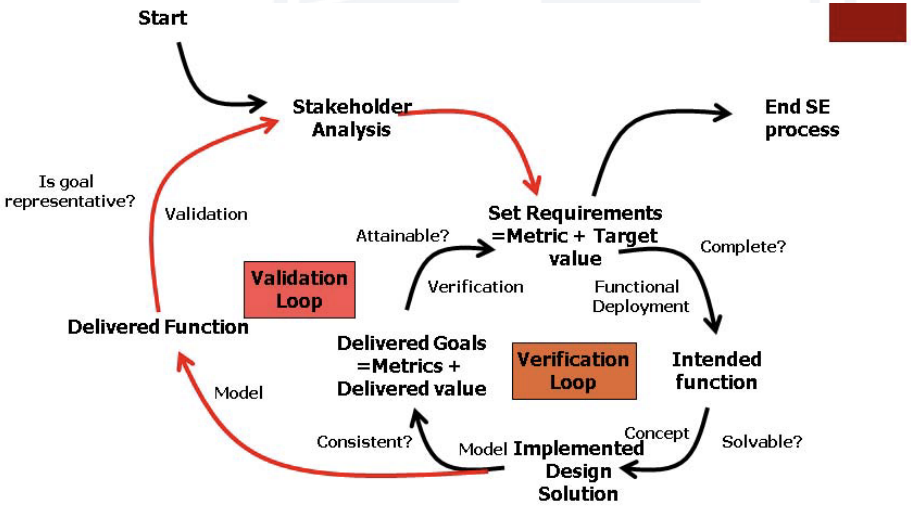
Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Requirement Elicitation

Requirements Elicitation

- Software engineers work with a range of system stakeholders to find out about the application domain, the services that the system should provide, the required system performance, hardware constraints, other systems, etc.

- Stages include:
  Requirements discovery,
  Requirements classification and organization,
  Requirements prioritization and negotiation,
  Requirements specification.

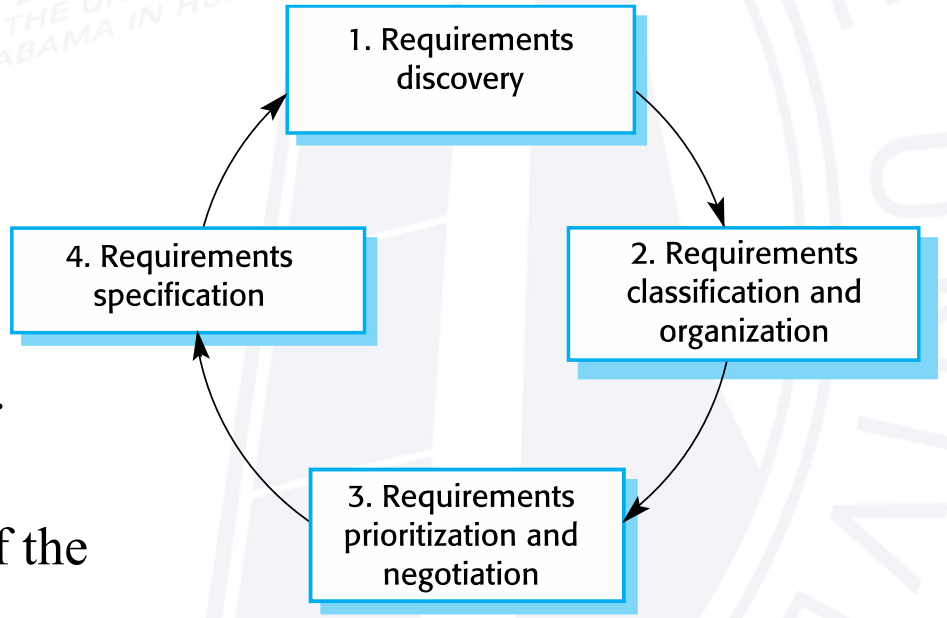


Problems of Requirements Elicitation

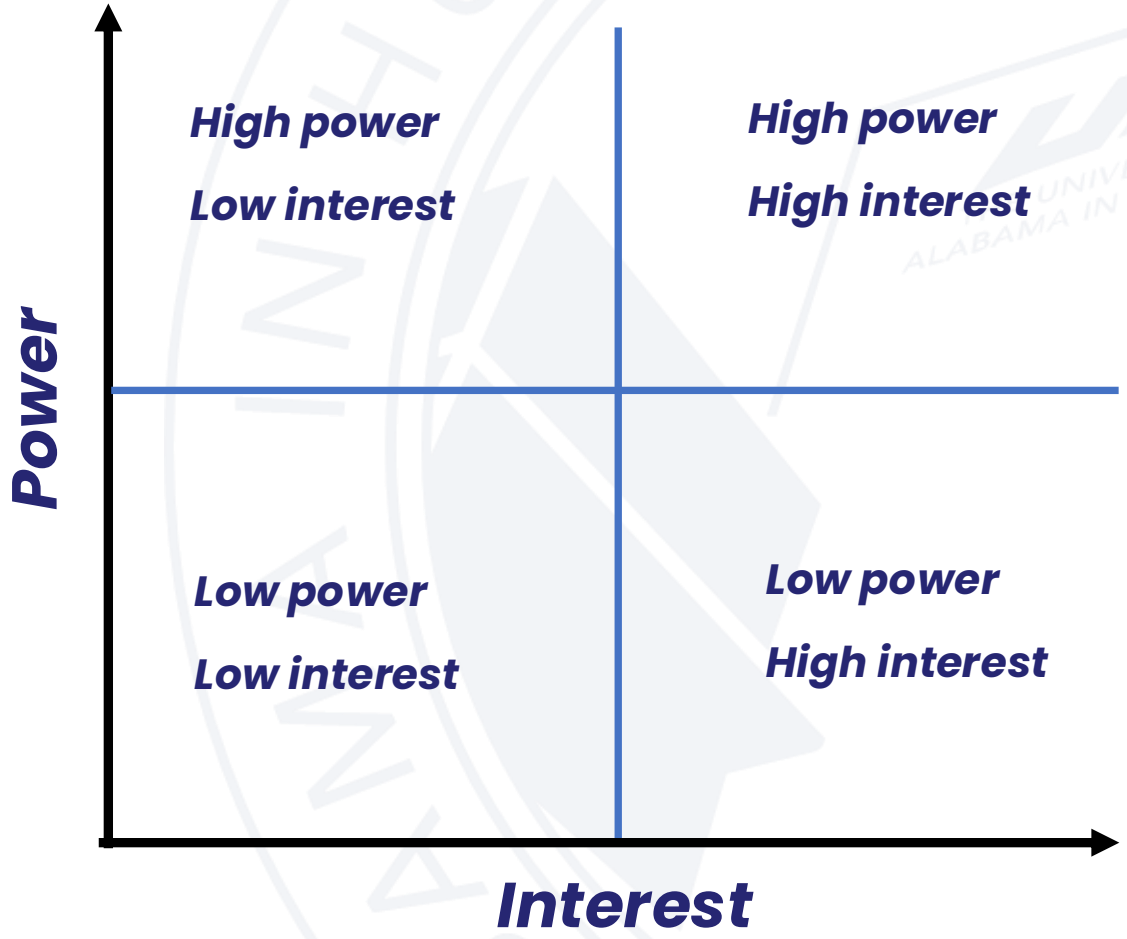
- Stakeholders don’t know what they really want.
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment may change.

The Requirements Elicitation and Analysis Process

- Requirements **discovery**
  - Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- Requirements **classification** and **organisation**
  - Groups related requirements and organises them into coherent clusters.
- **Prioritisation** and **negotiation**
  - Prioritising requirements and resolving requirements conflicts.
- Requirements **specification**
  - Requirements are documented and input into the next round of the spiral.



Stakeholder Analysis



Interviewing

- Formal or informal interviews with stakeholders are part of most RE processes.

- Types of interview
  - Closed interviews based on pre-determined list of questions
  - Open interviews where various issues are explored with stakeholders.

- Effective interviewing
  - Be open-minded, avoid pre-conceived ideas about the requirements and are willing to listen to stakeholders.
  - Prompt the interviewee to get discussions going using a springboard question, a requirements proposal, or by working together on a prototype system.

Interviewing in Practice

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviewers need to be open-minded without pre-conceived ideas of what the system should do
- You need to prompt the use to talk about the system by suggesting requirements rather than simply asking them what they want.

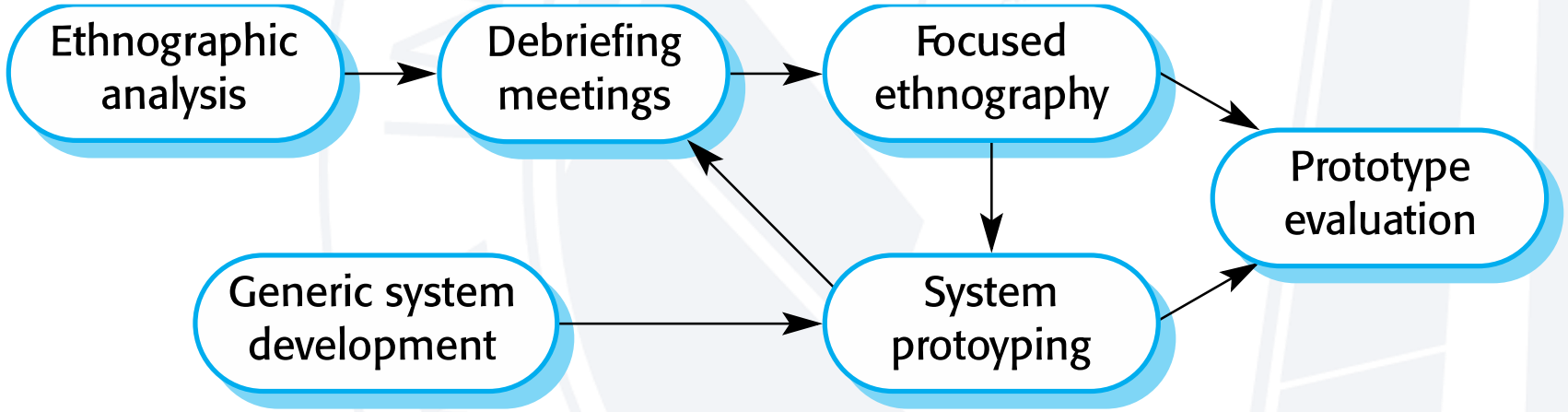
Problems with Interviews

- Application specialists may use language to describe their work that isn't easy for the requirements engineer to understand.
- Interviews are not good for understanding domain requirements
- Requirements engineers cannot understand specific domain terminology;
- Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.

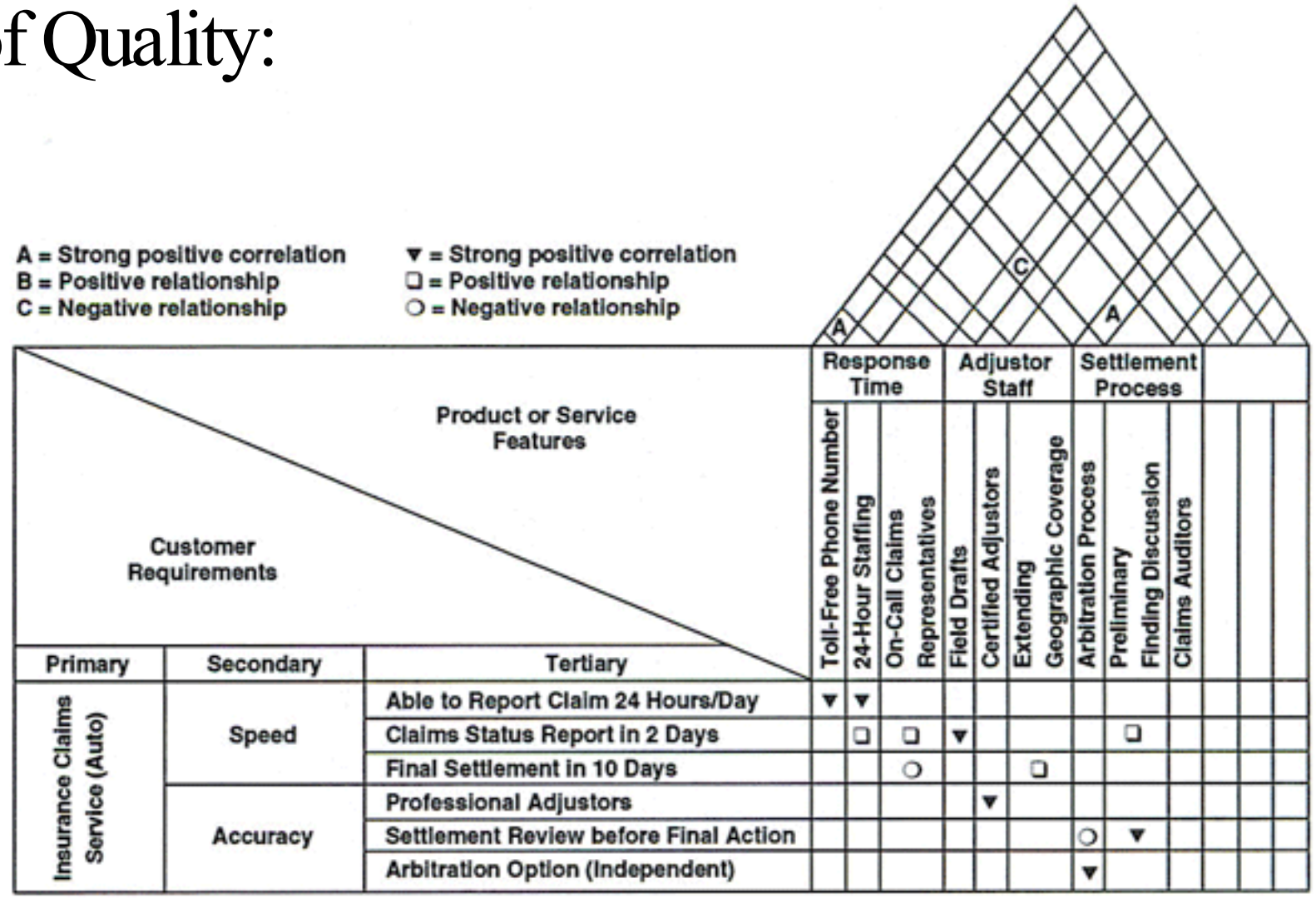
Ethnography

- Application specialists may use language to describe their work that isn't easy for the requirements engineer to understand.
- Interviews are not good for understanding domain requirements
- Requirements engineers cannot understand specific domain terminology;
- Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.

Ethnography and Prototyping for Requirements Analysis



House of Quality:



Quality Function Deployment's House of Quality (partly completed)

Requirement Specification

Requirements Specification

- The process of writing down the user and system requirements in a requirements document.
- User requirements have to be understandable by end-users and customers who do not have a technical background.
- System requirements are more detailed requirements and may include more technical information.
- The requirements may be part of a contract for the system development
- It is therefore important that these are as complete as possible.

Ways of Writing a System Requirements Specification

Notation	Description
Natural language	The requirements are written using numbered sentences in natural language. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a standard form or template. Each field provides information about an aspect of the requirement.
Design description languages	This approach uses a language like a programming language, but with more abstract features to specify the requirements by defining an operational model of the system. This approach is now rarely used although it can be useful for interface specifications.
Graphical notations	Graphical models, supplemented by text annotations, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on mathematical concepts such as finite-state machines or sets. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Requirements and Design

- In principle, requirements should state what the system should do and the design should describe how it does this.

- In practice, requirements and design are inseparable
  - A system architecture may be designed to structure the requirements;
  - The system may inter-operate with other systems that generate design requirements;
  - The use of a specific architecture to satisfy non-functional requirements may be a domain requirement.
  - This may be the consequence of a regulatory requirement.

Natural Language Specification

- Requirements are written as natural language sentences supplemented by diagrams and tables.

- Used for writing requirements because it is expressive, intuitive and universal. This means that the requirements can be understood by users and customers.

Guidelines for Writing Requirements

- Invent a standard format and use it for all requirements.
- Use language in a consistent way. Use shall for mandatory requirements, should for desirable requirements.
- Use text highlighting to identify key parts of the requirement.
- Avoid the use of computer jargon.
- Include an explanation (rationale) of why a requirement is necessary.

Problems with Natural Language

- Lack of clarity
  - Precision is difficult without making the document difficult to read.
- Requirements confusion
  - Functional and non-functional requirements tend to be mixed-up.
- Requirements amalgamation
  - Several different requirements may be expressed together.

Example Requirements for the Insulin Pump Software System

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. (Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. (A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.)

Form-Based Specifications

- Definition of the function or entity.
- Description of inputs and where they come from.
- Description of outputs and where they go to.
- Information about the information needed for the computation and other entities used.
- Description of the action to be taken.
- Pre and post conditions (if appropriate).
- The side effects (if any) of the function.

A Structured Specification of a Requirement for an Insulin Pump (1 of 2)

Insulin Pump/Control Software/SRS/3.3.2

Function Compute insulin dose: safe sugar level

Description

Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.

Inputs Current sugar reading (r2); the previous two readings (r0 and r1).

Source Current sugar reading from sensor. Other readings from memory.

Outputs CompDose—the dose in insulin to be delivered.

Destination Main control loop.

A Structured Specification of a Requirement for an Insulin Pump (2 of 2)

Action

CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.

Requirements

Two previous readings so that the rate of change of sugar level can be computed.

Pre-condition

The insulin reservoir contains at least the maximum allowed single dose of insulin.

Post-condition ro is replaced by rl then rl is replaced by r2.

Side effects None.

Tabular Specification

- Used to supplement natural language.
- Particularly useful when you have to define a number of possible alternative courses of action.
- For example, the insulin pump systems bases its computations on the rate of change of blood sugar level and the tabular specification explains how to calculate the insulin requirement for different scenarios.

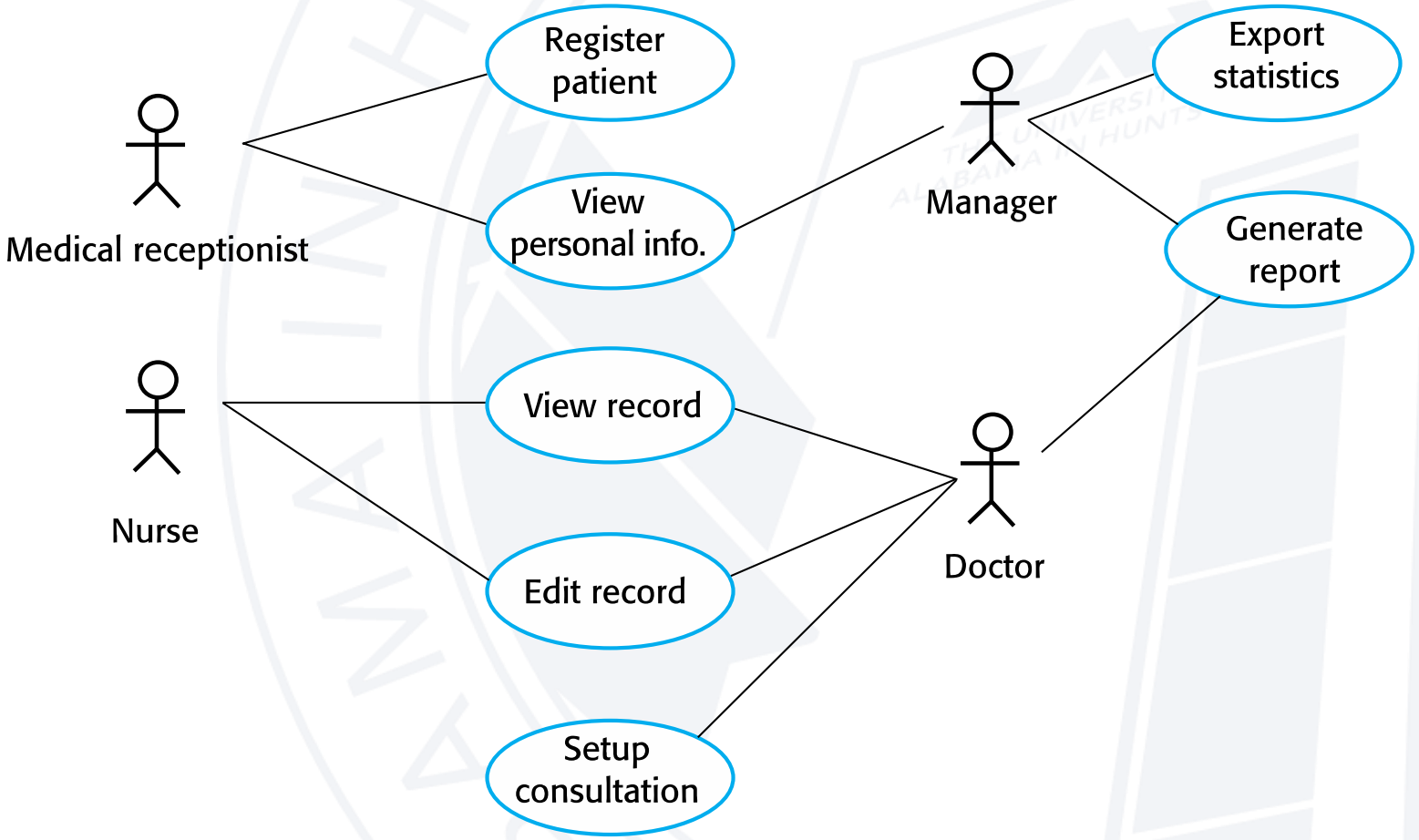
Tabular Specification

Condition	Action
Sugar level falling (r2 < r1)	CompDose = 0
Sugar level stable (r2 = r1)	CompDose = 0
Sugar level increasing and rate of increase decreasing ((r2 – r1) < (r1 – r0))	CompDose = 0
Sugar level increasing and rate of increase stable or increasing ((r2 – r1) ≥ (r1 – r0))	CompDose = round ((r2 – r1)/4) If rounded result = 0 then CompDose = MinimumDose

Use Cases

- Use-cases are a kind of scenario that are included in the UML.
- Use cases identify the actors in an interaction and which describe the interaction itself.
- A set of use cases should describe all possible interactions with the system.
- High-level graphical model supplemented by more detailed tabular description (see Chapter 5).
- UML sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing in the system.

Use Cases for the Mentcare System



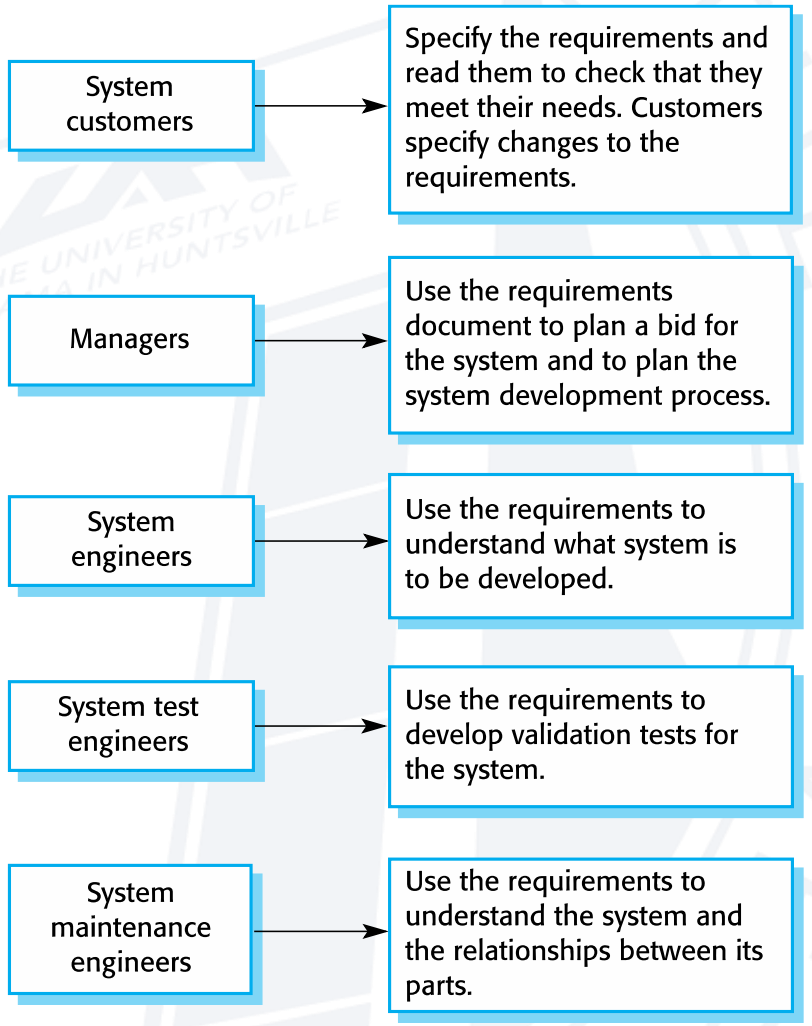
Requirements Document

- The requirements document is the official statement of what is required of the system developers.
- Should include both a definition of user requirements and a specification of the system requirements.
- It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.

Users of a Requirements Document

Requirements Document Variability:

- Information in requirements document depends on type of system and the approach to development used.
- Systems developed incrementally will, typically, have less detail in the requirements document.
- Requirements documents standards have been designed e.g. I E E E standard. These are mostly applicable to the requirements for large systems engineering projects.



The Structure of a Requirements Document (1 of 2)

Chapter	Description
Preface	This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version.
Introduction	This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.
Glossary	This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted.

NASA cFS

The Structure of a Requirements Document (1 of 2)

Chapter	Description
System requirements specification	This should describe the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on.

Requirement Validation

Outline

- Verification and Validation
  - What is their role?
  - Position in the lifecycle
- Testing
  - Aircraft flight testing (experimental vs. certification)
  - Spacecraft testing (“shake and bake”)
  - Caveats
- Technical Risk Management
  - Risk Matrix
  - Iron Triangle in Projects: Cost, Schedule, Scope > Risk
  - System Safety
- Flight Readiness Review (FRR)

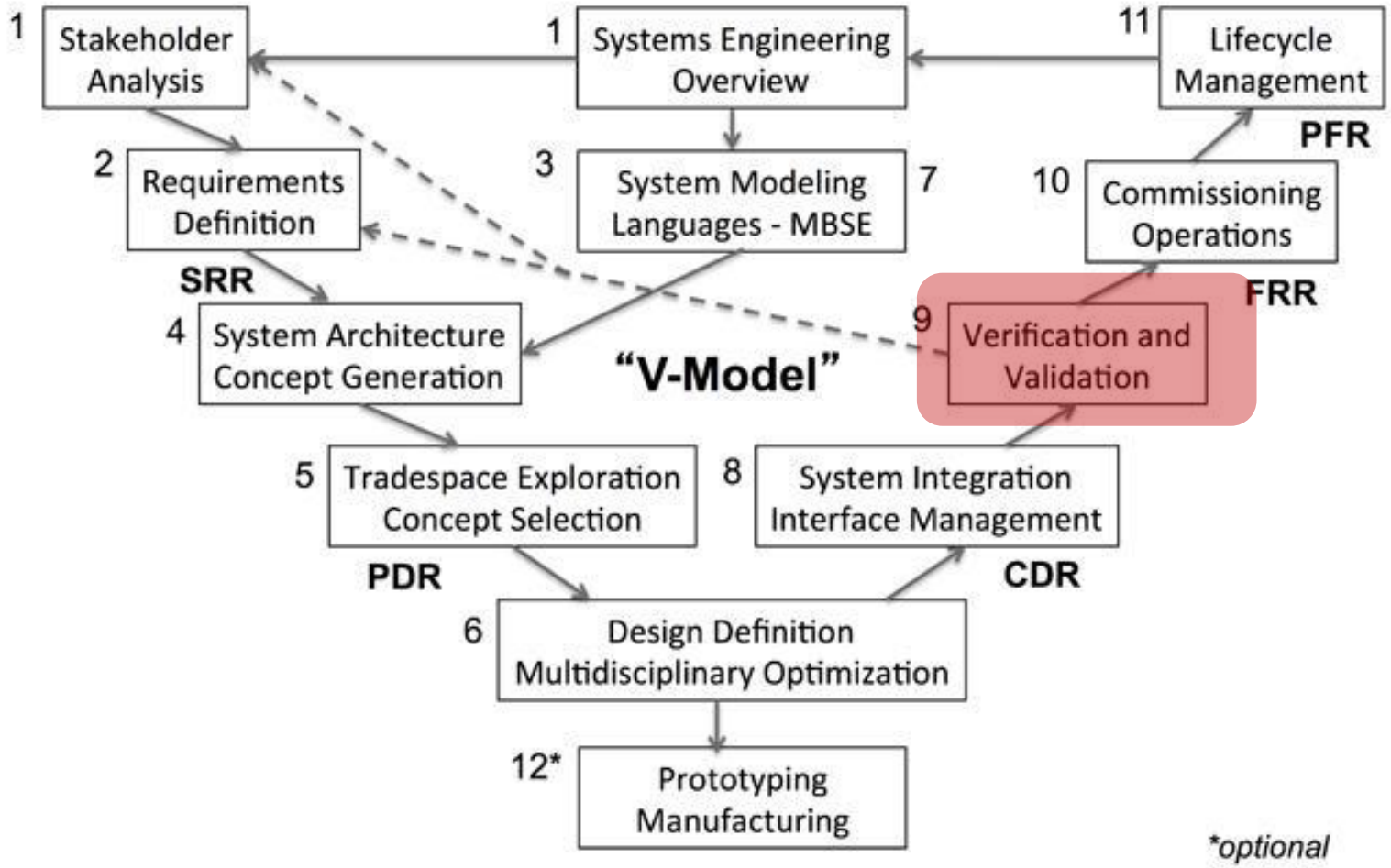
Readings related to this lecture

- NASA/SP-2007-6105
  - Section 5.3 (pp. 83-97)
  - Section 5.4 (pp. 98-105)
  - Appendix E (p. 284)
  - Appendix I (p. 301)

- Leveson, N., “A New Accident Model for Engineering Safer Systems”, Safety Science, Vol. 42, No. 4, April 2004

Requirements Validation

- Concerned with demonstrating that the 9 requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important
  - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.



Requirements Checking

- **Validity.** Does the system provide the functions which best support the customer's needs?
- **Consistency.** Are there any requirements conflicts?
- **Completeness.** Are all functions required by the customer included?
- **Realism.** Can the requirements be implemented given available budget and technology
- **Verifiability.** Can the requirements be checked?

Requirements Validation Techniques

- **Requirements reviews**
  - Systematic manual analysis of the requirements.
- **Prototyping**
  - Using an executable model of the system to check requirements.
- **Test-case generation**
  - Developing tests for requirements to check testability.

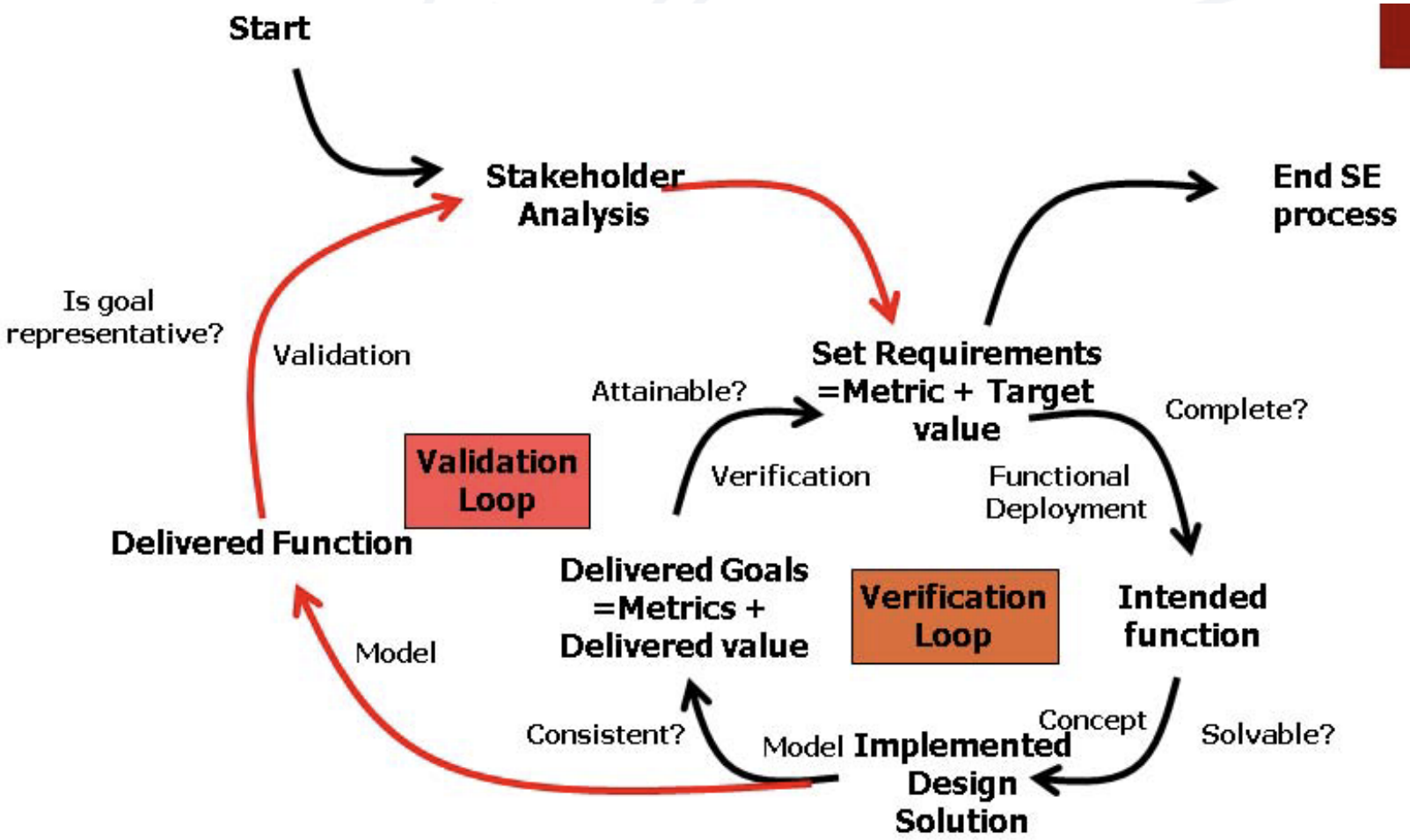
Requirements Reviews

- **Regular reviews** should be held while the requirements definition is being formulated.
- Both client and contractor staff should be involved in reviews.
- Reviews may be **formal** (with completed documents) or **informal**. Good communications between developers, customers and users can resolve problems at an early stage.

Review Checks

- **Verifiability**
  - Is the requirement realistically testable?
- **Comprehensibility**
  - Is the requirement properly understood?
- **Traceability**
  - Is the origin of the requirement clearly stated?
- **Adaptability**
  - Can the requirement be changed without a large impact on other requirements?

Verification and Validation



Differences between V & V

Differences Between Verification and Validation Testing

Verification Testing

Verification testing relates back to the approved requirements set (such as an SRD) and can be performed at different stages in the product life cycle. Verification testing includes: (1) any testing used to assist in the development and maturation of products, product elements, or manufacturing or support processes; and/or (2) any engineering-type test used to verify the status of technical progress, verify that design risks are minimized, substantiate achievement of contract technical performance, and certify readiness for initial validation testing. Verification tests use instrumentation and measurements and are generally accomplished by engineers, technicians, or operator-maintainer test personnel in a controlled environment to facilitate failure analysis.

Validation Testing

Validation relates back to the ConOps document. Validation testing is conducted under realistic conditions (or simulated conditions) on any end product to determine the effectiveness and suitability of the product for use in mission operations by typical users and to evaluate the results of such tests. Testing is the detailed quantifying method of both verification and validation. However, testing is required to validate final end products to be produced and deployed.

Verification Was the end product realized right?

- During development
- Check if requirements are met
- Typically in the laboratory
- Component/subsystem centric

Validation Was the right end product realized?

-During or after integration
-Typically in real or simulated mission environment
-Check if stakeholder intent is met
- Full-up system

Concept Question

What is your name?

How would you classify the following activities? *

	Verification	Validation	No sure
Testing handling of a new car in snow conditions in Alaska	○	○	○
Frontal crash test in the lab	○	○	○
Testing of a new toy in a Kindergarten	○	○	○
Vehicle emissions testing on a dynamo	○	○	○
Satellite vibration testing on a shake table	○	○	○
Field testing of Google glasses with 1,500 pilot users	○	○	○

Product Verification Process

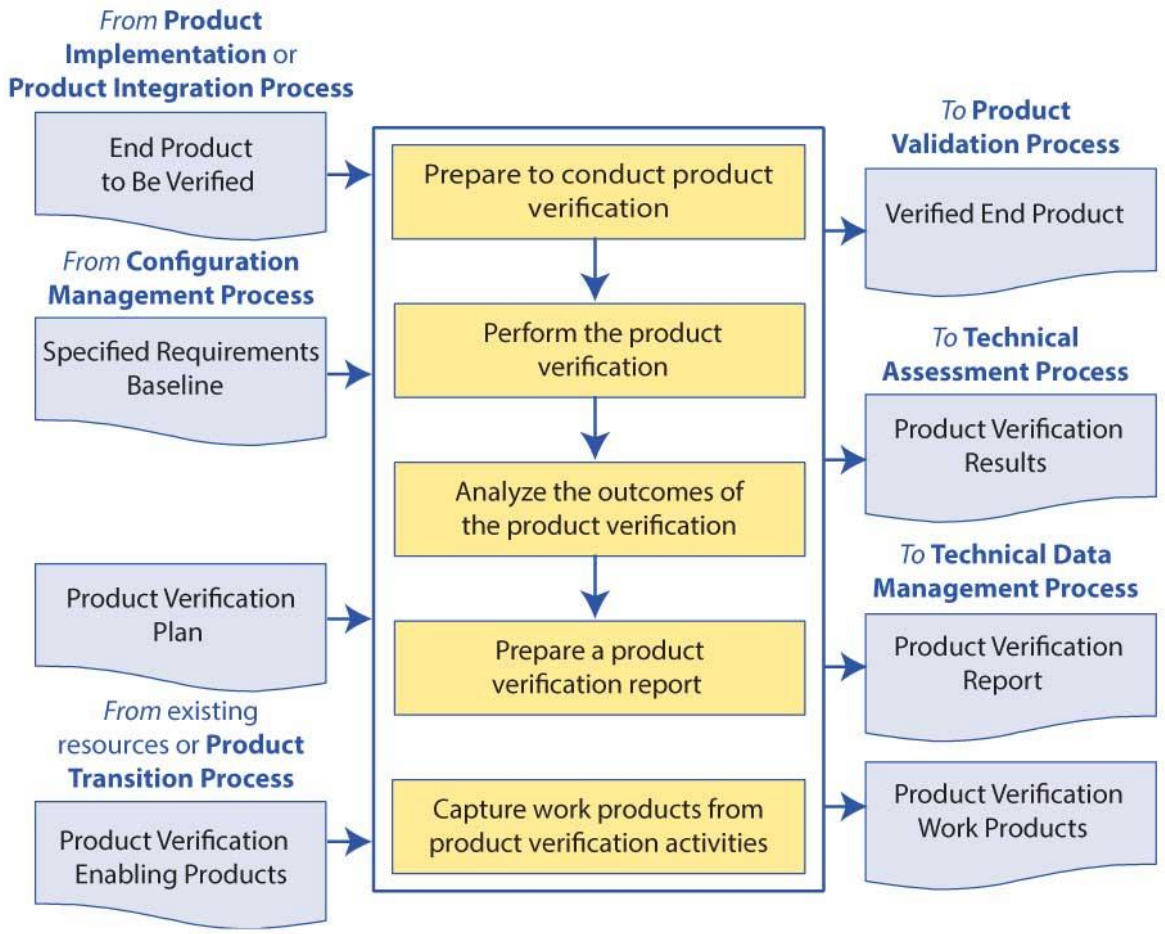


FIGURE 5.3-1 Product Verification Process

Types of verification:

-Analysis

-Demonstration

-Inspection

-Test

Outputs:

-Discrepancy reports

-Verified product

-Compliance documentation

NASA Life-Cycle Phases

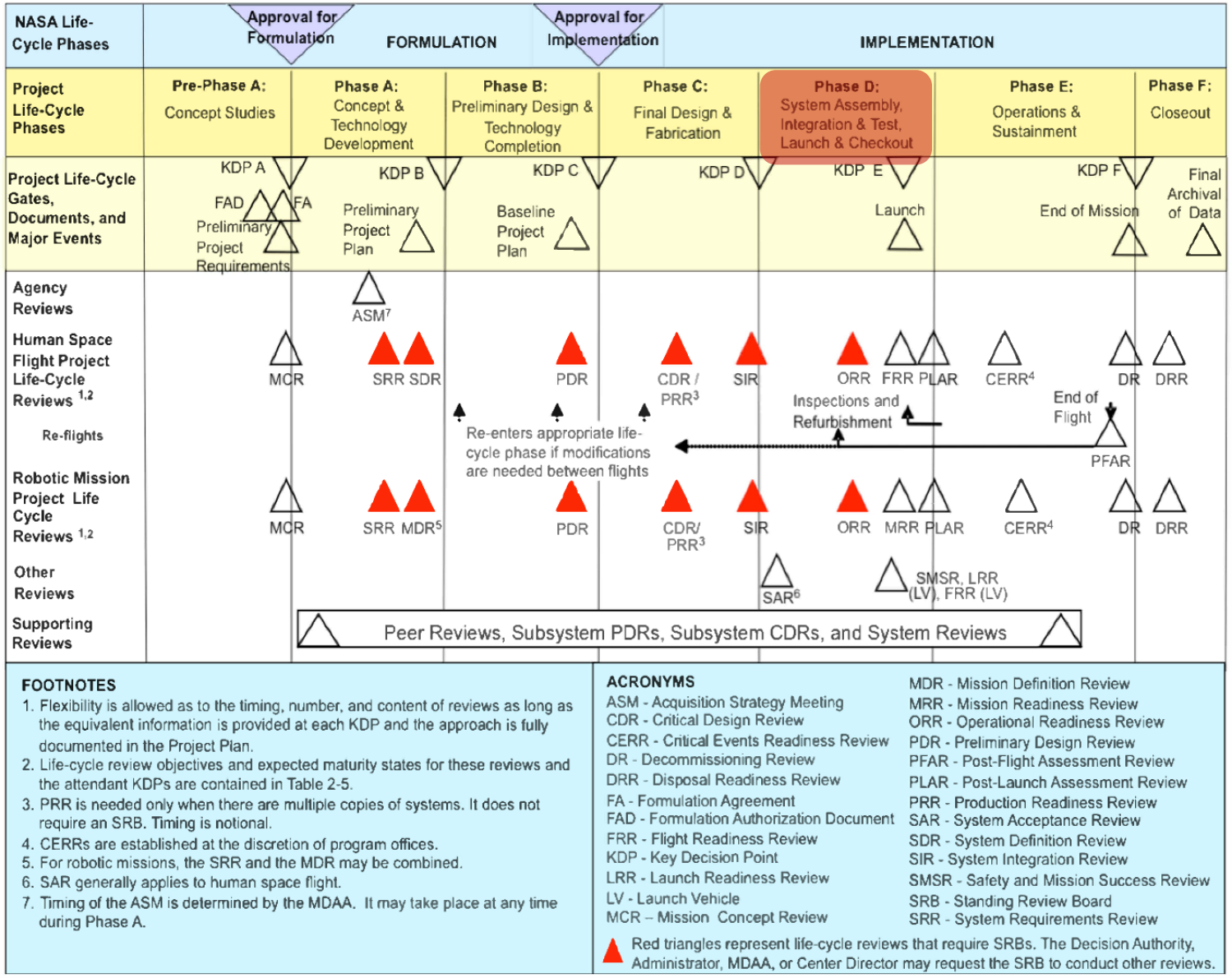


Figure 1: NASA Project Life Cycle

NASA Life-Cycle Reviews

Review	Title	Purpose
SRR	Program Requirement Review	The P/SRR is used to ensure that the program requirements are properly formulated and correlated with the Agency and mission directorate strategic objectives
P/SDR	Program Definition Review, or System Definition Review	The P/SDR ensures the readiness of the program for making a program commitment agreement to approve project formulation startups during program Implementation phase.
MCR	Mission Concept Review	The MCR affirms the mission need and examines the proposed mission's objectives and the concept for meeting those objectives
SRR	System Requirement Review	The SRR examines the functional and performance requirements defined for the system and the preliminary program or project plan and ensures that the requirements and the selected concept will satisfy the mission
MDR	Mission Definition Review	The MDR examines the proposed requirements, the mission architecture, and the flow down to all functional elements of the mission to ensure that the overall concept is complete, feasible, and consistent with available resources
SDR	System Definition Review	The SDR examines the proposed system architecture and design and the flow down to all functional elements of the system.
PDR	Preliminary Design Review	The PDR demonstrates that the preliminary design meets all system requirements with acceptable risk and within the cost and schedule constraints and establishes the basis for proceeding with detailed design. It will show that the correct design options have been selected, interfaces have been identified, and verification methods have been described
CDR	Critical Design review	The CDR demonstrates that the maturity of the design is appropriate to support proceeding with full scale fabrication, assembly, integration, and test. CDR determines that the technical effort is on track to complete the flight and ground system development and mission operations, meeting mission performance requirements within the identified cost and schedule constraints.
PRR	Production Readiness Review	A PRR is held for FS&GS projects developing or acquiring multiple or similar systems greater than three or as determined by the project. The PRR determines the readiness of the system developers to efficiently produce the required number of systems. It ensures that the production plans; fabrication, assembly, and integration enabling products; and personnel are in place and ready to begin production.

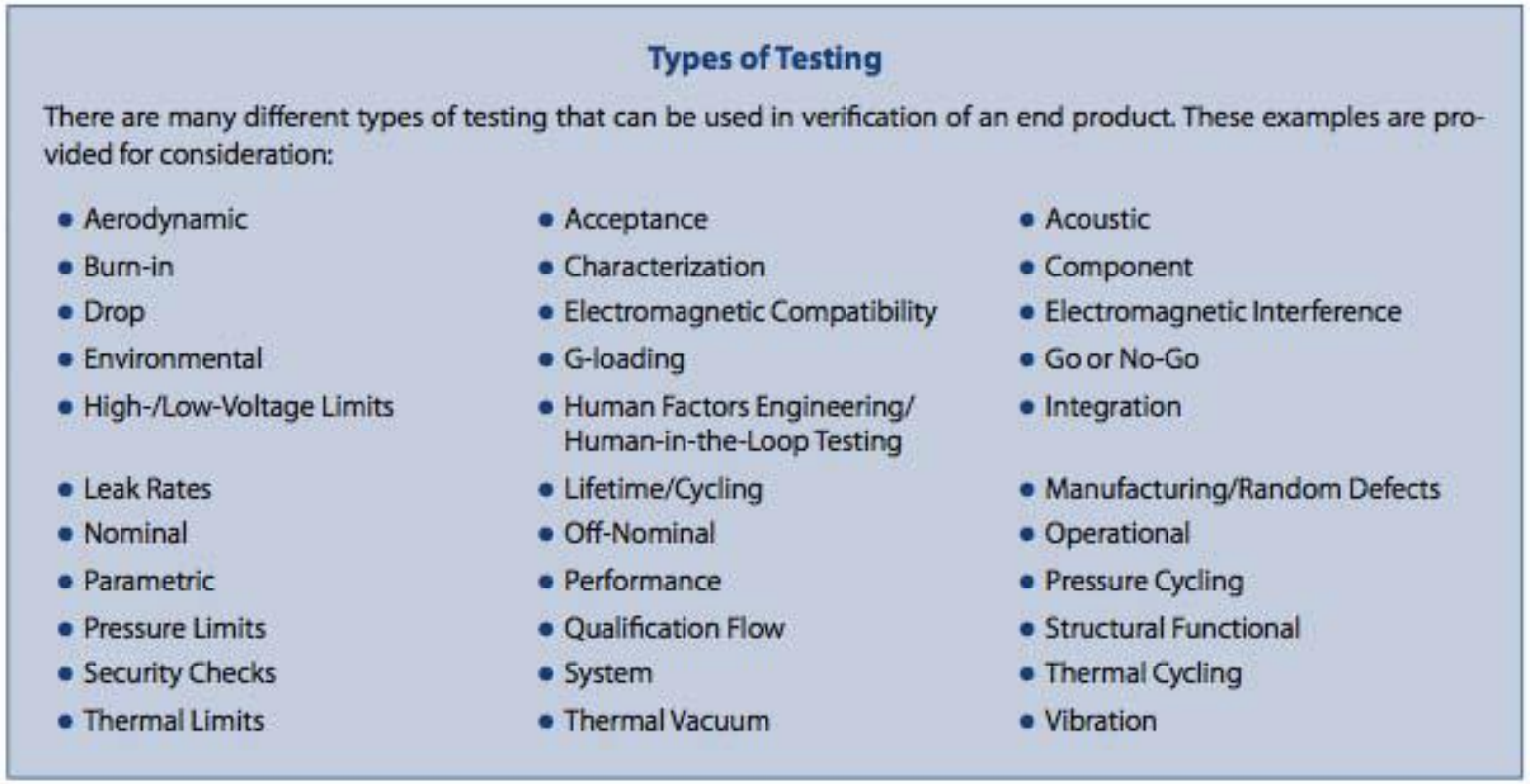
NASA Life-Cycle Reviews

Review	Title	Purpose
SIR	System Integration Review	An SIR ensures that the system is ready to be integrated. Segments, components, and subsystems are available and ready to be integrated into the system. Integration facilities, support personnel, and integration plans and procedures are ready for integration.
TRR	Test Readiness Review	A TRR ensures that the test article (hardware/software), test facility, support personnel, and test procedures are ready for testing and data acquisition, reduction, and control.
SAR	System Acceptance Review	The SAR verifies the completeness of the specific end products in relation to their expected maturity level and assesses compliance to stakeholder expectations. The SAR examines the system, its end products and documentation, and test data and analyses that support verification. It also ensures that the system has sufficient technical maturity to authorize its shipment to the designated operational facility or launch site.
ORR	Operational Readiness Review	The ORR examines the actual system characteristics and the procedures used in the system or end product's operation and ensures that all system and support (flight and ground) hardware, software, personnel, procedures, and user documentation accurately reflect the deployed state of the system.
FRR	Flight Readiness Review	The FRR examines tests, demonstrations, analyses, and audits that determine the system's readiness for a safe and successful flight or launch and for subsequent flight operations. It also ensures that all flight and ground hardware, software, personnel, and procedures are operationally ready.
PLAR	Post-Launch Assessment Review	A PLAR is a post-deployment evaluation of the readiness of the spacecraft systems to proceed with full, routine operations. The review evaluates the status, performance, and capabilities of the project evident from the flight operations experience since launch. This can also mean assessing readiness to transfer responsibility from the development organization to the operations organization. The review also evaluates the status of the project plans and the capability to conduct the mission with emphasis on near-term operations and mission-critical events. This review is typically held after the early flight operations and initial checkout.
CERR	Critical Event Readiness Review	A CERR confirms the project's readiness to execute the mission's critical activities during flight operation.
PFAR	Post-Flight Assessment Review	The PFAR evaluates the activities from the flight after recovery. The review identifies all anomalies that occurred during the flight and mission and determines the actions necessary to mitigate or resolve the anomalies for future flights.
DR	Decommissioning Review	A DR confirms the decision to terminate or decommission the system and assesses the readiness of the system for the safe decommissioning and disposal of system assets.

Outline

- Verification and Validation
  - What is their role?
  - Position in the lifecycle
- Testing
  - Aircraft flight testing (experimental vs. certification)
  - Spacecraft testing (“shake and bake”)
  - Caveats
- Technical Risk Management
  - Risk Matrix
  - Iron Triangle in Projects: Cost, Schedule, Scope > Risk
  - System Safety
- Flight Readiness Review (FRR)

Types of Testing



Types of Testing

There are many different types of testing that can be used in verification of an end product. These examples are provided for consideration:

- Aerodynamic
- Burn-in
- Drop
- Environmental
- High-/Low-Voltage Limits
- Leak Rates
- Nominal
- Parametric
- Pressure Limits
- Security Checks
- Thermal Limits
- Acceptance
- Characterization
- Electromagnetic Compatibility
- G-loading
- Human Factors Engineering/ Human-in-the-Loop Testing
- Lifetime/Cycling
- Off-Nominal
- Performance
- Qualification Flow
- System
- Thermal Vacuum
- Acoustic
- Component
- Electromagnetic Interference
- Go or No-Go
- Integration
- Manufacturing/Random Defects
- Operational
- Pressure Cycling
- Structural Functional
- Thermal Cycling
- Vibration

Source: NASA SE Handbook, Section 5.3 Product Verification

Aircraft Testing

- **Ground Testing**
  - Weights and Balance (determine mass, CG …)
  - Engine Testing (in “hush house”, outdoors)
  - Fatigue Testing (static and dynamic structural)
  - Avionics checkout
  - Pre-flight Testing (extended checklist)

- **Flight Testing**
  - Flight Performance Testing (rate of climb, range …)
  - Stability and Controls (stall speed, trim, flutter …)
  - Weapons testing (live fire tests, LO ..)

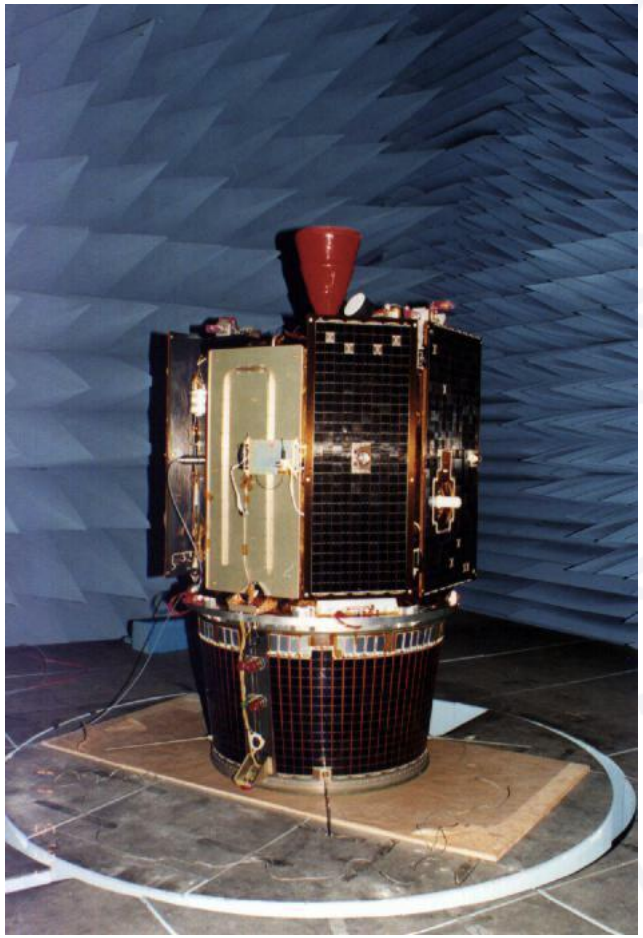
F/A-18C Hush House Testing (ca. 1995)



Spacecraft Integration Testing (NASA)



Anechoic Chamber Testing



Radio Frequency Anechoic Chamber Facility

The radio frequency anechoic chamber is used to design, manufacture, and test spacecraft antenna systems. The facility is also used for electromagnetic compatibility and electromagnetic interference testing of spacecraft antenna systems

Clementine Spacecraft

Testing Caveats

- Testing is critical, but expensive
  - Test rig, chamber, sensors, DAQ equipment …
- How much testing of components?
  - Trust parts vendors or retest everything?
- Calibration of sensors and equipment
  - If sensors are not calibrated properly can lead to erroneous conclusions
- “Test as you Fly, Fly as you test”
  - To what extent do the test conditions reflect actual operational usage?
- Simulated Tests
  - Use “dummy” components if the real ones are not available
  - Simulated operations (e.g. 0g vs. 1g) … are they representative?
- Failures often occur outside any test scenarios

Appendix E: Validation Matrix

Table E-1 Validation Requirements Matrix

Validation Product #	Activity	Objective	Validation Method	Facility or Lab	Phase	Performing Organization	Results
Unique identifier for validation product	Describe evaluation by the customer/sponsor that will be performed	What is to be accomplished by the customer/ sponsor evaluation	Validation method for the System X requirement (analysis, inspection, demonstration, or test)	Facility or laboratory used to perform the validation	Phase in which the verification/ validation will be performed[a]	Organization responsible for coordinating the validation activity	Indicate the objective evidence that validation activity occurred
1	Customer/ sponsor will evaluate the candidate displays	1. Ensure legibility is acceptable 2. Ensure overall appearance is acceptable	Test	xxx	Phase A	xxx	

a. Example: (1) during product selection process, (2) prior to final product selection (if COTS) or prior to PDR, (3) prior to CDR, (4) during box-level functional, (5) during system-level functional, (6) during end-to-end functional, (7) during integrated vehicle functional, (8) during on-orbit functional.

Appendix I : V&V Plan Outline

Appendix I: Verification and Validation Plan Sample Outline

1. **Introduction**
   - 1.1 Purpose and Scope
   - 1.2 Responsibility and Change Authority
   - 1.3 Definitions
2. **Applicable and Reference Documents**
   - 2.1 Applicable Documents
   - 2.2 Reference Documents
   - 2.3 Order of Precedence
3. **System X Description**
   - 3.1 System X Requirements Flow Down
   - 3.2 System X Architecture
   - 3.3 End Item Architectures
     - 3.3.1 System X End Item A
     - 3.3.*n* System X End Item *n*
   - 3.4 System X Ground Support Equipment
   - 3.5 Other Architecture Descriptions
4. **Verification and Validation Process**
   - 4.1 Verification and Validation Management Responsibilities
   - 4.2 Verification Methods
     - 4.2.1 Analysis
     - 4.2.2 Inspection
     - 4.2.3 Demonstration
     - 4.2.4 Test
       - 4.2.4.1 Qualification Testing
       - 4.2.4.2 Other Testing
   - 4.3 Validation Methods
   - 4.4 Certification Process
   - 4.5 Acceptance Testing
5. **Verification and Validation Implementation**
   - 5.1 System X Design and Verification and Validation Flow
   - 5.2 Test Articles
   - 5.3 Support Equipment
   - 5.4 Facilities
6. **System X End Item Verification and Validation**
   - 6.1 End Item A
     - 6.1.1 Developmental/Engineering Unit Evaluations
     - 6.1.2 Verification Activities
       - 6.1.2.1 Verification Testing
         - 6.1.2.1.1 Qualification Testing
         - 6.1.2.1.2 Other Testing

The degree to which V&V is taken seriously and resources are made available is critical for project outcome:

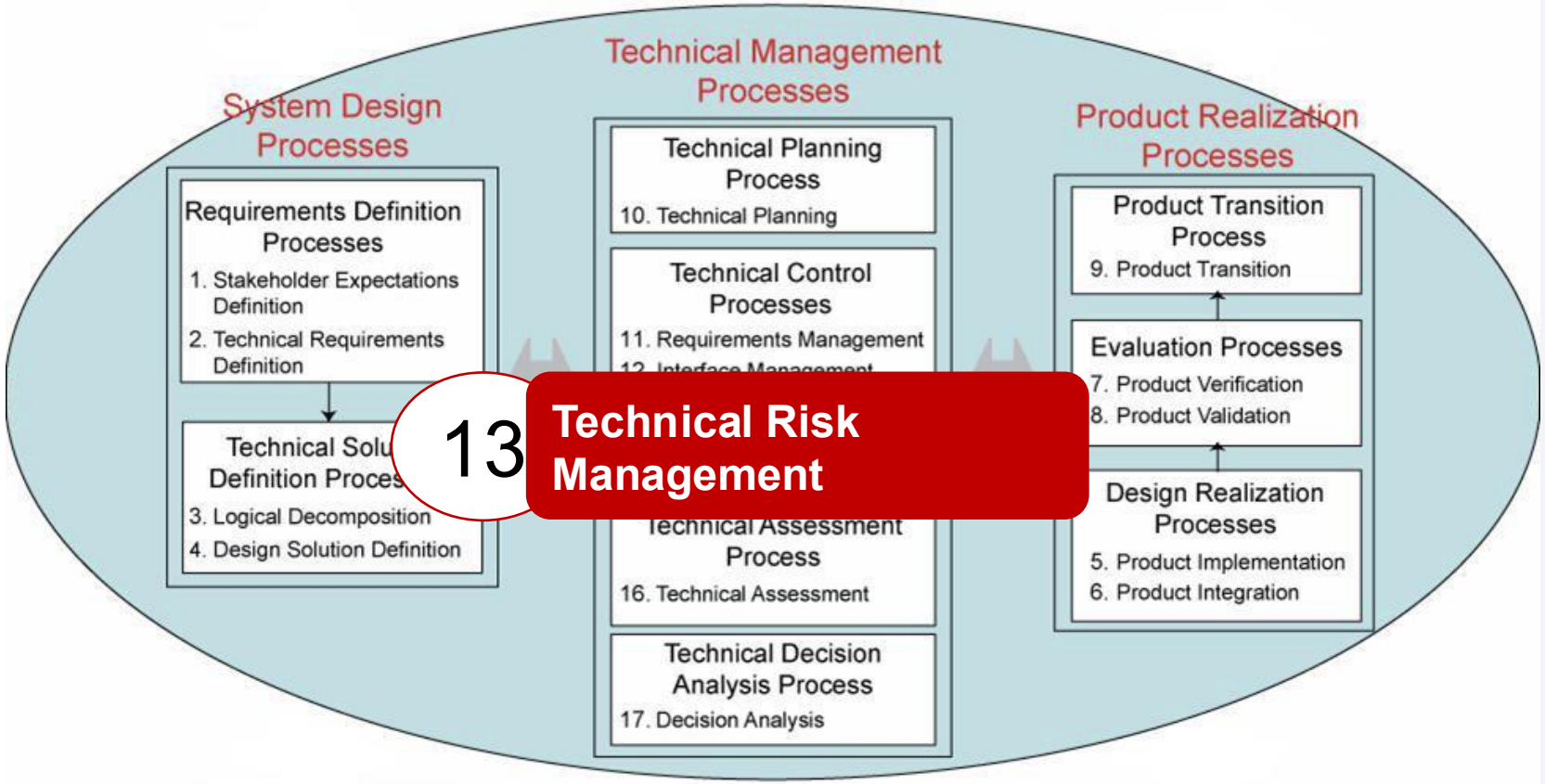
-# of dedicated QA personnel
-Interaction/working with suppliers
-Planning ahead for tests
-End-to-end functional testing
-Can often “piggy-back” on existing facilities, equipment …
-Document outcomes well and follow-up with discrepancies

This work is often not glamorous (except for some flight testing) but critical !

Outline

- Verification and Validation
  - What is their role?
  - Position in the lifecycle
- Testing
  - Aircraft flight testing (experimental vs. certification)
  - Spacecraft testing (“shake and bake”)
  - Caveats
- Technical Risk Management
  - Risk Matrix
  - Iron Triangle in Projects: Cost, Schedule, Scope > Risk
  - System Safety
- Flight Readiness Review (FRR)

Technical Risk Management



Importance of Technical Risk Management

- Risk is defined as the combination of:
  - The probability that a program or project will experience an undesired event
  - The consequences, impact, or severity of the undesired event, were it to occur

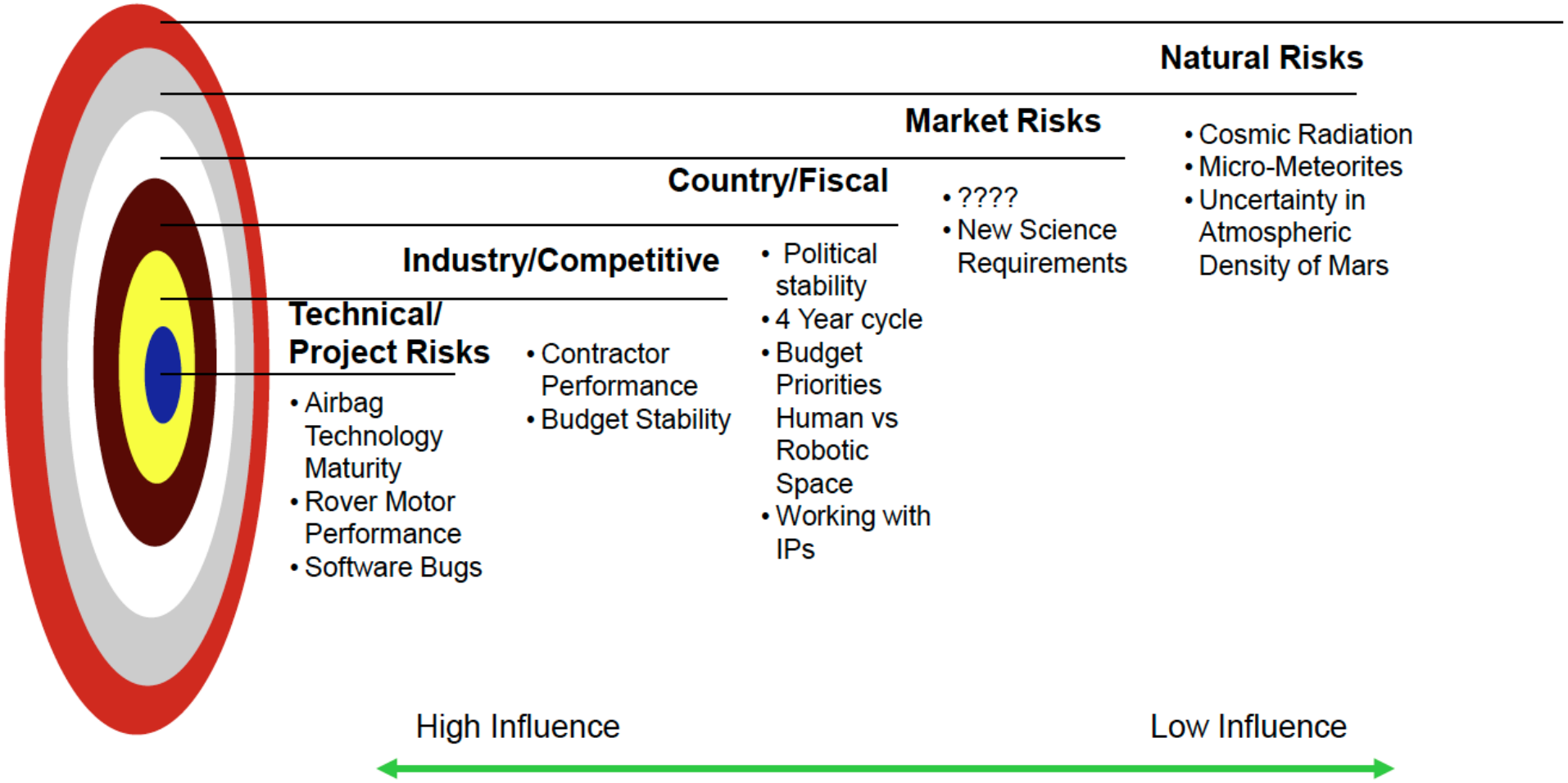
- The undesired event might come from technical or programmatic sources (e.g. a cost overrun, schedule slippage, safety mishap, health problem, malicious activities, environmental impact, or failure to achieve a needed scientific or technological objective or success criteria)

- Technical Risk Management is an organized, systematic risk-informed decision-making discipline that proactively identifies, analyzes, plans, tracks, controls, communicates, documents, and manages risk to increase the likelihood of achieving project goals

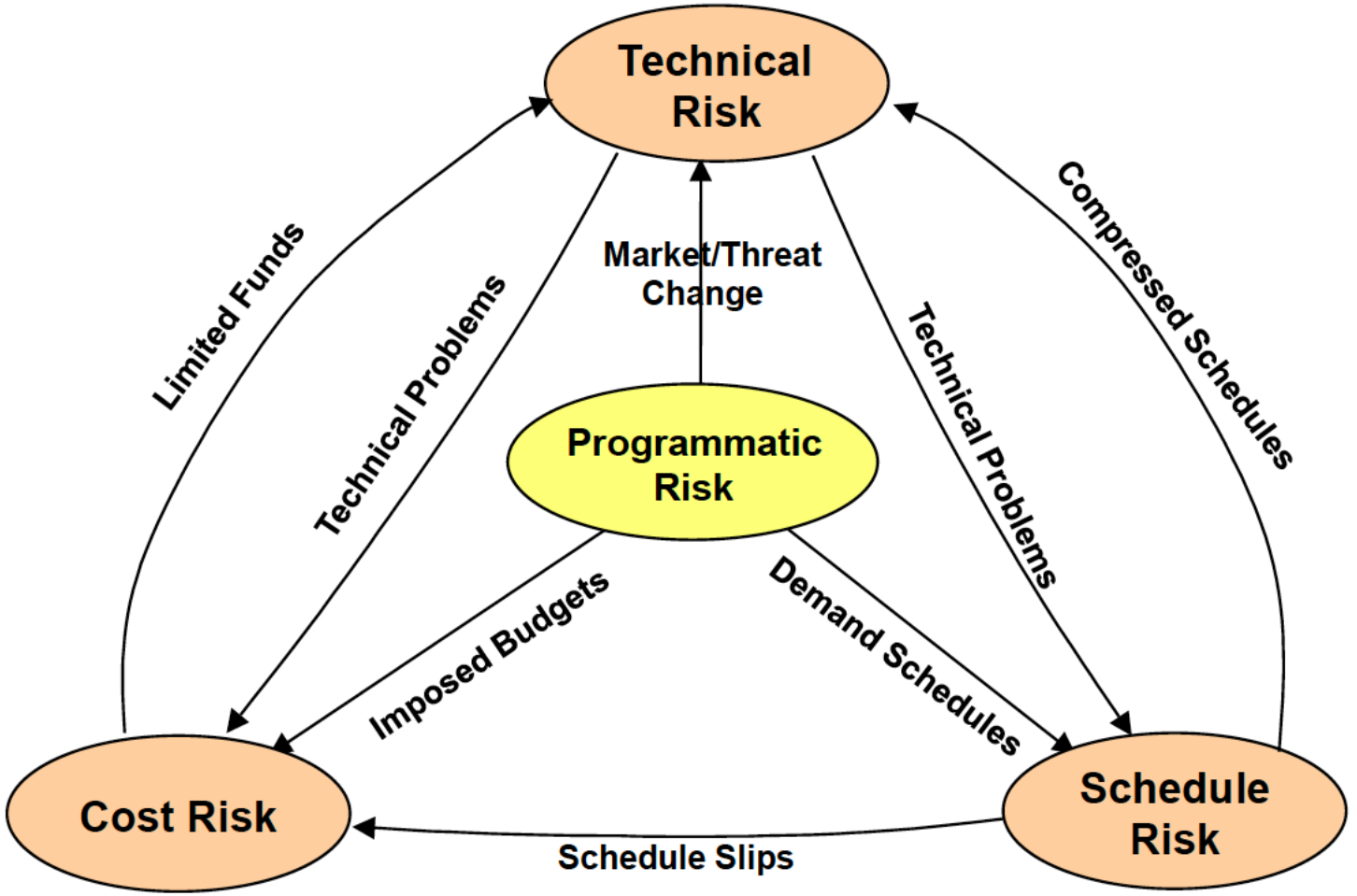
What is Risk?

- Risk is a measure of future uncertainties in achieving program technical performance goals within defined cost and schedule constraints
  - Risks can be associated with all aspects of a technical effort, e.g., threat, technology maturity, supplier capability, design maturation, performance against plan, etc., as these aspects relate within the systems structure and with interfacing products.
- Risks have three components:
  1. Future root cause
  2. Probability or likelihood of that future root cause occurring
  3. Consequences (or effect) of that future occurrence

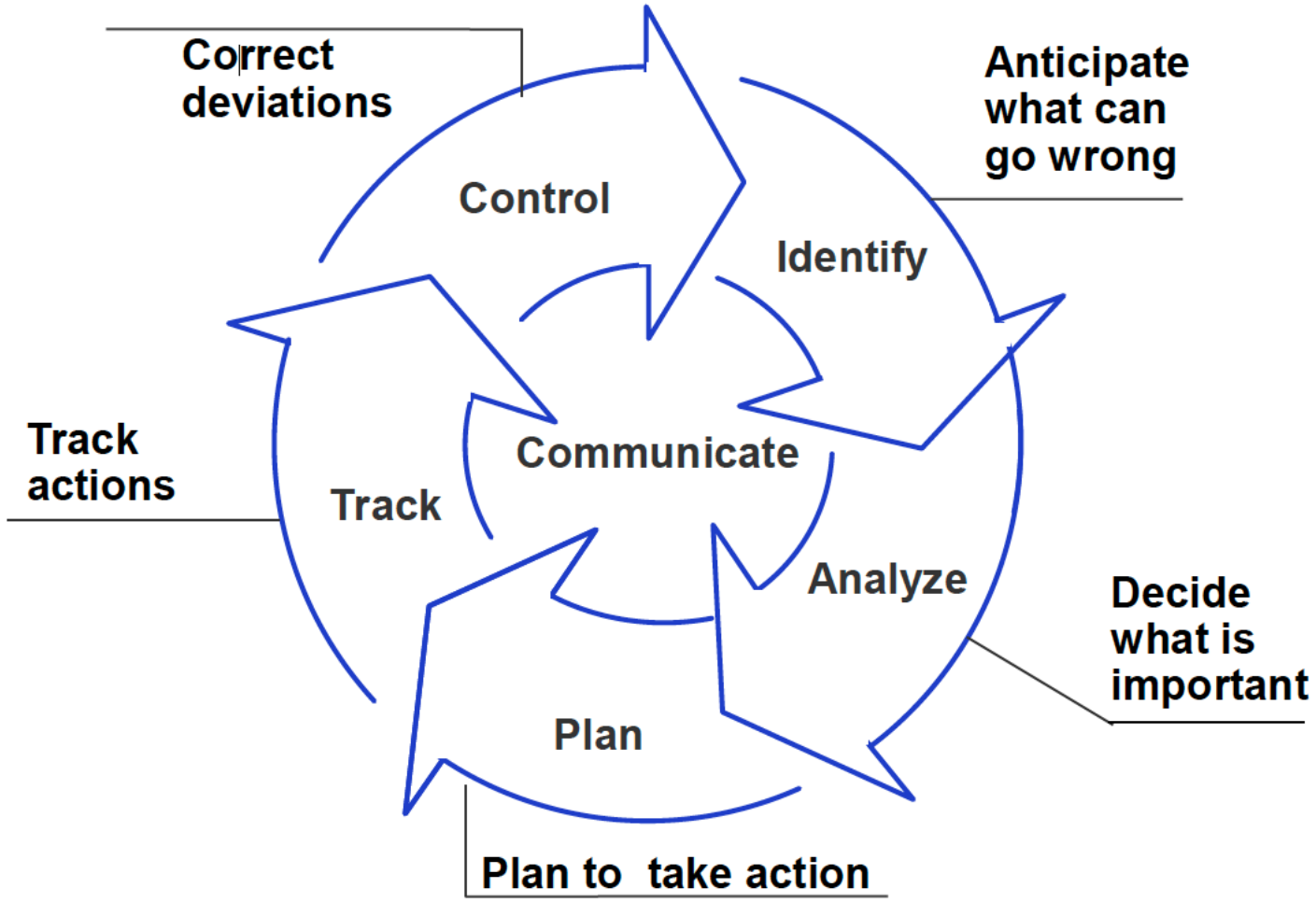
Layers of Risk Model (e.g. for Mars Missions)



Risk Categories – “Iron” Triangle



A Risk Management Framework



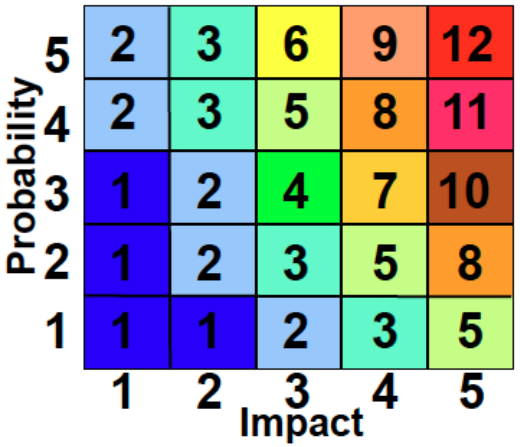
Risk ID/Assessment



- Brainstorm Risks
  - Probability that a particular event will occur
  - Impact or Consequence if the event does indeed occur
- Aggregate Into Categories
  - Rule of Thumb Limit @ N<=20
- Score (Based on Opinion & Data)
- Involve All Stakeholders

Risk ID/Assessment

Attribute: Probability		
Level	Value	Criteria
5	Near certainty	Everything points to this becoming a problem, always has
4	Very likely	High chance of this becoming a problem
3	Likely (50/50)	There is an even chance this may turn into a problem
2	Unlikely	Risk like this may turn into a problem once in awhile
1	Improbable	Not much chance this will become problem



Attribute: Impact				
Level	Value	Technical Criteria	Cost Criteria	Schedule Criteria
5	Catastrophic	Can't control the vehicle OR Can't perform the mission	> $10 Million	Slip to level I milestones
4	Critical	Loss of mission, but asset recoverable in time	$ 10 M ≤ X < $ 5 Million	Slip to level II milestones
3	Moderate	Mission degraded below nominal specified	$ 5 M ≤ X < $ 1 Million	Slip to level III milestones
2	Marginal	Mission performance margins reduced	$ 1 M ≤ X < $ 100 K	Loss of more than one month schedule margin
1	Negligible	Minimum to no impact	Minimum to no impact	Minimum to no impact

Requirement Change

Outline

- Definition
  - Challenges
  - Requirement Evolution
- Design Structure Matrix (DSM)
  - Characteristics, Taxonomy, Components, Integration Analysis
  - People-based, Activity-based, and Parameter-based DSM
- Requirement Topics
  - Topic modeling
  - Selected Results

Requirements Change Management (1 of 2)

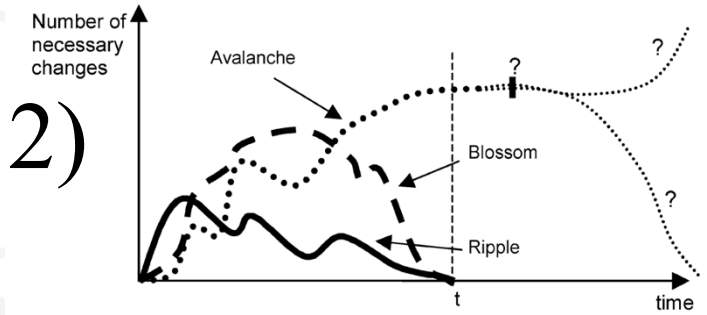
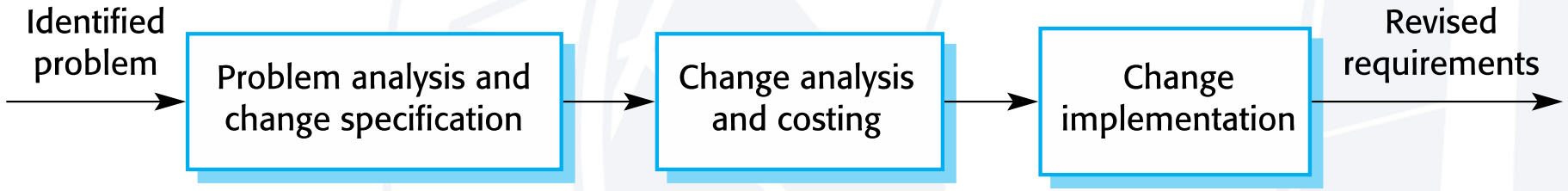


Fig. 17 Change effort and system behaviour

- Deciding if a requirements change should be accepted
  - Problem analysis and change specification
    - During this stage, the problem or the change proposal is analyzed to check that it is valid. This analysis is fed back to the change requestor who may respond with a more specific requirements change proposal, or decide to withdraw the request.
  - Change analysis and costing
    - The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements. Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.
  - Change implementation
    - The requirements document and, where necessary, the system design and implementation, are modified. Ideally, the document should be organized so that changes can be easily implemented.

Requirements Change Management (2 of 2)



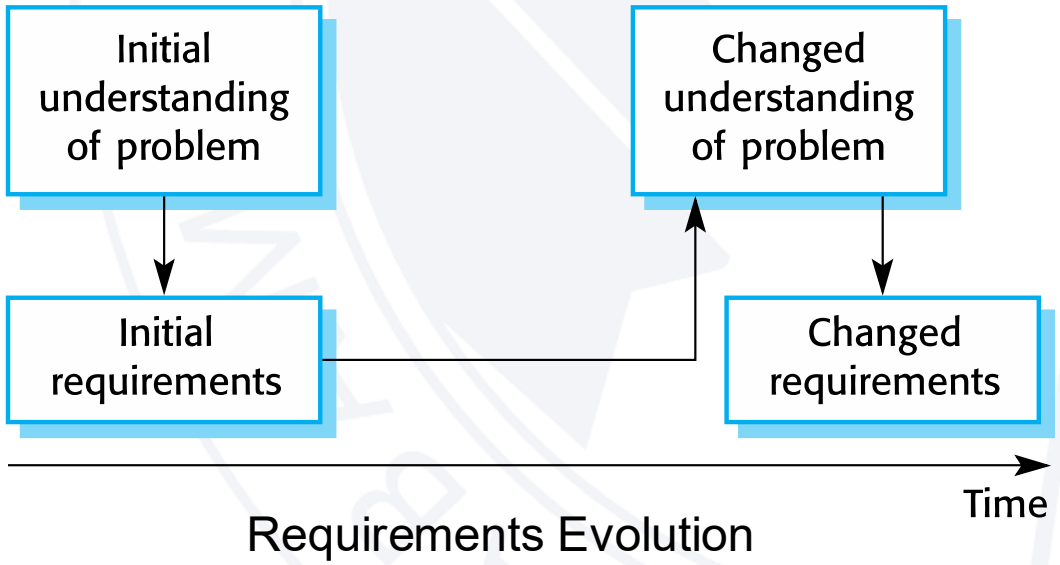
Changing Requirements (1 of 2)

- The business and technical environment of the system always changes after installation.
  - New hardware may be introduced, it may be necessary to interface the system with other systems, business priorities may change (with consequent changes in the system support required), and new legislation and regulations may be introduced that the system must necessarily abide by.

- The people who pay for a system and the users of that system are rarely the same people.
  - System customers impose requirements because of organizational and budgetary constraints. These may conflict with end-user requirements and, after delivery, new features may have to be added for user support if the system is to meet its goals.

Changing Requirements (2 of 2)

- Large systems usually have a diverse user community, with many users having different requirements and priorities that may be conflicting or contradictory.
  - The final system requirements are inevitably a compromise between them and, with experience, it is often discovered that the balance of support given to different users has to be changed.



Requirements Evolution

Requirements Management

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development.
- New requirements emerge as a system is being developed and after it has gone into use.
- You need to keep track of individual requirements and maintain links between dependent requirements so that you can assess the impact of requirements changes. You need to establish a formal process for making change proposals and linking these to system requirements.

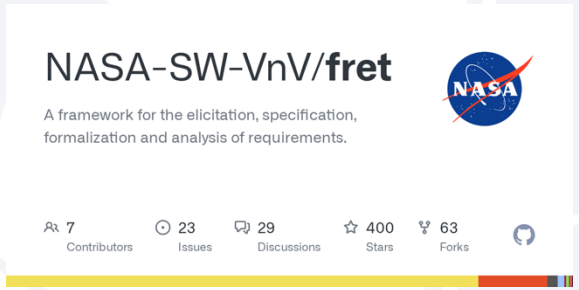




Figure 1: FRET dashboard

Requirements Management Planning

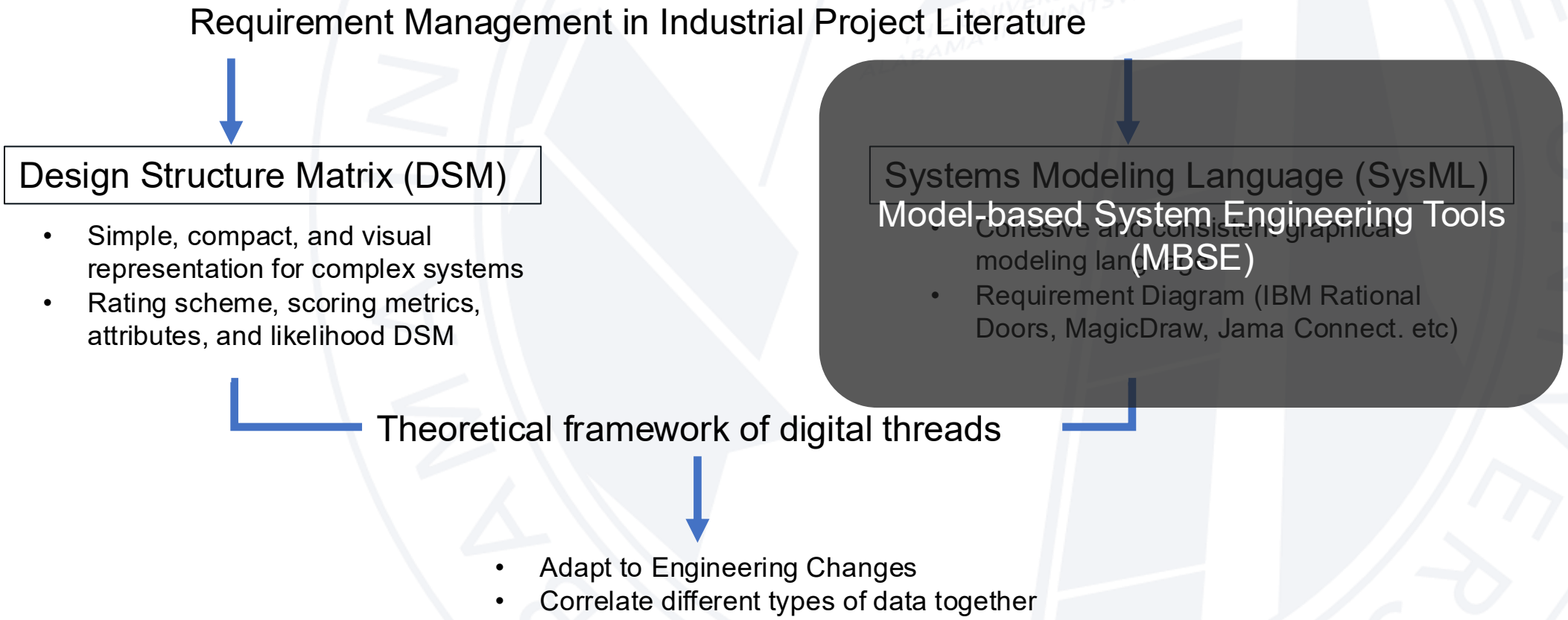
- Establishes the level of requirements management detail that is required.
- Requirements management decisions:
  - Requirements identification Each requirement must be uniquely identified so that it can be cross-referenced with other requirements.
  - A change management process This is the set of activities that assess the impact and cost of changes.
  - Traceability policies – these policies define the relationships between each requirement and between the requirements and the system design that should be recorded.
  - Tool support - tools that may be used range from specialist requirements management systems to spreadsheets and simple database systems.

Outline

- Definition
  - Challenges
  - Requirement Evolution
- Design Structure Matrix (DSM)
  - Characteristics, Taxonomy, Components, Integration Analysis
  - People-based, Activity-based, and Parameter-based DSM
- Requirement Topics
  - Topic modeling
  - Selected Results

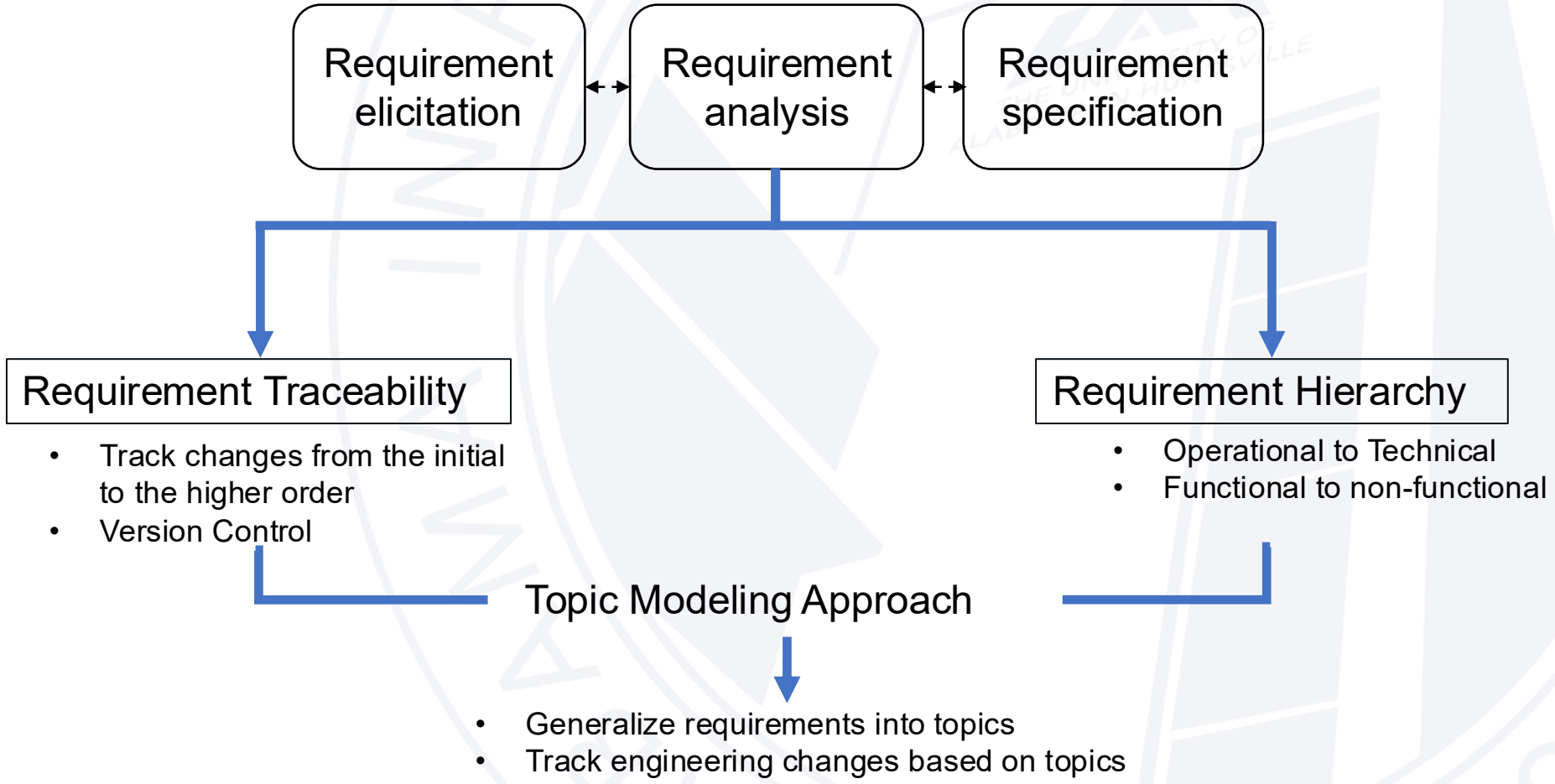
Literature Review

Current Solutions:

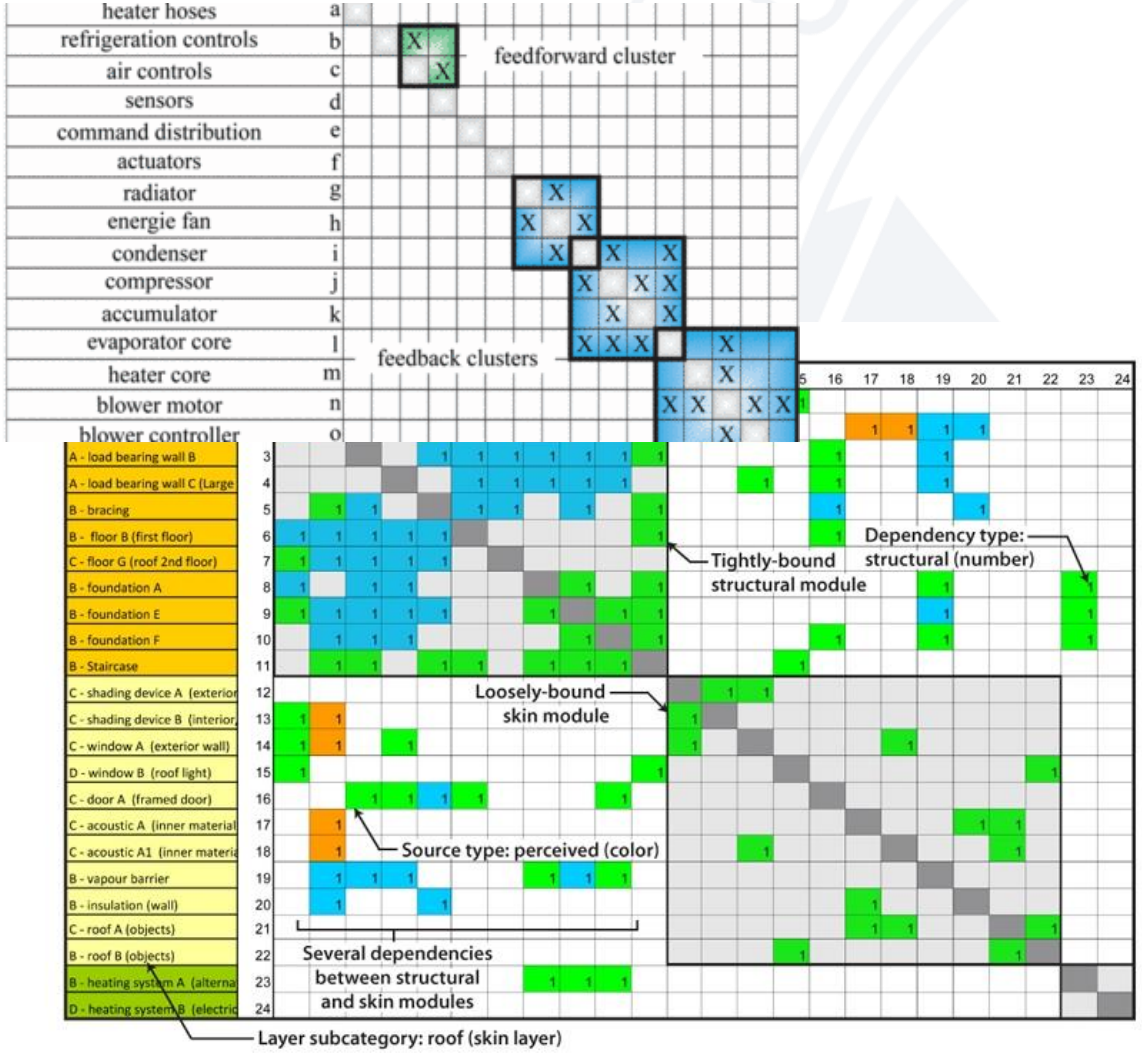


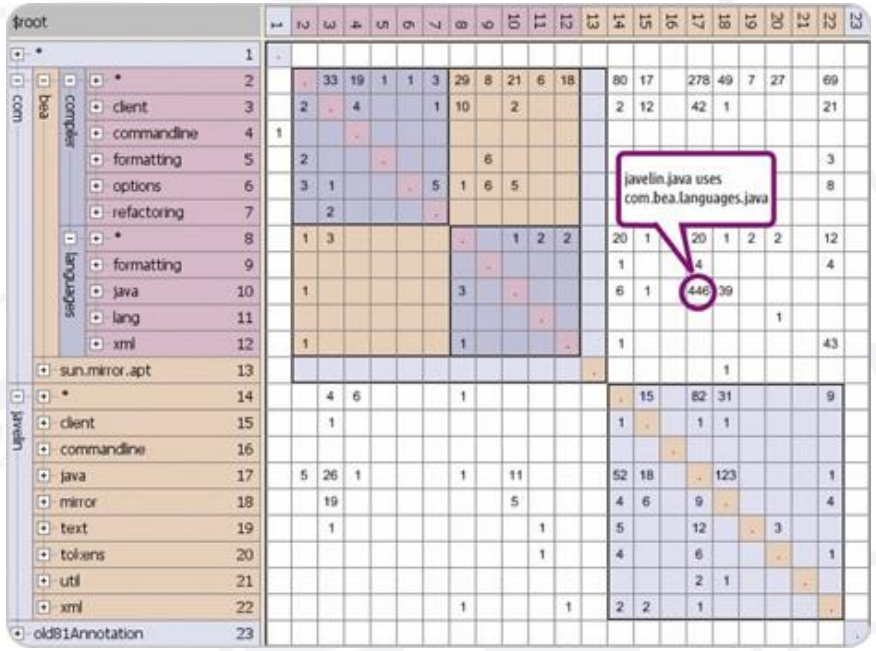
Literature Review

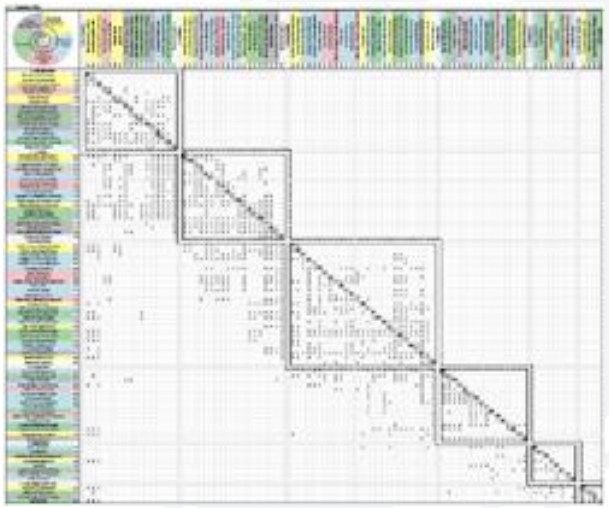
Requirement Management: (Morkos, 2012, Jiao, 2006)



Design Structure Matrix (DSM)





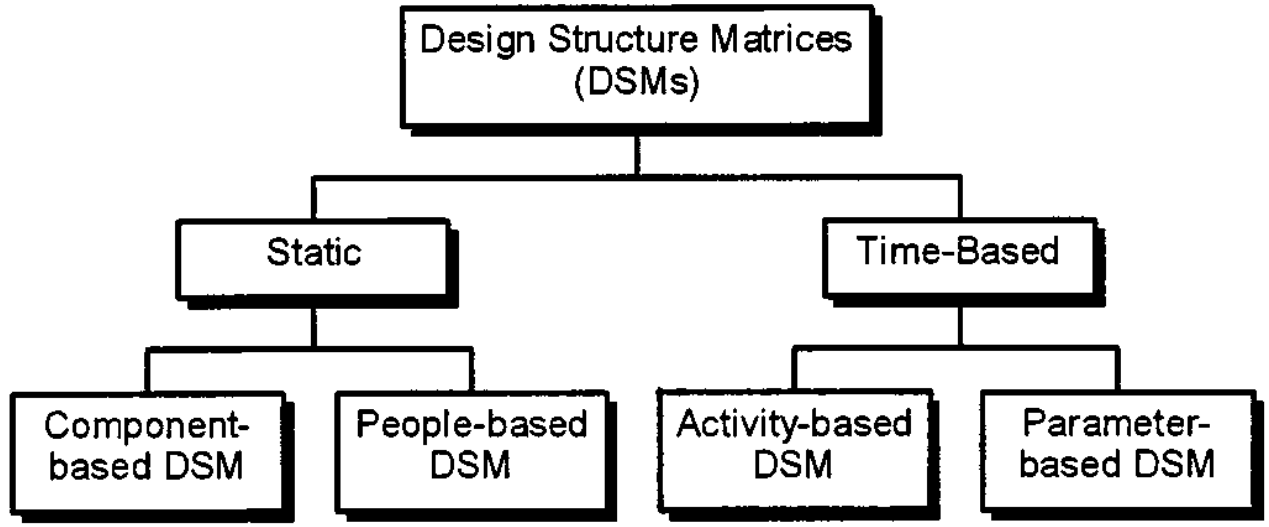


Characteristics of DSM

- Square/least Matrix
- Subsystems represented in some order (perhaps arbitrary)
- Relationships captured through couplings (either X, Dot, Shaded Box, or a number representing a strength or importance of the coupling)
- Feedbacks indicate iteration
- Couplings are assumed to be explicitly identifiable

Taxonomy for a DSM

- Component-based (Product) - Static
- People-based (Organization) - Static
- Activity-based (Process) – Time-based
- Parameter-based (Low Level Process) – Time-based



Component-based DSM

Modeling interactions of components requires:

1) Decompose system into elements;
2) Understand and document interactions between elements; and
3) Analyze potential reintegration of elements via clustering (integration analysis).

One issue: Do we fully understand how one components interacts with another?

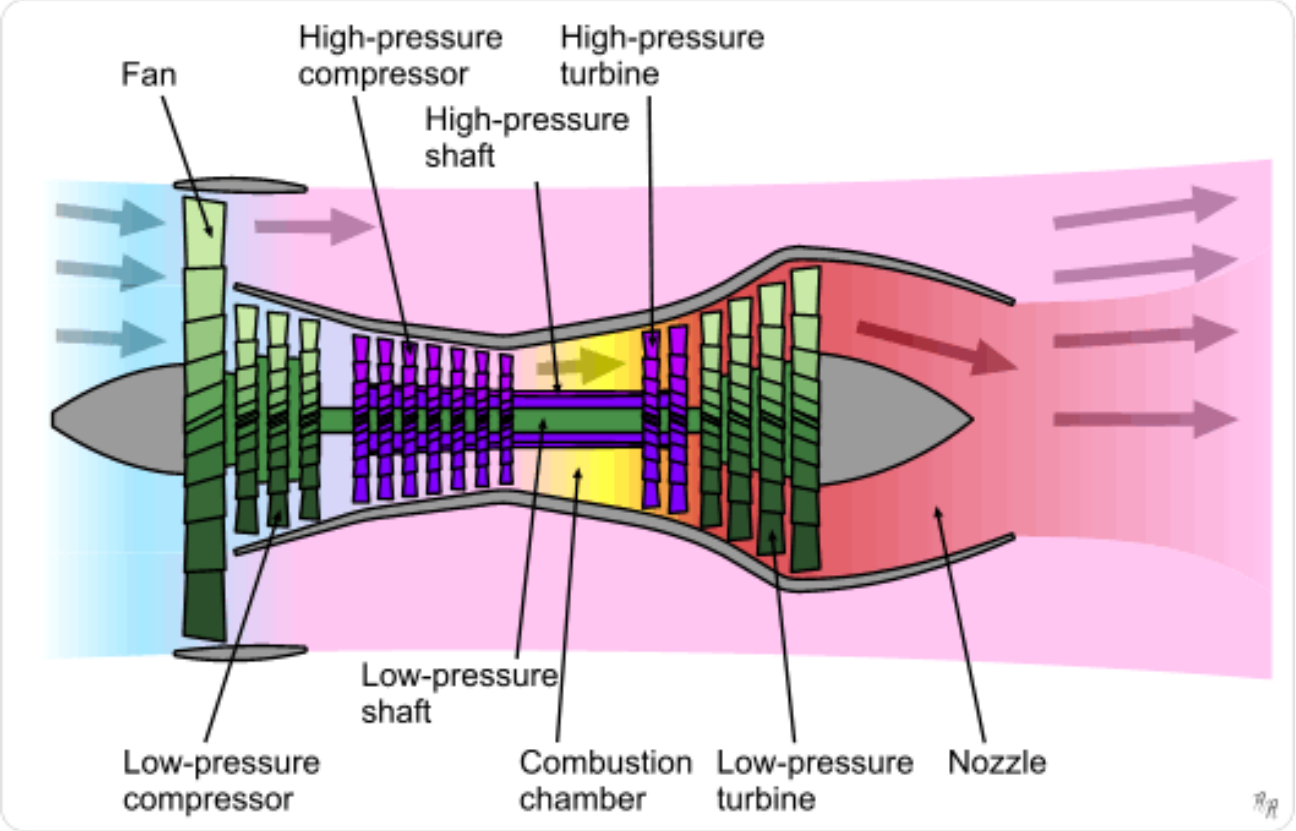
Component DSM

Simple Taxonomy of System Element Interactions (could be more) – Table I

Spatial	Associations of physical space and alignment, needs for adjacency or orientation between two elements
Energy	Needs for energy transfer/exchange between two elements (e.g. power supply)
Information	Needs for data or signal exchange between two elements
Material	Needs for material exchange between two elements

Component DSM

Simplified Representation of Components of a Jet Engine





What would the DSM look like?

Take a moment and come up with one

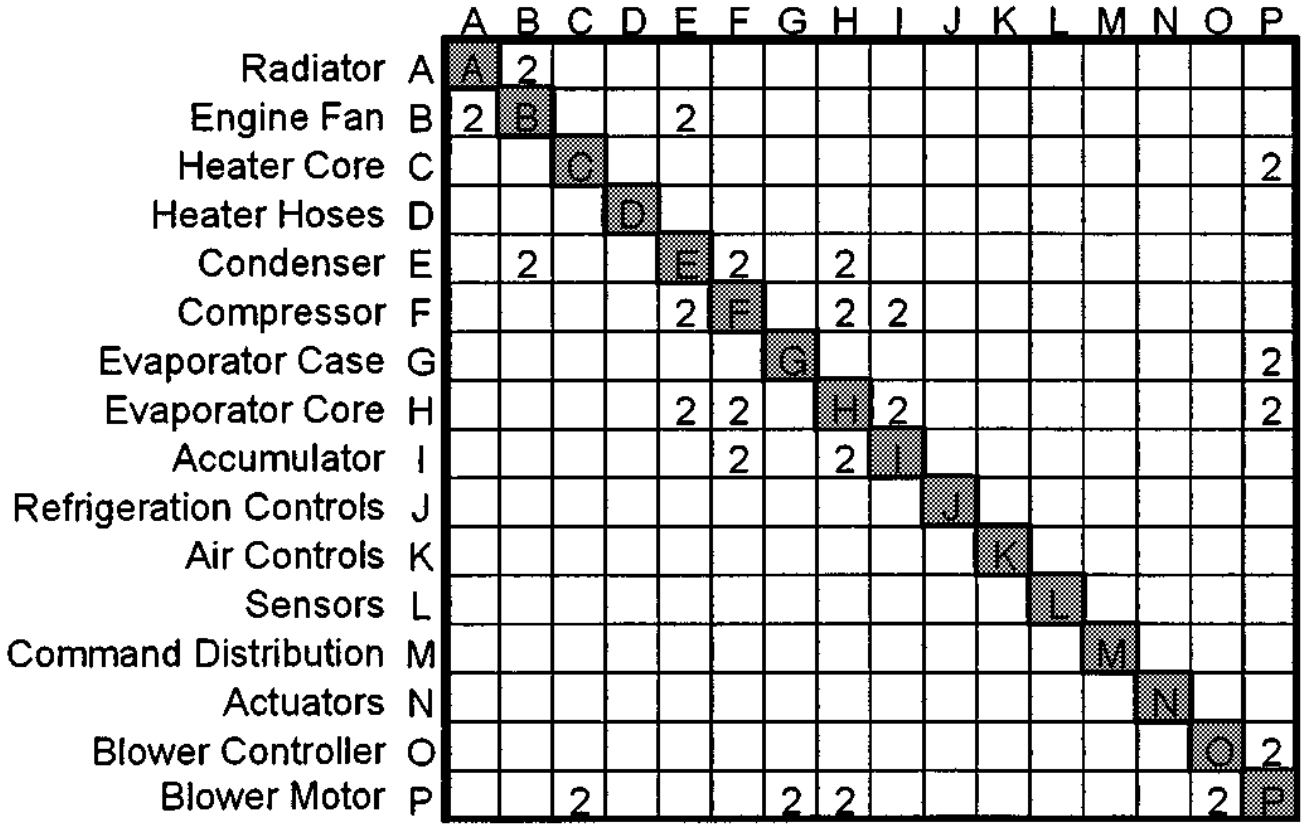
Component DSM

Spatial Interaction Quantification (one example) – Table II

Required	+2	Associations of physical space and alignment, needs for adjacency or orientation between two elements
Desired	+1	Needs for energy transfer/exchange between two elements (e.g. power supply)
Indifferent	0	Needs for data or signal exchange between two elements
Undesired	-1	Needs for material exchange between two elements
Detrimental	-2	Physical adjacency must be prevented to achieve functionality

Go back to your jet engine DSM and now insert spatial interaction quantifications

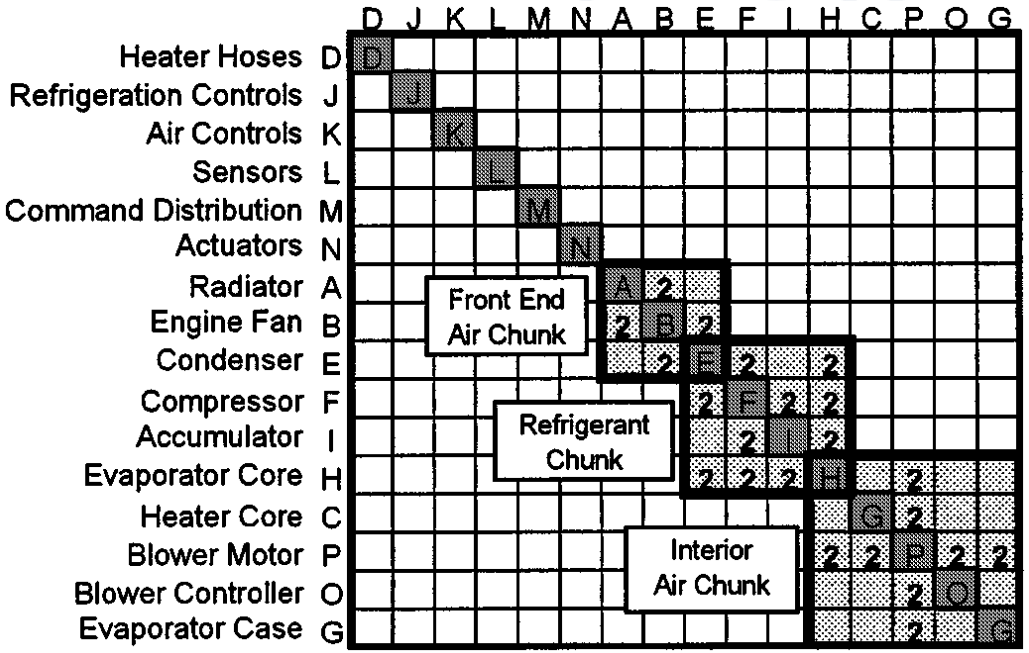
Component-based DSM: Materials Interactions for Climate Control System



If we trust these interactions and weightings, how can we use this DSM?

Take the next step…

Design Structure Matrix (DSM)



Front-End Air Cluster: set of components at the fore of engine compartment involved with heat transfer to exterior air.

Refrigerant Cluster: air conditioning components

Interior Air Cluster: components at the front of the passenger compartment involved in modifying interior air temperature.

Significance of overlapping elements? Shows where integration efforts should be placed.

Remaining components can be placed in convenient clusters as desired.

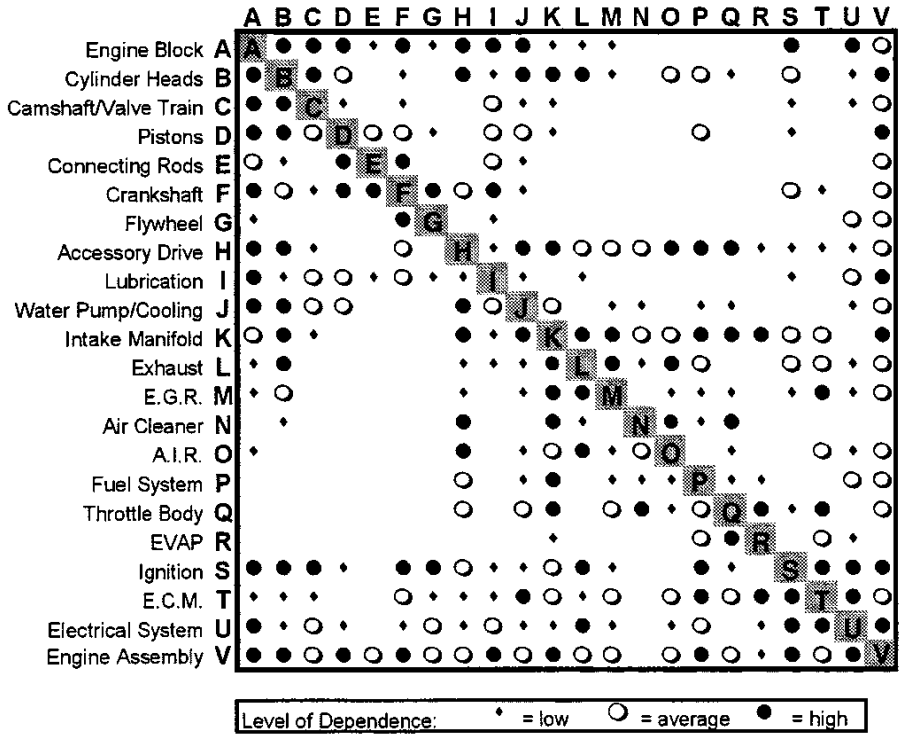
DSM - Integration Analysis

- The term Integration Analysis is used to describe the act of clustering
- It supports modularization
- It demonstrates rationale behind architecting decisions
- It fosters innovation by demonstrating where opportunities might lie

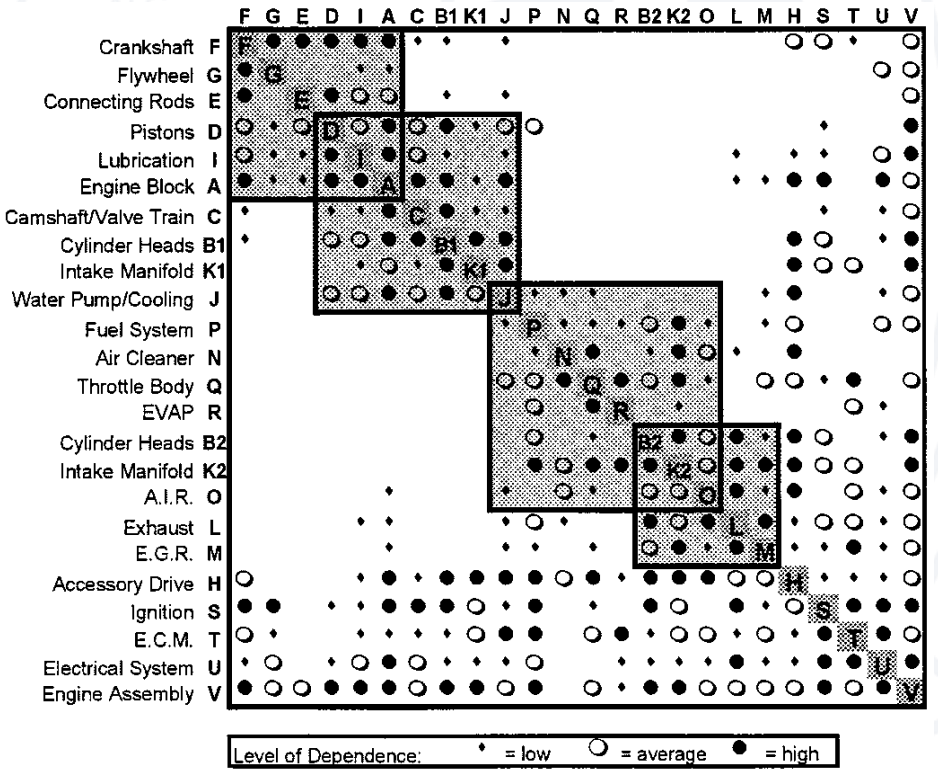
People-based DSM

- Modeling an organization requires:
  1. Decompose organization into teams with specific responsibilities, functions, and/or assignments
  2. Document interactions between teams
  3. Analyze the clustering of teams into ‘metateams’
- One issue: team members may see their roles differently and therefore provide interaction data that is not consistent with others

People-based (Teams) DSM



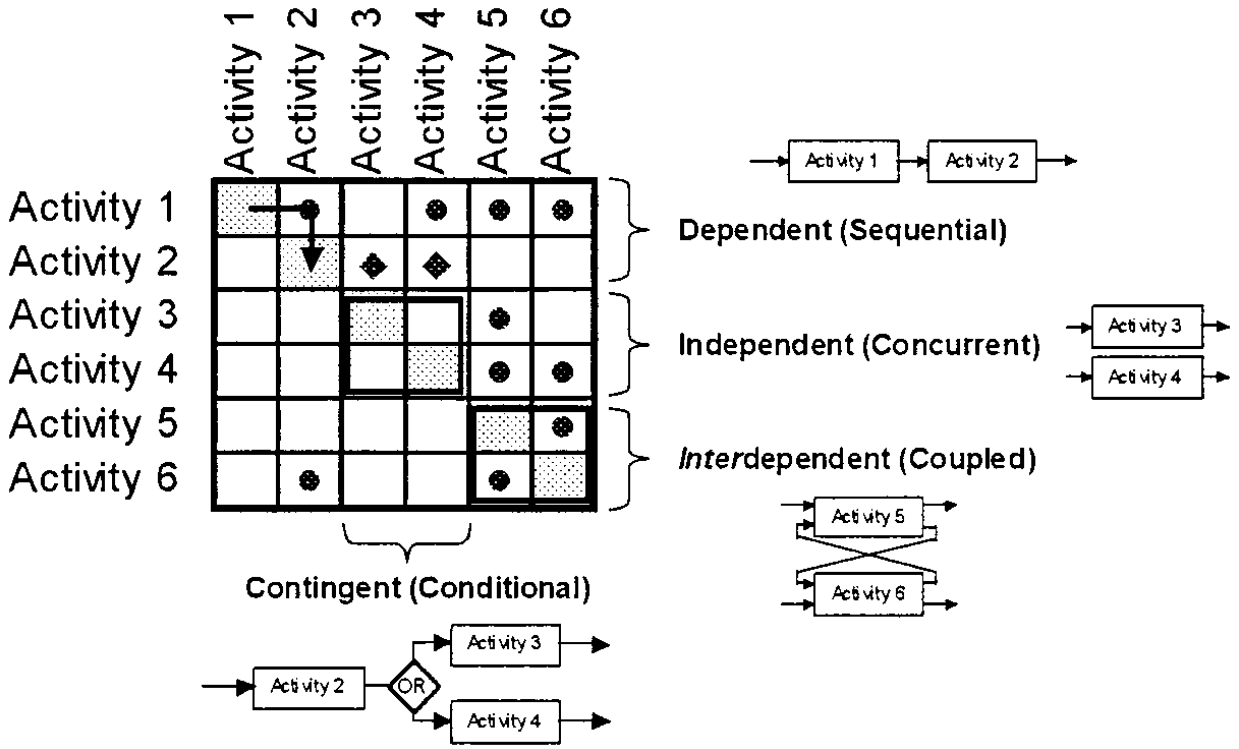
Automobile Engine Development Organization: frequency and direction of information flow between product development teams (PDTs) captured



Metateam Organization based on cluster analysis of PDT interactions

Overlaps show participation in multiple metateams. B and K twice - participate in 3 metateams

Activity-based (Task) DSM



Metateam Organization based on cluster analysis of PDT interactions

Overlaps show participation in multiple metateams. B and K twice - participate in 3 metateams

Activity-based DSM

- Modeling a process requires:

    1. Decompose the process into activities;

    2. Document the information flow among the activities (their integration); and

    3. Analyze the sequencing of the activities into a maximally feed-forward process flow.

- One issue: How detailed do we need to get? Different stakeholders require different degrees of fidelity/decomposition.

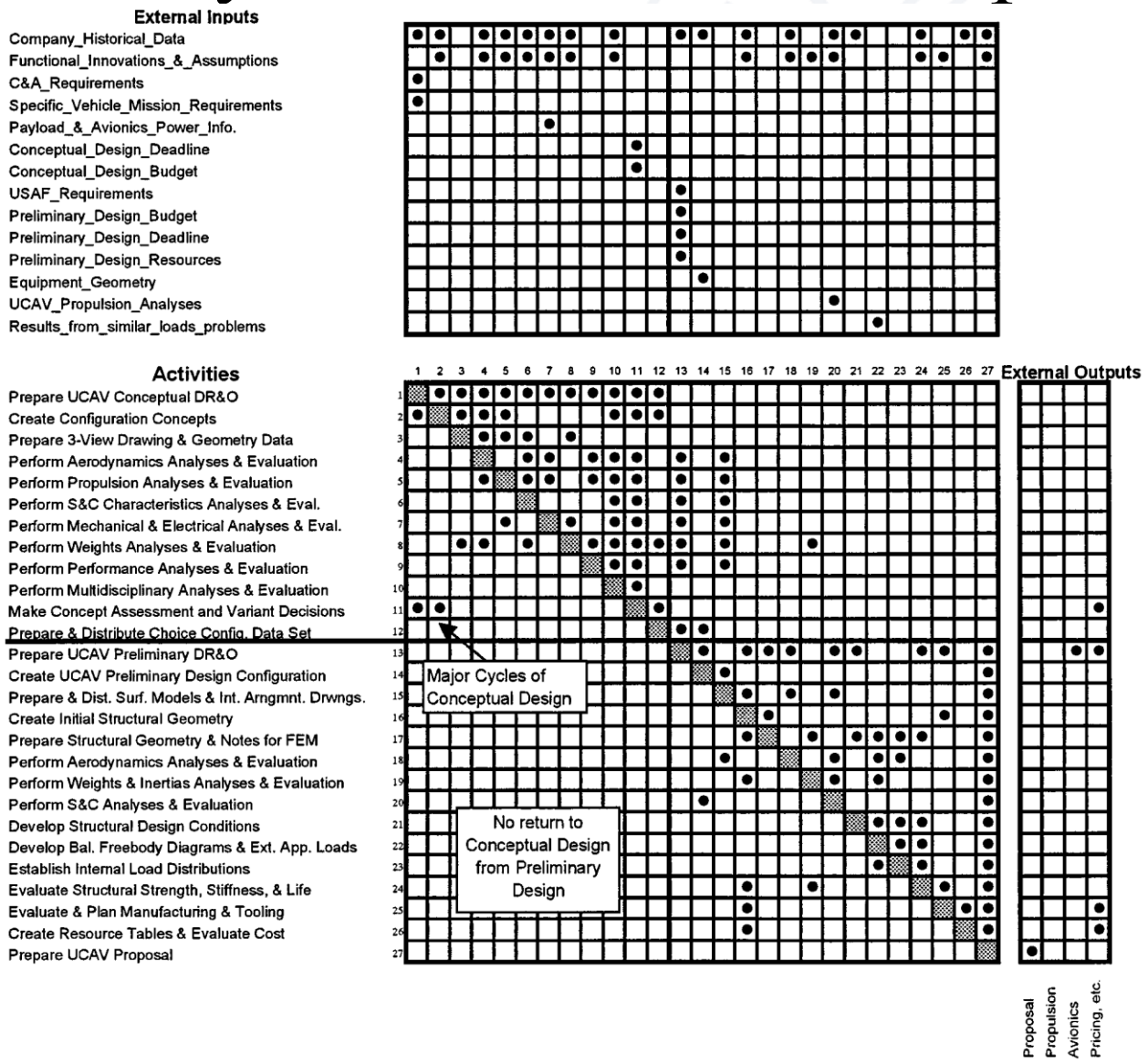
Activity-based DSM

- Built by finding people knowledgeable about each activity and getting answers to questions like:

    1. What outputs or products must activity produce?
    2. To where do these outputs go (another activity or outside the process)?
    3. What inputs does the activity need?
    4. From where do these inputs come (another activity or outside the process)?

All very subjective but necessary for any large-scale, complex engineered system design process.

Activity-based DSM – Example 1

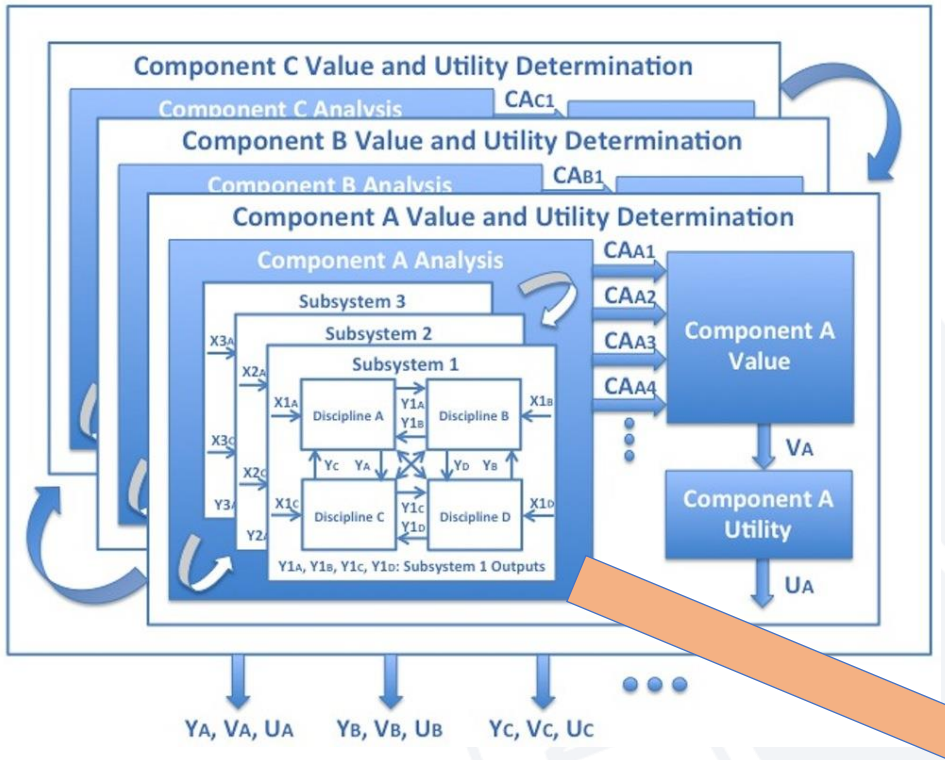


External and Internal Inputs and Outputs can be specified separately – particularly important as different teams build different Process DSMs that must be linked.

Activity/Process DSMs:

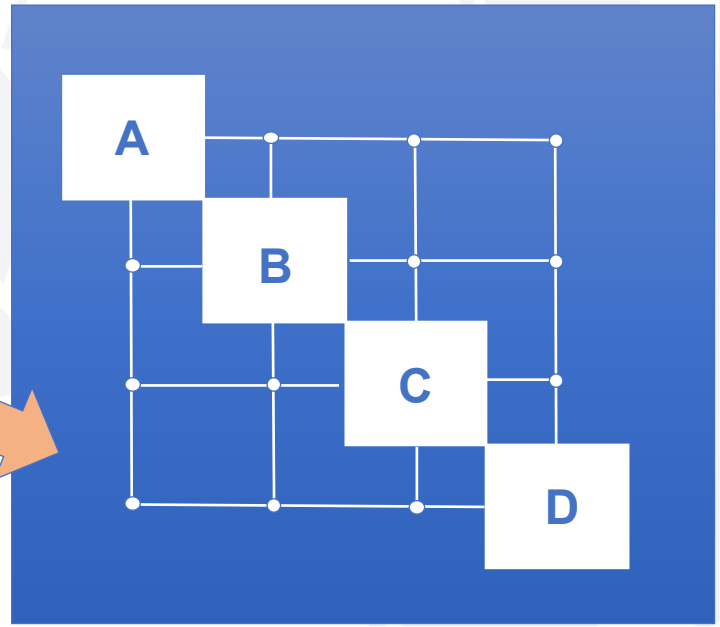
- Aggregation (combining activities)
- Decomposition (breaking up activities)
- Tearing (identify which interactions can be ‘torn’)

Parameter-based DSM



What are the most significant considerations?

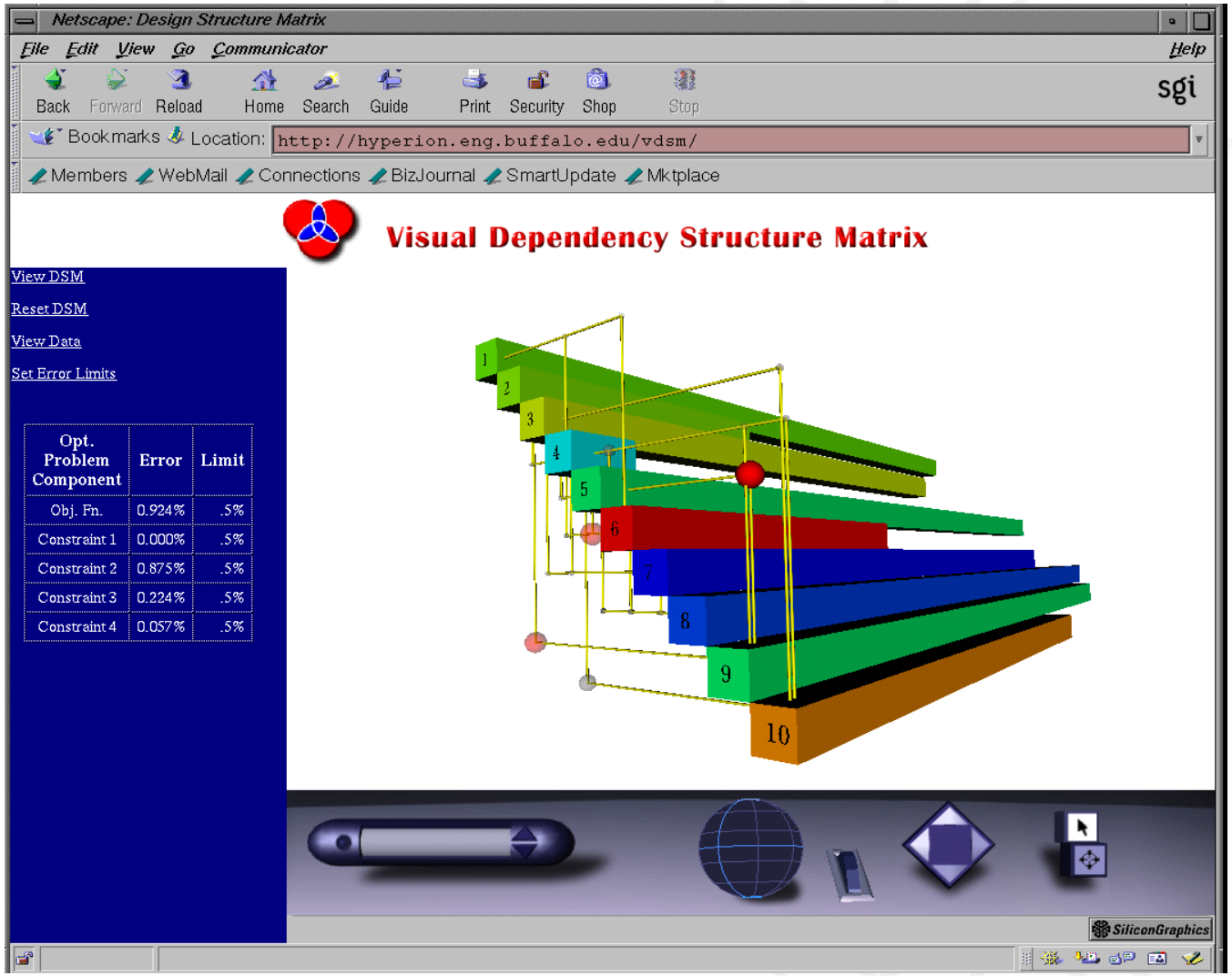
The greatest relevance for MDO is for the coupled analysis of the disciplinary subsystem analyses



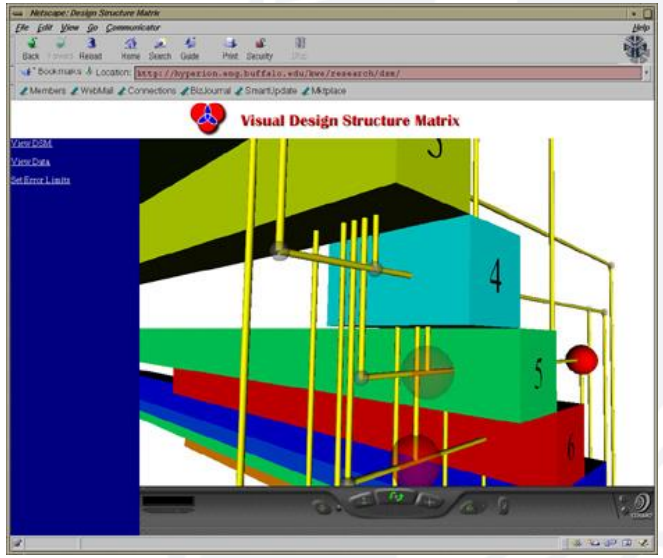
Facilitating Trades using DSM

- Trade-off Accuracy and Efficiency
  - Subsystem Level Fidelity Trade-offs – Allow different degrees of fidelity (error) within subsystem analyses in order to increase system level efficiency
  - System Scheduling and Reduction – Optimally order participating subsystem analyses and reduce numbers of couplings (feedbacks) without significant loss in accuracy
- Trade-space Exploration
  - Use visualization to capture the large amounts of data in subspaces to enable improved and more insightful decision-making

Design Structure Matrix (DSM)

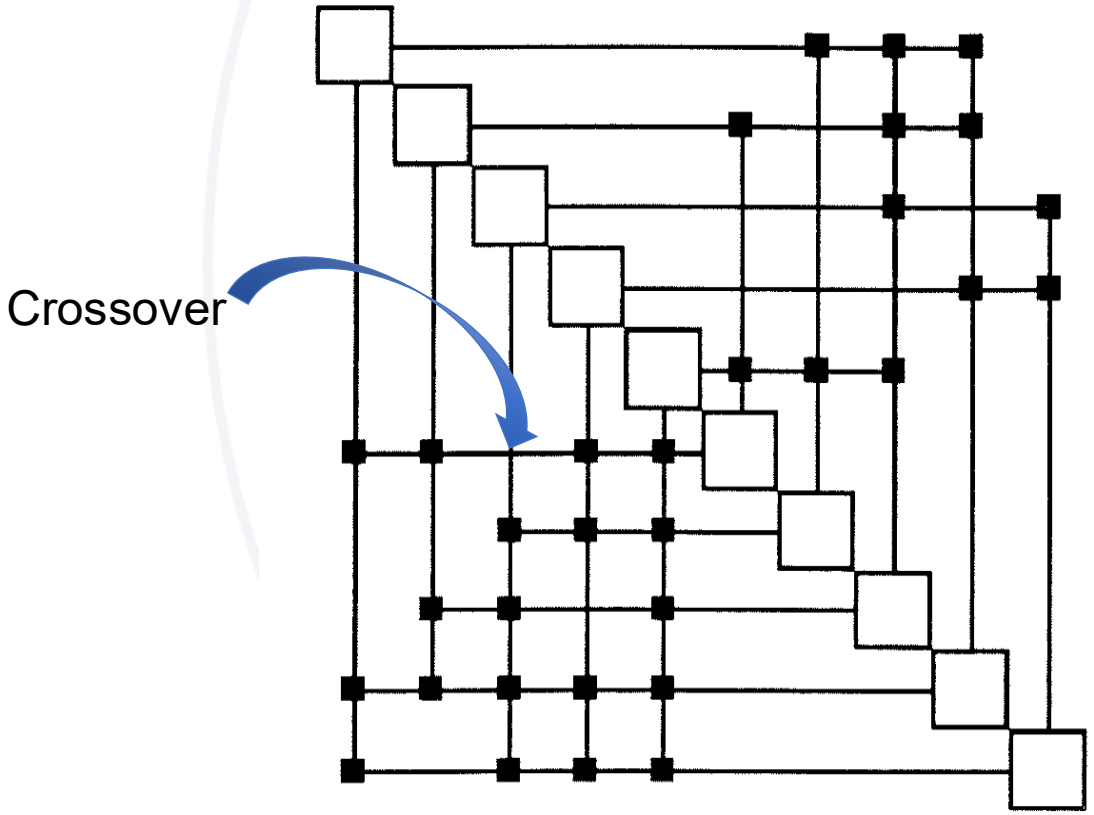


Reducing couplings, using underlying error modeling, can be achieved using sensitivity based coupling strength information



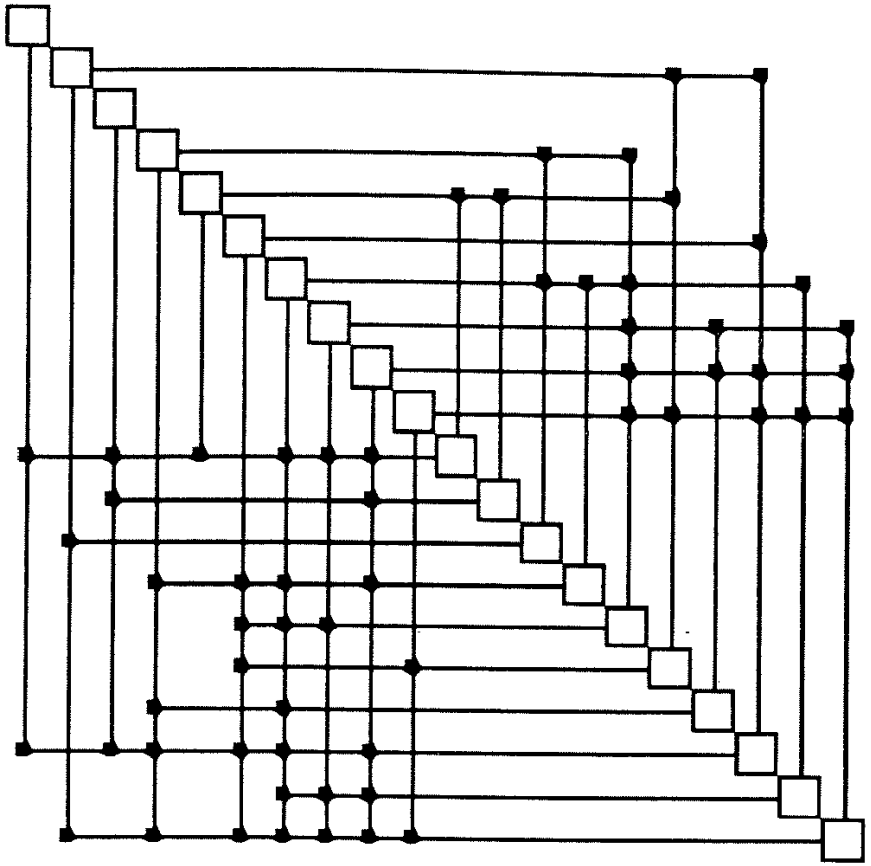
DSM – Feedbacks and Crossovers

- What is the implication of a crossover?



DSM – Feedbacks and Crossovers

- A 20 subsystem system…
  - Does it make sense to minimize crossovers?



Cyclomatic Complexity

- developed by Thomas McCabe - a metric that measures the number of linearly independent paths through a program's source code

M = E - N + 2P

where

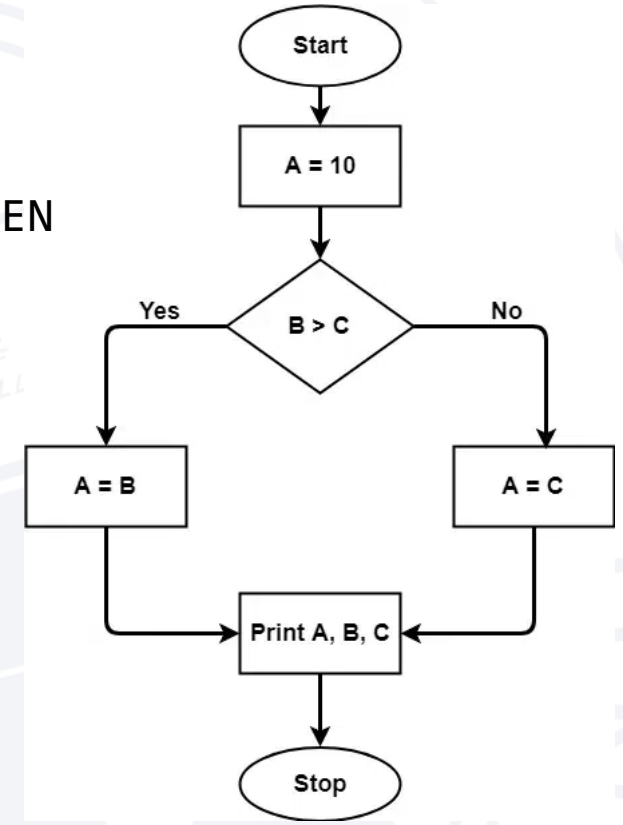
E = the number of edges in the control flow graph
N = the number of nodes in the control flow graph
P = the number of connected components

M = E - N + P (exit point is directly connected back to the entry point)

where

E = the number of edges in the control flow graph
N = the number of nodes in the control flow graph
P = the number of connected components

```
A = 10
    IF B > C THEN
       A = B
    ELSE
       A = C
    ENDIF
Print A
Print B
Print C
```



How to Calculate Cyclomatic Complexity?

Cyclomatic complexity is 7-7+2 = 2.

Cyclomatic Complexity

- Determining the independent path executions thus proven to be very helpful for Developers and Testers.
- It can make sure that every path has been tested at least once.
- Thus help to focus more on uncovered paths.
- Code coverage can be improved.
- Risks associated with the program can be evaluated.
- These metrics being used earlier in the program help in reducing the risks.

```
int module1 (int x, int y) {
   while (x!=y){
      if(x>y)
         x=x-y,
      else y=y-x;
      }
   return x;
}
```

What is the Cyclomatic Complexity of the above module?

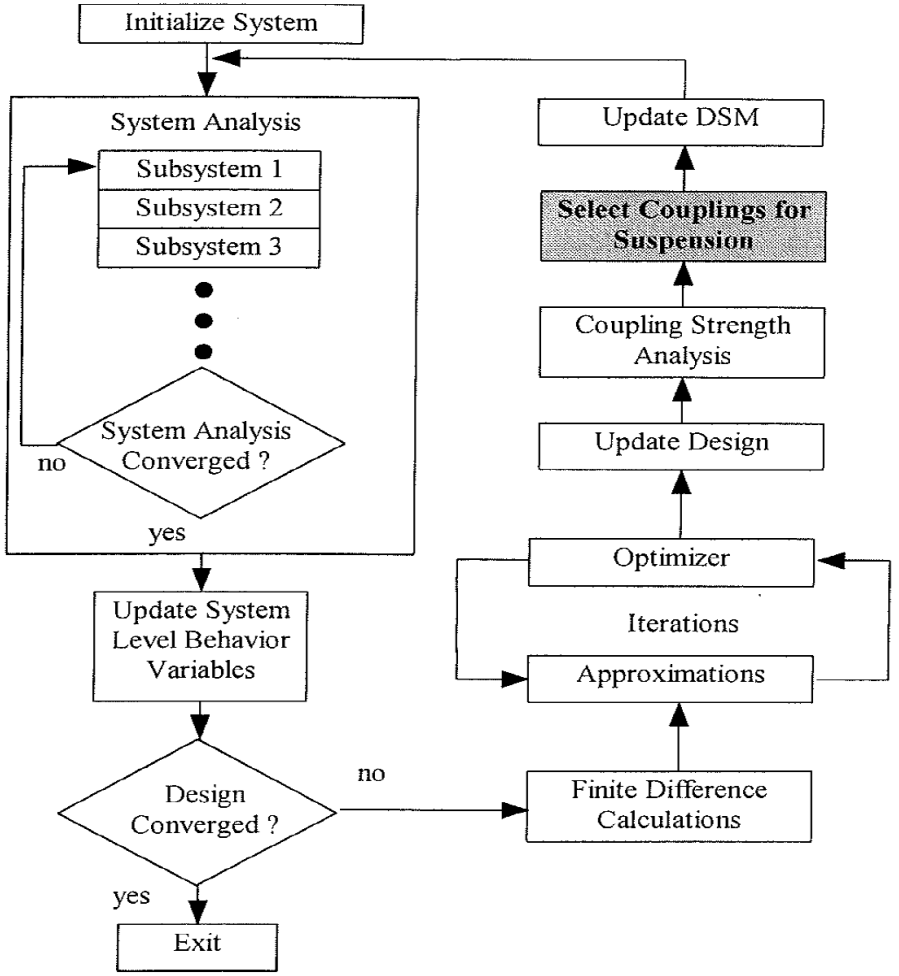
(A) 1

(B) 2

(C) 3

(D) 4

MDO Process


Initialize System
System Analysis
Subsystem 1
Subsystem 2
Subsystem 3
System Analysis Converged ?
no
yes
Update System Level Behavior Variables
Design Converged ?
no
yes
Exit
Finite Difference Calculations
Approximations
Iterations
Optimizer
Update Design
Coupling Strength Analysis
Select Couplings for Suspension
Update DSM

Outline

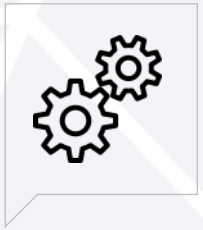
- Definition
  - Challenges
  - Requirement Evolution
- Design Structure Matrix (DSM)
  - Characteristics, Taxonomy, Components, Integration Analysis
  - People-based, Activity-based, and Parameter-based DSM
- Requirement Topics
  - Topic modeling
  - Selected Results

Requirements Documents:



Project 1:

- Pipe threading stations



Project 2:

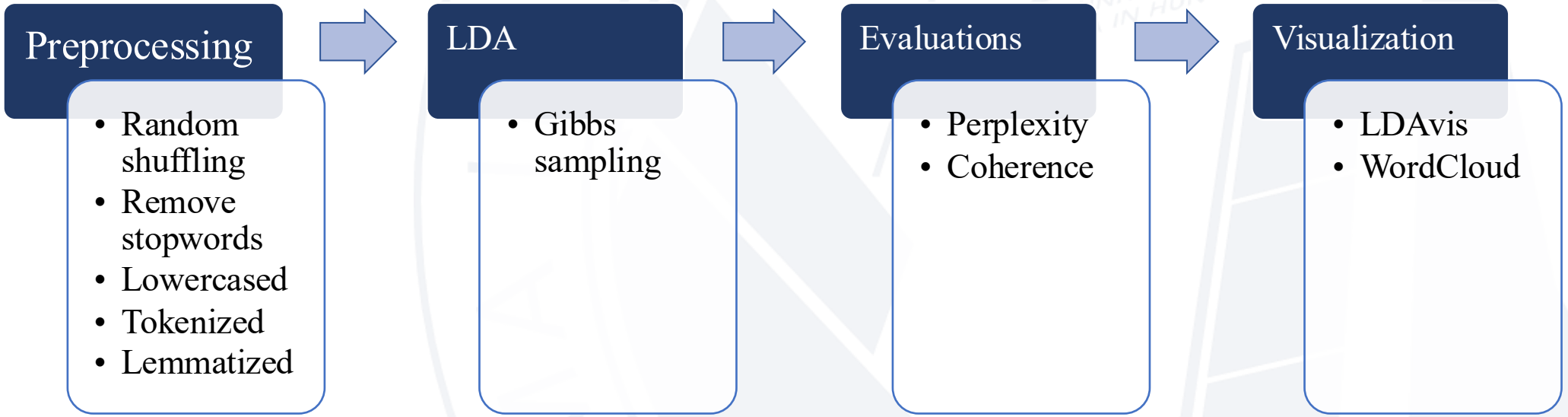
- Automatic spooling system



Project 3:

- Gas recirculation bypass flaps design

Methodology: Data Pipeline Framework



Chen, C., Mullis, J., & Morkos, B. (2021, August). A topic modeling approach to study design requirements. In International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (Vol. 85383, p. V03AT03A021). American Society of Mechanical Engineers.

Latent Dirichlet Allocation:

- Assumptions:
  - Bag-of-words
  - Each corpus contains a mixture of topics
  - All random variables are exchangeable (conditional independent and identical distributed)

- Parameter estimation: applied Gibbs sampling to estimate p($\boldsymbol{\theta}$, $\mathbf{z}$ | w, α, β),
  - Document-topic distribution $\boldsymbol{\theta}_{dk}$
  - Term-topic distribution Φ_{nk}

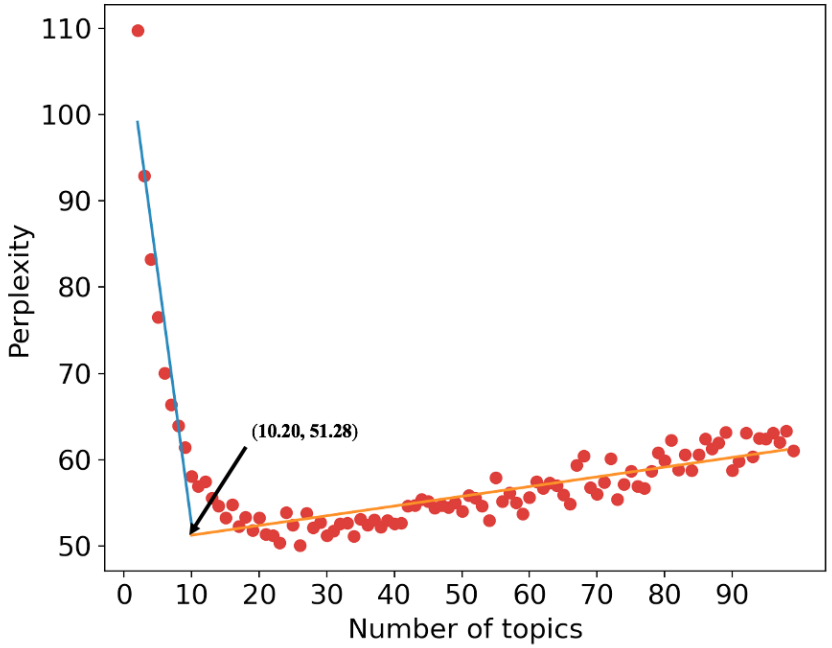
$$p(\mathcal{D}|\alpha,\beta) = \prod_{d=1}^{M} \int p(\theta_d|\alpha) \left(\prod_{n=1}^{N_d} \sum_{z_{dn}} p(z_{dn}|\theta_d) p(w_{dn}|z_{dn},\beta) \right) d\theta_d$$

(Blei et al., 2003)

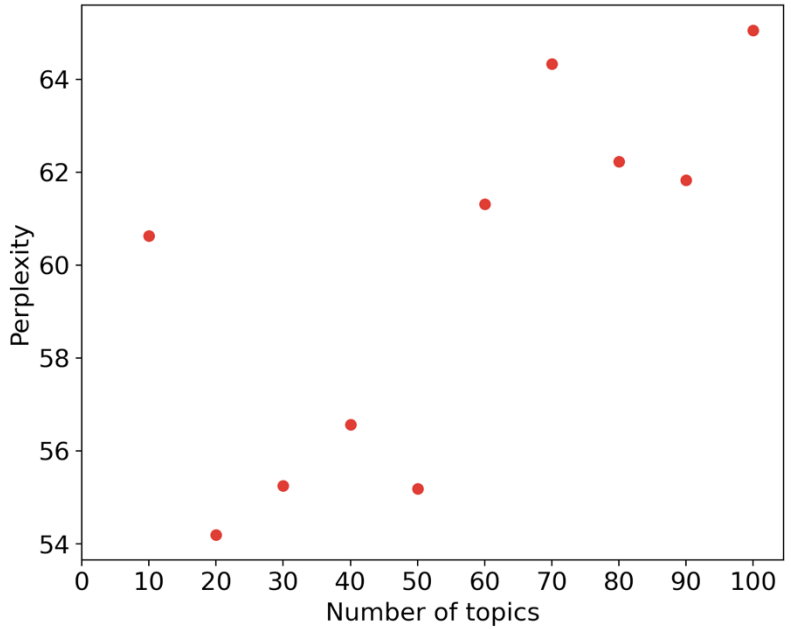
- Evaluations:
  - Perplexity: $P = e^{-\sum_{d=1}^{M} P(w_d) log P(w_d)}$
  - Coherence (C_v): An average value of cosine similarity, normalized pointwise mutual information (NPMI), and boolean sliding window (Röder et al., 2015)

Chen, C., Mullis, J., & Morkos, B. (2021, August). A topic modeling approach to study design requirements. In International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (Vol. 85383, p. V03AT03A021). American Society of Mechanical Engineers.

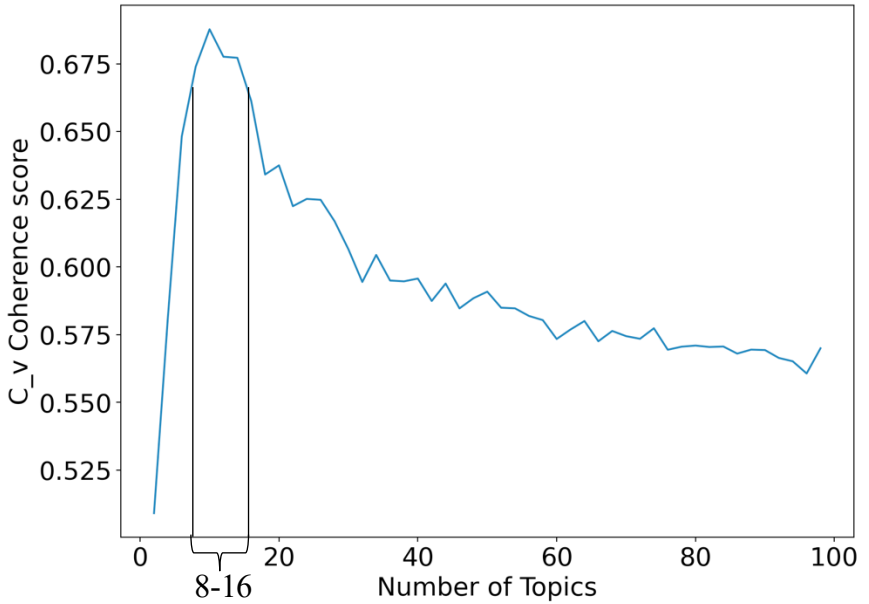
Selected Results:



L method



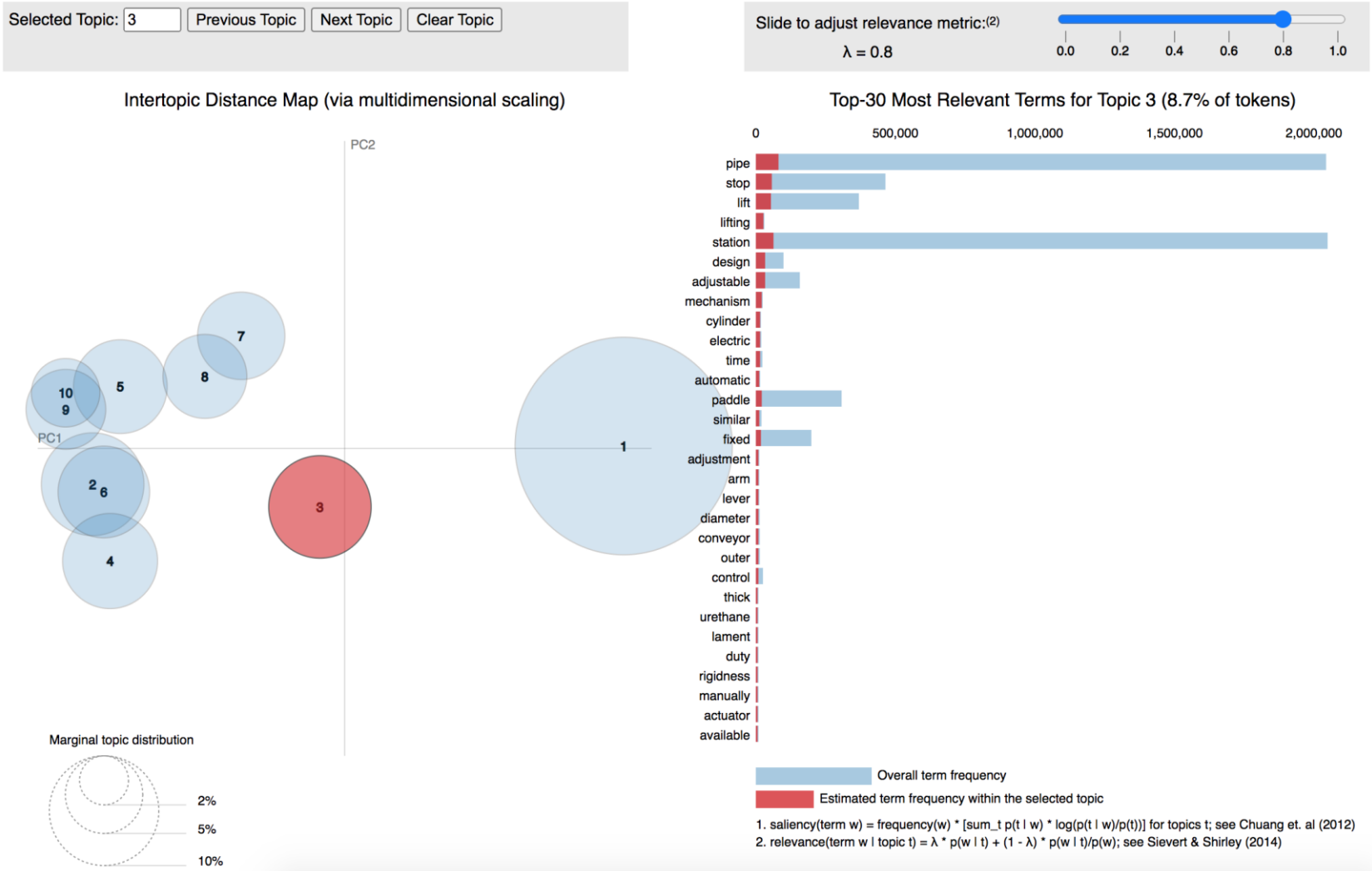
$\alpha = 10/k$ method



Meanings of coherence score:
0.3 = poor
0.4 = low
0.5-0.6 = okay
0.7 = nice
0.8+ = unlikely

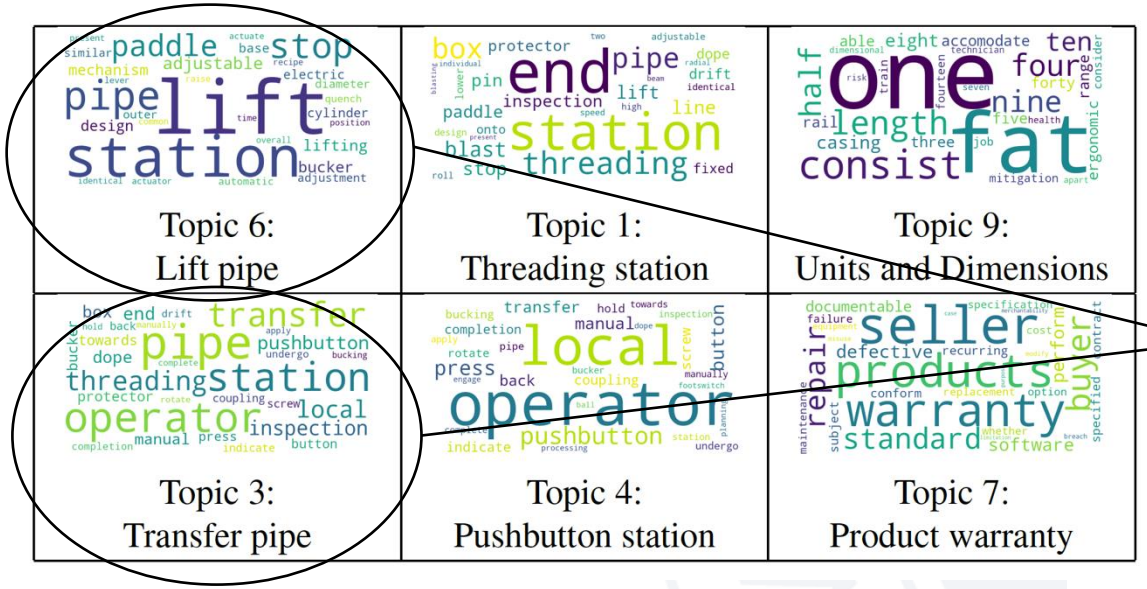
Chen, C., Mullis, J., & Morkos, B. (2021, August). A topic modeling approach to study design requirements. In International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (Vol. 85383, p. V03AT03A021). American Society of Mechanical Engineers.

Selected Results:

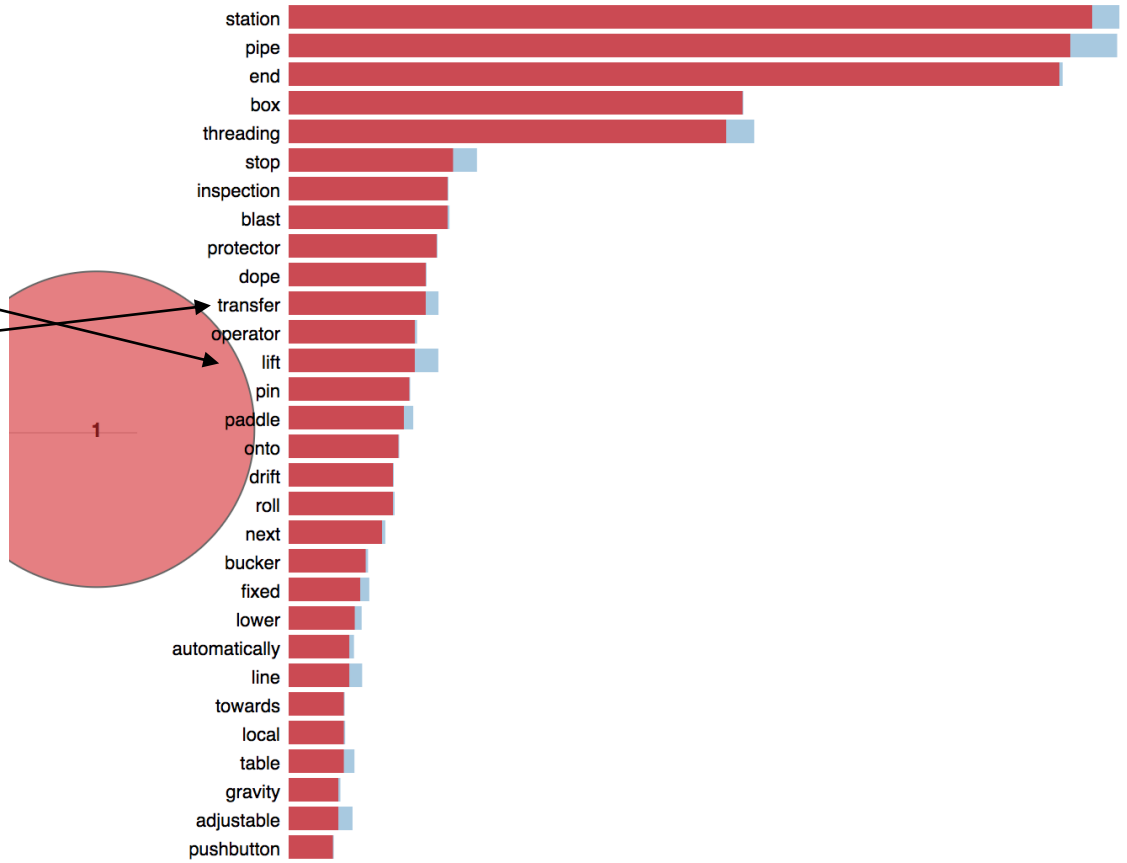


Selected Results:

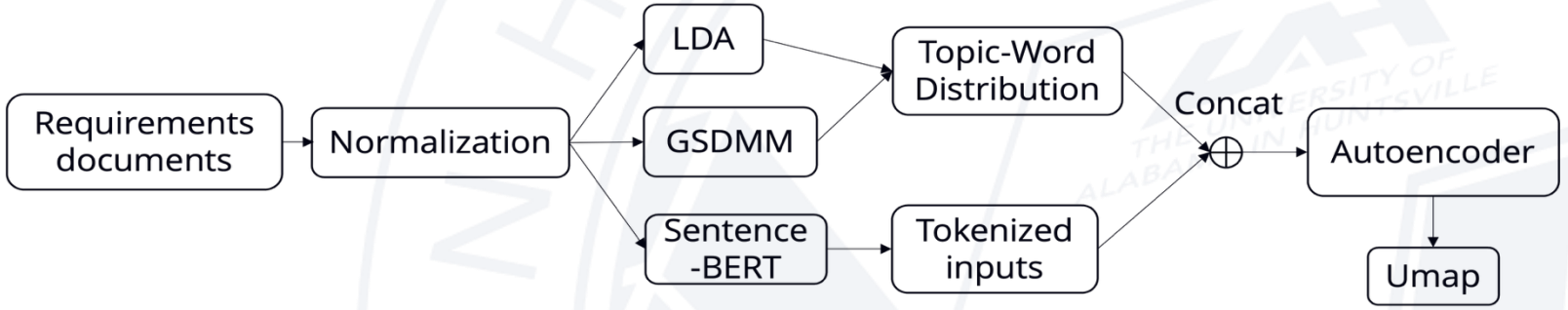
Samples of generated topics
(15 topics)



Topic 1
(10 topics)



Methodology: Data Pipeline Framework



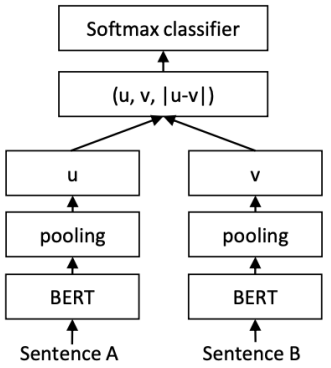
- Latent Dirichlet Allocation (LDA) (Blei et al., 2003)

$$p(\mathcal{D}|\alpha,\beta) = \prod_{d=1}^{M} \int p(\theta_d|\alpha) \left(\prod_{n=1}^{N_d} \sum_{z_{dn}} p(z_{dn}|\theta_d) p(w_{dn}|z_{dn},\beta) \right) d\theta_d$$

- Gibbs sampling algorithm for the Dirichlet multinomial mixture (GSDMM) (Yin, et al., 2014)

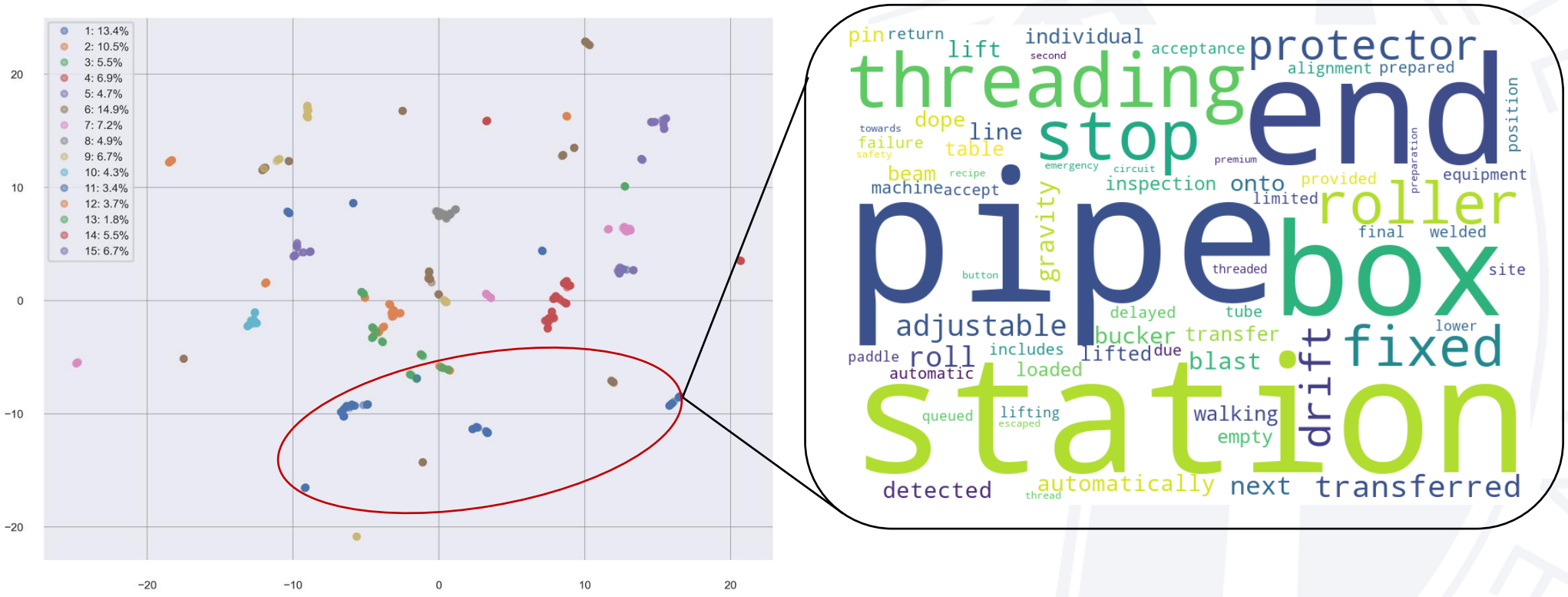
$$p(\mathcal{D}) = \sum_{k=1}^{K} \prod_{w \in d} p(w|z=k) p(z=k)$$

- Sentence-BERT (Reimers, et al., 2014)



Chen, C., & Morkos, B. (2023). Exploring topic modelling for generalising design requirements in complex design. Journal of Engineering Design, 34(11), 922-940.

Selected Results:



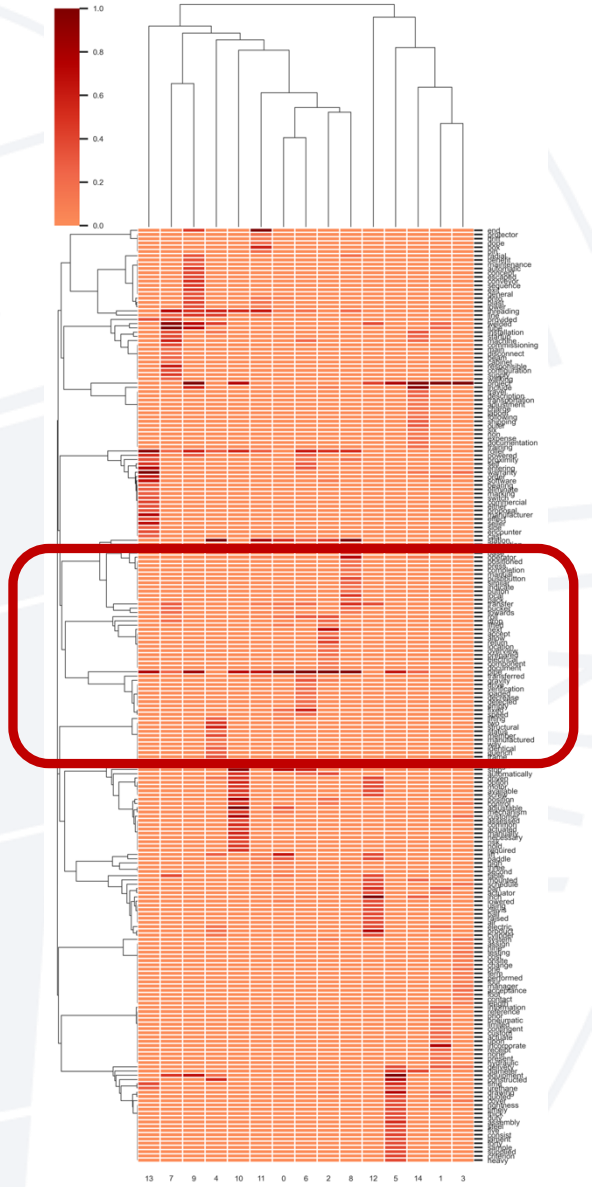
Topics and words distribution in subspace with a highlighted topic 1 for Project 1

Chen, C., & Morkos, B. (2023). Exploring topic modelling for generalising design requirements in complex design. Journal of Engineering Design, 34(11), 922-940.

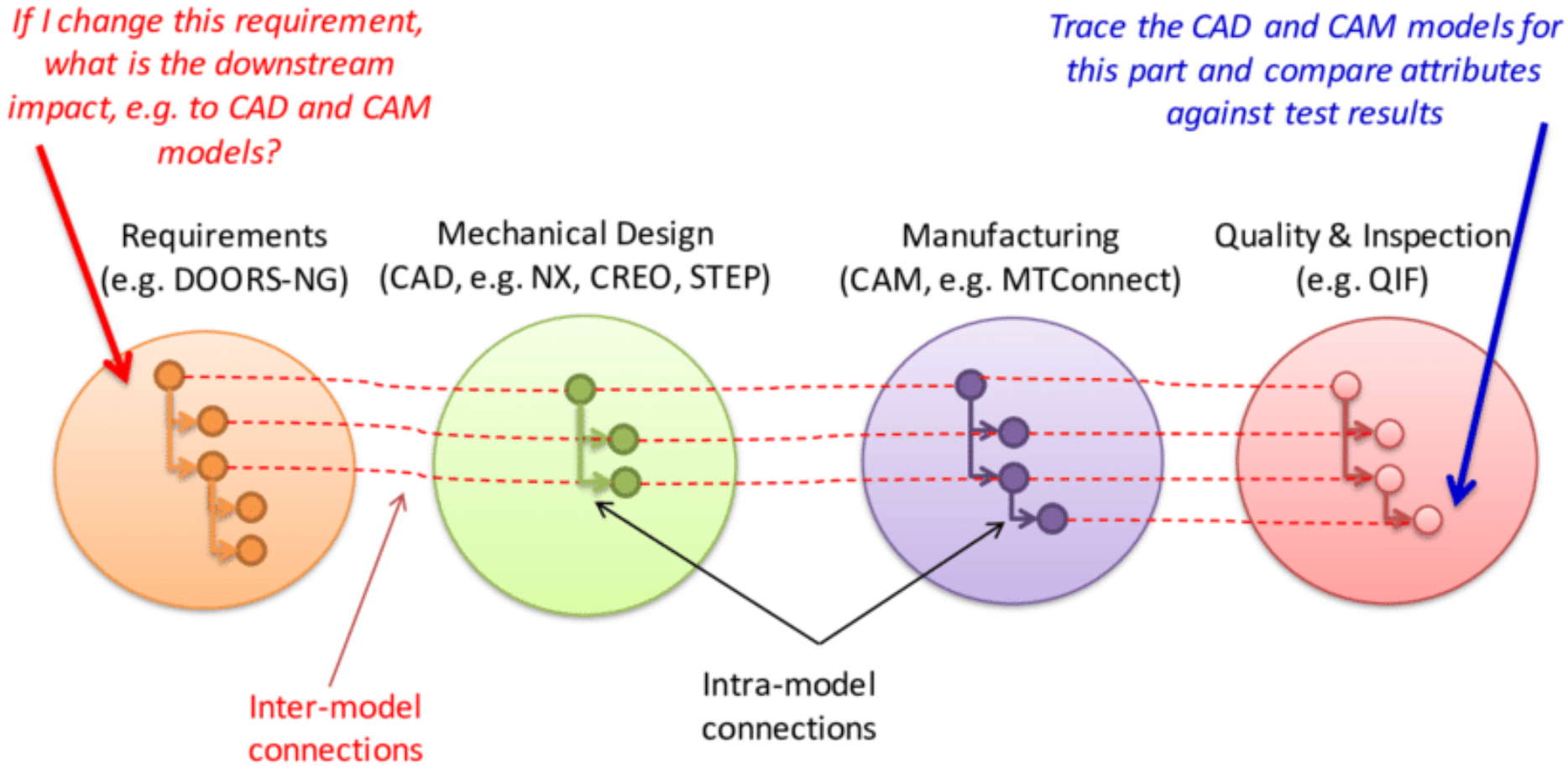
Selected Results:



	Project 1		Project 2		Project 3	
	Coherence	Silhouette	Coherence	Silhouette	Coherence	Silhouette
LDA	0.4124	-	0.4156	-	0.4001	-
GSDMM	0.4751	-	0.5346	-	0.3856	-
LDA_BERT	0.5000	0.2881	**0.5579**	0.3730	**0.4272**	**0.3812**
GSDMM_BERT	**0.5480**	**0.3987**	0.5327	**0.3766**	0.3716	0.3538



Digital Threads:



Bajaj, M., & Hedberg Jr, T. (2018, July). System lifecycle handler—spinning a digital thread for manufacturing. In INCOSE International Symposium (Vol. 28, No. 1, pp. 1636-1650).

Current Research Direction:

